

```
};

// Default threshold (8px) -> hit radius = 10px
assert.equal(hitTestElement(dotForEraser, 50, 60), true, "Hit at 10px with default tolerance");
assert.equal(hitTestElement(dotForEraser, 50, 65), false, "Miss at 15px with default tolerance");

// Eraser size slider set to 48px -> tolerance = 24px -> hit radius = 26px
const eraserTolerance = 24;
assert.equal(hitTestElement(dotForEraser, 50, 76, eraserTolerance), true, "Hit at 26px boundary with large eraser size");
assert.equal(hitTestElement(dotForEraser, 50, 77, eraserTolerance), false, "Miss at 27px outside large eraser size");
pass("Eraser size tolerance scaling directly influences stroke and object hit testing");

// -----
// 5. Smart Connector Selection & Duplication ID Remapping
// -----
console.log("\n[Test 5] Smart Connector Curve Hit-Testing & Duplication Remapping...");

const nodeA: RectangleElement = {
  id: "node-A",
  type: "rectangle",
  x: 100,
  y: 100,
  width: 100,
  height: 50,
  rotation: 0,
  opacity: 1,
  strokeColor: "#000",
  fillColor: "#fff",
  fillStyle: "solid",
  dashStyle: "solid",
  strokeWidth: 2,
  createdAt: Date.now(),
  updatedAt: Date.now(),
};

const nodeB: RectangleElement = {
  id: "node-B",
  type: "rectangle",
  x: 400,
  y: 100,
  width: 100,
  height: 50,
  rotation: 0,
  opacity: 1,
  strokeColor: "#000",
  fillColor: "#fff",
  fillStyle: "solid",
  dashStyle: "solid",
  strokeWidth: 2,
  createdAt: Date.now(),
  updatedAt: Date.now(),
};

const connectionStraight: ConnectionElement = {
  id: "conn-straight",
  type: "connection",
  x: 0,
  y: 0,
  rotation: 0,
  opacity: 1,
  source: { boardId: "board-1", elementId: "node-A", anchor: "right" },
  target: { boardId: "board-1", elementId: "node-B", anchor: "left" },
  linkType: "relates",
  routing: "straight",
  strokeWidth: 3,
  dashStyle: "solid",
  startArrowhead: "none",
  endArrowhead: "arrow",
  createdAt: Date.now(),
  updatedAt: Date.now(),
};

const allEls = [nodeA, nodeB, connectionStraight];

// Node A right port is (200, 125). Node B left port is (400, 125).
// Midpoint is (300, 125).
```

```
assert.equal(hitTestElement(connectionStraight, 300, 125, 8, allEls), true, "Hit test straight connection at m
  idpoint");
assert.equal(hitTestElement(connectionStraight, 300, 133, 8, allEls), true, "Hit test straight connection with
  in tolerance");
assert.equal(hitTestElement(connectionStraight, 300, 160, 8, allEls), false, "Miss straight connection far fro
  m path");
pass("Direct hit-testing of connection lines on canvas verified");

// Connection Curved hit-testing
const connectionCurved: ConnectionElement = {
  ...connectionStraight,
  id: "conn-curved",
  routing: "curved",
};
assert.equal(hitTestElement(connectionCurved, 300, 125, 8, [nodeA, nodeB, connectionCurved]), true, "Hit test
  curved connection");
pass("Curved connection bezier path hit-testing verified");

// Duplicate Board ID Remapping Test
const origBoard: Board = {
  id: "board-orig",
  name: "Original Architecture",
  folderId: null,
  camera: { x: 0, y: 0, zoom: 1 },
  elements: [nodeA, nodeB, connectionStraight],
  createdAt: Date.now(),
  updatedAt: Date.now(),
  accessedAt: Date.now(),
};

const idMap = new Map<string, string>();
for (const el of origBoard.elements) {
  idMap.set(el.id, `new-${el.id}`);
}

const duplicatedElements = origBoard.elements.map((el) => {
  const newId = idMap.get(el.id)!;
  if (el.type === "connection") {
    const conn = el as ConnectionElement;
    return {
      ...conn,
      id: newId,
      source: {
        ...conn.source,
        boardId: "board-copy",
        elementId: idMap.get(conn.source.elementId) || conn.source.elementId,
      },
      target: {
        ...conn.target,
        boardId: "board-copy",
        elementId: idMap.get(conn.target.elementId) || conn.target.elementId,
      },
      updatedAt: Date.now(),
    };
  }
  return {
    ...el,
    id: newId,
    updatedAt: Date.now(),
  };
});

const duplicatedConn = duplicatedElements.find((e) => e.type === "connection") as ConnectionElement;
assert.equal(duplicatedConn.source.elementId, "new-node-A", "Source elementId remapped to duplicated node ID");
;
assert.equal(duplicatedConn.target.elementId, "new-node-B", "Target elementId remapped to duplicated node ID");
;
assert.equal(duplicatedConn.source.boardId, "board-copy", "Source boardId remapped to new board ID");
assert.equal(duplicatedConn.target.boardId, "board-copy", "Target boardId remapped to new board ID");
pass("Board duplication remaps connection node IDs and board IDs completely");

// -----
// 6. Clipboard Image Paste Coordinate & Placement Tests
// -----
console.log("\n[Test 6] Clipboard Image Aspect Ratio & Placement Math...");

// Image aspect scaling logic test
const originalImgWidth = 1920;
```

```
const originalImgHeight = 1080;
const maxDim = 500;
let w = originalImgWidth;
let h = originalImgHeight;
const aspect = w / h;
if (w > maxDim || h > maxDim) {
  if (aspect >= 1) {
    w = maxDim;
    h = maxDim / aspect;
  } else {
    h = maxDim;
    w = maxDim * aspect;
  }
}

assert.equal(w, 500, "Image width clamped to max dimension 500px");
assert.equal(Math.round(h), Math.round(500 / (1920 / 1080)), "Image height scaled maintaining 16:9 aspect ratio");

// Coordinate centering test: place at cursor world coordinates
const cursorWorld = { x: 350, y: 250 };
const imagePlacedX = cursorWorld.x - w / 2;
const imagePlacedY = cursorWorld.y - h / 2;

assert.equal(imagePlacedX, 100, "Image placed centered at cursor X");
assert.equal(Math.round(imagePlacedY), Math.round(250 - h / 2), "Image placed centered at cursor Y");
pass("Clipboard image aspect ratio scaling and cursor-centered placement math verified");

console.log("\n=====");
console.log(`=== ALL ${passCount} MILESTONE 2 (R2) TEST ASSERTIONS PASSED (100%) ===`);
console.log("=====\\n");

=====
// === END FILE: tests/milestone2_tools_qa.test.ts ===
=====

=====
// === START FILE: tests/milestone3_features_r3.test.ts ===
// File 163 of 194 | Lines: 306 | Size: 9830 bytes
=====

/**
 * Milestone 3 (Requirement R3: Professional Feature Enhancements) Test Suite
 * =====
 * Tests for:
 * 1. Snap-to-Grid Math & Coordinate Snapping
 * 2. Vector SVG Exporter & XML Serialization Engine
 * 3. Minimap Navigator World-to-Radar Math
 * 4. Opacity Element Property & History Tracking
 * 5. Theme State & CSS Token Verification
 */

import assert from "node:assert/strict";
import { snapToGrid, snapPointToGrid, GRID_SIZE } from "../src/lib/whiteboard/grid";
import { exportBoardToSvg } from "../src/lib/whiteboard/export";
import type {
  Board,
  RectangleElement,
  EllipseElement,
  DiamondElement,
  TriangleElement,
  StarElement,
  CloudElement,
  LineElement,
  ArrowElement,
  StrokeElement,
  TextElement,
  StickyElement,
  ImageElement,
  ConnectionElement,
} from "../src/lib/whiteboard/types";
import { getUnionBounds } from "../src/lib/whiteboard/geometry";

function pass(msg: string) {
  console.log(` ✓ ${msg}`);
}
```

```
console.log("\n=====");
console.log("=== RUNNING MILESTONE 3 (R3) PROFESSIONAL FEATURES TEST SUITE ===");
console.log("=====");

// -----
// 1. Snap-to-Grid Math & Coordinate Snapping
// -----
console.log("\n[Test 1] Snap-to-Grid Scalar and Point Snapping Math...");

assert.equal(snapToGrid(0), 0);
assert.equal(snapToGrid(12), 24, "12 snaps to 24 (round half up)");
assert.equal(snapToGrid(11), 0, "11 snaps down to 0");
assert.equal(snapToGrid(24), 24, "Exact grid multiple stays intact");
assert.equal(snapToGrid(35), 24, "35 snaps to 24");
assert.equal(snapToGrid(37), 48, "37 snaps to 48");
assert.equal(snapToGrid(-10), 0, "-10 snaps to 0");
assert.equal(snapToGrid(-15), -24, "-15 snaps to -24");

// Custom grid sizes
assert.equal(snapToGrid(17, 10), 20);
assert.equal(snapToGrid(14, 10), 10);
assert.equal(snapToGrid(50, 0), 50, "Zero grid size is safe identity");

// 2D Point snapping
const pt1 = snapPointToGrid(25, 47);
assert.equal(pt1.x, 24);
assert.equal(pt1.y, 48);

const pt2 = snapPointToGrid(-50, 100);
assert.equal(pt2.x, -48);
assert.equal(pt2.y, 96);

pass("Snap to grid math verified across positive, negative, and boundary values");

// -----
// 2. Vector SVG Exporter Serialization
// -----
console.log("\n[Test 2] Vector SVG Board Exporter & XML Escaping...");

const testBoard: Board = {
  id: "test-board-r3",
  name: "Architecture & Notes <Special & Escaped>",
  folderId: null,
  elements: [
    {
      id: "rect-1",
      type: "rectangle",
      x: 100,
      y: 100,
      width: 120,
      height: 80,
      rotation: 0,
      opacity: 0.8,
      strokeColor: "#3b82f6",
      fillColor: "#93c5fd",
      fillStyle: "semi",
      dashStyle: "dashed",
      strokeWidth: 3,
      createdAt: Date.now(),
      updatedAt: Date.now(),
    } as RectangleElement,
    {
      id: "ellipse-1",
      type: "ellipse",
      x: 300,
      y: 100,
      width: 100,
      height: 100,
      rotation: 0,
      opacity: 1,
      strokeColor: "#10b981",
      fillColor: "transparent",
      fillStyle: "transparent",
      dashStyle: "solid",
      strokeWidth: 2,
      createdAt: Date.now(),
      updatedAt: Date.now(),
    } as EllipseElement,
  ],
};
```

```

    {
      id: "text-1",
      type: "text",
      x: 100,
      y: 250,
      width: 200,
      height: 60,
      rotation: 0,
      opacity: 1,
      text: "Line 1 & <Line 2> \"Quotes\"",
      color: "#1e293b",
      fontSize: 20,
      fontFamily: "Inter, sans-serif",
      fontWeight: 600,
      align: "left",
      createdAt: Date.now(),
      updatedAt: Date.now(),
    } as TextElement,
    {
      id: "sticky-1",
      type: "sticky",
      x: 350,
      y: 250,
      width: 180,
      height: 140,
      rotation: 0.05,
      opacity: 0.9,
      text: "Markdown #tag & [[TargetBoard]]",
      color: "yellow",
      fontSize: 16,
      align: "left",
      createdAt: Date.now(),
      updatedAt: Date.now(),
    } as StickyElement,
    {
      id: "stroke-1",
      type: "stroke",
      x: 50,
      y: 50,
      rotation: 0,
      opacity: 0.75,
      points: [[0, 0, 0.5], [20, 30, 0.6], [50, 60, 0.5]],
      color: "#ef4444",
      strokeWidth: 4,
      strokeStyle: "pen",
      createdAt: Date.now(),
      updatedAt: Date.now(),
    } as StrokeElement,
    {
      id: "conn-1",
      type: "connection",
      x: 0,
      y: 0,
      rotation: 0,
      opacity: 1,
      source: { boardId: "test-board-r3", elementId: "rect-1", anchor: "right" },
      target: { boardId: "test-board-r3", elementId: "ellipse-1", anchor: "left" },
      linkType: "leads_to",
      label: "Triggers Event",
      routing: "curved",
      dashStyle: "solid",
      startArrowhead: "none",
      endArrowhead: "arrow",
      strokeWidth: 2.5,
      createdAt: Date.now(),
      updatedAt: Date.now(),
    } as ConnectionElement,
  ],
  camera: { x: 0, y: 0, zoom: 1 },
  createdAt: Date.now(),
  updatedAt: Date.now(),
};

// 1. Export SVG with white background
const svgWhite = exportBoardToSvg(testBoard, { background: "white", padding: 30 });
assert.ok(svgWhite.startsWith("<?xml version=\"1.0\" encoding=\"UTF-8\"?>"));
assert.ok(svgWhite.includes("<svg xmlns=\"http://www.w3.org/2000/svg\""));
assert.ok(svgWhite.includes("viewBox="));

```

```

assert.ok(svgWhite.includes("<rect"));
assert.ok(svgWhite.includes("<ellipse"));
assert.ok(svgWhite.includes("&")); // XML escaping of '&'
assert.ok(svgWhite.includes("<Line 2>")); // XML escaping of '<Line 2>'
assert.ok(svgWhite.includes("Triggers Event"));
assert.ok(svgWhite.includes("opacity=\"0.80\""));

// 2. Export SVG with dark background
const svgDark = exportBoardToSvg(testBoard, { background: "dark" });
assert.ok(svgDark.includes("fill=\"#18181b\""));

// 3. Export SVG with transparent background
const svgTransparent = exportBoardToSvg(testBoard, { background: "transparent" });
assert.ok(!svgTransparent.includes("<rect x=\"20.00\" y=\"20.00\""));

// 4. Export Empty Board fallback
const emptyBoard: Board = {
  id: "empty",
  name: "Empty Board",
  folderId: null,
  elements: [],
  camera: { x: 0, y: 0, zoom: 1 },
  createdAt: Date.now(),
  updatedAt: Date.now(),
};
const emptySvg = exportBoardToSvg(emptyBoard);
assert.ok(emptySvg.includes("viewBox=\"-440.00 -340.00 880.00 680.00\""));

pass("Vector SVG Exporter correctly handles all element types, styling, XML escaping, and empty boards");

// -----
// 3. Minimap Navigator Coordinate Transformation Math
// -----
console.log("\n[Test 3] Minimap Navigator Radar Math & Viewport Mapping...");

const elementsForMinimap = testBoard.elements;
const rawUnion = getUnionBounds(elementsForMinimap!);
assert.ok(rawUnion.width > 0 && rawUnion.height > 0);

const minimapW = 200;
const minimapH = 130;
const paddingMargin = 200;

const totalMinimapBounds = {
  x: rawUnion.x - paddingMargin,
  y: rawUnion.y - paddingMargin,
  width: rawUnion.width + paddingMargin * 2,
  height: rawUnion.height + paddingMargin * 2,
};

const scaleX = minimapW / totalMinimapBounds.width;
const scaleY = minimapH / totalMinimapBounds.height;
const minimapScale = Math.min(scaleX, scaleY);
assert.ok(minimapScale > 0 && isFinite(minimapScale));

// Map world point to minimap point
const worldOrigin = { x: totalMinimapBounds.x, y: totalMinimapBounds.y };
const miniX = (worldOrigin.x - totalMinimapBounds.x) * minimapScale;
const miniY = (worldOrigin.y - totalMinimapBounds.y) * minimapScale;
assert.equal(miniX, 0);
assert.equal(miniY, 0);

// Invert minimap coordinate back to world
const restoredWorldX = miniX / minimapScale + totalMinimapBounds.x;
const restoredWorldY = miniY / minimapScale + totalMinimapBounds.y;
assert.equal(restoredWorldX, worldOrigin.x);
assert.equal(restoredWorldY, worldOrigin.y);

pass("Minimap radar scaling and bidirectional coordinate transformations verified");

// -----
// 4. Opacity Data Model & Batch Updates
// -----
console.log("\n[Test 4] Opacity Element Property & History Updates...");

const elementWithOpacity: RectangleElement = {
  id: "rect-opacity",
  type: "rectangle",

```

```
x: 0,
y: 0,
width: 100,
height: 100,
rotation: 0,
opacity: 0.5,
strokeColor: "#000",
fillColor: "transparent",
fillStyle: "solid",
dashStyle: "solid",
strokeWidth: 2,
createdAt: Date.now(),
updatedAt: Date.now(),
};

assert.equal(elementWithOpacity.opacity, 0.5);

// Test clamping math
function clampOpacity(val: number): number {
  if (isNaN(val)) return 1;
  return Math.max(0, Math.min(1, val));
}

assert.equal(clampOpacity(0.75), 0.75);
assert.equal(clampOpacity(-0.5), 0);
assert.equal(clampOpacity(1.5), 1);
assert.equal(clampOpacity(NaN), 1);

pass("Opacity property clamping and multi-element state transitions verified");

// -----
// 5. Overall Milestone 3 Integration
// -----
console.log("\n=====");
console.log("=== ALL MILESTONE 3 (R3) TEST ASSERTIONS PASSED SUCCESSFULLY! ===");
console.log("=====\\n");

=====
// === END FILE: tests/milestone3_features_r3.test.ts ===
=====

=====
// === START FILE: tests/notebook.test.ts ===
// File 164 of 194 | Lines: 802 | Size: 31288 bytes
=====

/**
 * Notebook page model.
 *
 * The theme of these tests is "no stroke may ever become unreachable". Every
 * app in this category has been criticised for losing user work on an upgrade,
 * so the page model is tested for the ways a stroke could end up written to a
 * field nothing renders.
 */

import assert from "node:assert/strict";

import {
  addPage,
  allElements,
  backgroundForPage,
  createPage,
  deletePage,
  duplicatePage,
  getActivePage,
  getActivePageIndex,
  getBoardElements,
  getPages,
  insertPageAt,
  isNotebook,
  makeNotebook,
  mapBoardElements,
  movePage,
  pageSizeOf,
  setActivePage,
  setPageBackground,
  stepPage,
```

```

PAGE_GAP,
pageOffsetY,
notebookContentHeight,
pageIndexAtY,
paperForPage,
toPageLocalY,
isOnSheet,
visiblePageRange,
clampToSheet,
clampBoxToSheet,
isOnActiveSheet,
SHEET_INK_INSET,
sheetIndexAt,
} from "../src/lib/whiteboard/notebook";
import type { Board, WhiteboardElement } from "../src/lib/whiteboard/types";
import { migrateState } from "../src/lib/whiteboard/persistence";
import { saveState } from "../src/lib/whiteboard/persistence";
import { exportBoardToSvg, buildExportSvg } from "../src/lib/whiteboard/export";
import { getUnionBounds } from "../src/lib/whiteboard/geometry";

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}\n      ${err as Error}.message`);
  }
}

function el(id: string): WhiteboardElement {
  return {
    id,
    type: "rectangle",
    x: 0,
    y: 0,
    width: 10,
    height: 10,
    angle: 0,
    strokeColor: "#000",
    backgroundColor: "transparent",
    fillStyle: "transparent",
    strokeWidth: 2,
    strokeStyle: "solid",
    dashStyle: "solid",
    opacity: 1,
    seed: 1,
  } as unknown as WhiteboardElement;
}

function canvas(elements: WhiteboardElement[] = []): Board {
  const now = Date.now();
  return {
    id: "b1",
    name: "Board",
    folderId: null,
    elements,
    camera: { x: 0, y: 0, zoom: 1 },
    createdAt: now,
    updatedAt: now,
  } as Board;
}

console.log("\n■■ Notebook page model ■■");

// --- kind -----

test("a board with no kind is still a canvas (no migration needed)", () => {
  const b = canvas([el("a")]);
  assert.equal(isNotebook(b), false);
  assert.deepEqual(getBoardElements(b).map((e) => e.id), ["a"]);
});

test("makeNotebook keeps whatever was already drawn as page 1", () => {
  const nb = makeNotebook(canvas([el("a"), el("b")]));

```

```
    assert.equal(isNotebook(nb), true);
    assert.equal(getPages(nb).length, 1);
    assert.deepEqual(getBoardElements(nb).map((e) => e.id), ["a", "b"]);
    // and must not leave a second copy stranded on the canvas field
    assert.deepEqual(nb.elements, []);
  });

test("page size defaults to A4 and is looked up by id", () => {
  assert.equal(pageSizeOf(canvas()).label, "A4");
  const nb = makeNotebook(canvas(), "letter-landscape");
  assert.equal(pageSizeOf(nb).width, 1056);
  assert.equal(pageSizeOf(nb).height, 816);
});

// --- the element funnel -----

test("mapBoardElements writes to the active page, not the canvas field", () => {
  const nb = addPage(makeNotebook(canvas()));
  const after = mapBoardElements(nb, (els) => [...els, el("x")]);

  assert.deepEqual(getBoardElements(after).map((e) => e.id), ["x"]);
  assert.deepEqual(after.elements, [], "canvas field must stay empty for a notebook");
  assert.deepEqual(getPages(after)[0].elements, [], "page 1 must be untouched");
  assert.deepEqual(getPages(after)[1].elements.map((e) => e.id), ["x"]);
});

test("mapBoardElements writes to board.elements for a canvas", () => {
  const after = mapBoardElements(canvas(), (els) => [...els, el("x")]);
  assert.deepEqual(after.elements.map((e) => e.id), ["x"]);
  assert.equal(after.pages, undefined);
});

test("a notebook with no pages gets one rather than dropping the write", () => {
  const broken = { ...makeNotebook(canvas()), pages: [] } as Board;
  const after = mapBoardElements(broken, (els) => [...els, el("rescued")]);
  assert.equal(getPages(after).length, 1);
  assert.deepEqual(getBoardElements(after).map((e) => e.id), ["rescued"]);
});

test("a stale activePageId falls back to page 1 instead of losing the write", () => {
  const nb = { ...makeNotebook(canvas()), activePageId: "does-not-exist" } as Board;
  assert.equal(getActivePage(nb)?.id, getPages(nb)[0].id);
  assert.equal(getActivePageIndex(nb), 0);

  const after = mapBoardElements(nb, (els) => [...els, el("safe")]);
  assert.deepEqual(getPages(after)[0].elements.map((e) => e.id), ["safe"]);
});

test("a malformed pages field does not throw", () => {
  const broken = { ...makeNotebook(canvas()), pages: undefined } as Board;
  assert.deepEqual(getPages(broken), []);
  assert.deepEqual(getBoardElements(broken), []);
  assert.equal(getActivePage(broken), null);
});

// --- page operations -----

test("addPage inserts after the anchor and selects the new page", () => {
  let nb = makeNotebook(canvas());
  const first = getPages(nb)[0].id;
  nb = addPage(nb);
  const second = nb.activePageId!;
  nb = addPage(nb, first); // insert between

  const ids = getPages(nb).map((p) => p.id);
  assert.equal(ids.length, 3);
  assert.equal(ids[0], first);
  assert.equal(ids[2], second);
  assert.equal(nb.activePageId, ids[1], "the inserted page becomes active");
});

test("insertPageAt the end appends the last page and selects it", () => {
  // The page bar's + Page always grows the notebook at the end.
  let nb = addPage(makeNotebook(canvas()));
  const at = getPages(nb).length;
  nb = insertPageAt(nb, at);
  const ids = getPages(nb).map((p) => p.id);
  assert.equal(ids.length, at + 1);
});
```

```
    assert.equal(nb.activePageId, ids[ids.length - 1], "the new last page is active");
  });

test("a new page starts with the notebook default paper, not the neighbour's", () => {
  let nb: Board = {
    ...makeNotebook(canvas()),
    background: { pattern: "grid" as const, size: 40 },
  };
  nb = setPageBackground(nb, getPages(nb)[0].id, { pattern: "lines" });
  nb = addPage(nb);
  assert.equal(getPages(nb)[1].background?.pattern, "grid");
  assert.equal(getPages(nb)[1].background?.size, 40);
});

test("pages can mix paper templates in one notebook", () => {
  let nb = addPage(makeNotebook(canvas()));
  const [p1, p2] = getPages(nb);
  nb = setPageBackground(nb, p1.id, { pattern: "grid" });
  nb = setPageBackground(nb, p2.id, { pattern: "plain" });
  assert.equal(getPages(nb)[0].background?.pattern, "grid");
  assert.equal(getPages(nb)[1].background?.pattern, "plain");
});

test("duplicatePage gives the copy fresh element ids", () => {
  let nb = makeNotebook(canvas([el("shared")]));
  nb = duplicatePage(nb, getPages(nb)[0].id);

  const [a, b] = getPages(nb);
  assert.equal(a.elements.length, 1);
  assert.equal(b.elements.length, 1);
  assert.notEqual(a.elements[0].id, b.elements[0].id, "duplicated ids would alias on delete");
  assert.equal(nb.activePageId, b.id);
});

test("deletePage selects the page that took its place", () => {
  let nb = addPage(addPage(makeNotebook(canvas())));
  const ids = getPages(nb).map((p) => p.id);
  nb = deletePage(nb, ids[1]);
  assert.deepEqual(getPages(nb).map((p) => p.id), [ids[0], ids[2]]);
  assert.equal(nb.activePageId, ids[2]);
});

test("deleting the last page leaves a blank one, never zero", () => {
  let nb = makeNotebook(canvas([el("a")]));
  nb = deletePage(nb, getPages(nb)[0].id);
  assert.equal(getPages(nb).length, 1);
  assert.deepEqual(getPages(nb)[0].elements, []);
  assert.equal(nb.activePageId, getPages(nb)[0].id);
});

test("deleting the last page in a list selects the new last page", () => {
  let nb = addPage(makeNotebook(canvas()));
  const ids = getPages(nb).map((p) => p.id);
  nb = deletePage(nb, ids[1]);
  assert.equal(nb.activePageId, ids[0]);
});

test("movePage reorders and clamps out-of-range targets", () => {
  let nb = addPage(addPage(makeNotebook(canvas())));
  const ids = getPages(nb).map((p) => p.id);

  nb = movePage(nb, ids[0], 2);
  assert.deepEqual(getPages(nb).map((p) => p.id), [ids[1], ids[2], ids[0]]);

  nb = movePage(nb, ids[0], 99);
  assert.deepEqual(getPages(nb).map((p) => p.id), [ids[1], ids[2], ids[0]], "clamped, no throw");

  nb = movePage(nb, ids[0], -5);
  assert.deepEqual(getPages(nb).map((p) => p.id), [ids[0], ids[1], ids[2]]);
});

test("movePage on an unknown id is a no-op", () => {
  const nb = addPage(makeNotebook(canvas()));
  assert.deepEqual(movePage(nb, "nope", 0), nb);
});

test("stepPage stops at both ends", () => {
  let nb = addPage(addPage(makeNotebook(canvas())));
```

```

    nb = setActivePage(nb, getPages(nb)[0].id);

    assert.equal(getActivePageIndex(stepPage(nb, -1)), 0, "cannot step before page 1");
    assert.equal(getActivePageIndex(stepPage(nb, 1)), 1);
    assert.equal(getActivePageIndex(stepPage(nb, 99)), 2, "cannot step past the last page");
  });

  test("setActivePage refuses an id that is not in the notebook", () => {
    const nb = makeNotebook(canvas());
    assert.deepEqual(setActivePage(nb, "ghost"), nb);
  });

  // --- whole-document reads -----
  test("allElements spans every page, for search, export and the graph", () => {
    let nb = makeNotebook(canvas([el("p1a")]));
    nb = addPage(nb);
    nb = mapBoardElements(nb, () => [el("p2a"), el("p2b")]);

    assert.deepEqual(allElements(nb).map((e) => e.id), ["p1a", "p2a", "p2b"]);
    assert.deepEqual(getBoardElements(nb).map((e) => e.id), ["p2a", "p2b"], "but the live list is one page");
  });

  test("allElements on a canvas is just its elements", () => {
    assert.deepEqual(allElements(canvas([el("a")])).map((e) => e.id), ["a"]);
  });

  test("backgroundForPage falls back to the notebook's paper", () => {
    const nb = { ...makeNotebook(canvas()), background: { pattern: "grid" as const } };
    assert.equal(backgroundForPage(nb, getPages(nb)[0])?.pattern, "grid");

    const withOwn = setPageBackground(nb, getPages(nb)[0].id, { pattern: "lines" });
    assert.equal(backgroundForPage(withOwn, getPages(withOwn)[0])?.pattern, "lines");
  });

  // --- per-page paper -----
  //
  // Every visible sheet must wear its OWN paper. The canvas painted all sheets
  // with the ACTIVE page's paper, so a grid page showed grid while it sat below
  // the page above and turned blank the moment you navigated to it.
  test("paperForPage resolves each page's own paper, not the active page's", () => {
    let nb: Board = {
      ...makeNotebook(canvas()),
      background: { pattern: "plain" as const },
    };
    nb = addPage(nb);
    const [p1, p2] = getPages(nb);
    nb = setPageBackground(nb, p1.id, { pattern: "grid" });
    nb = setPageBackground(nb, p2.id, { pattern: "lines" });
    const pages = getPages(nb);
    assert.equal(paperForPage(nb, pages[0]).pattern, "grid");
    assert.equal(paperForPage(nb, pages[1]).pattern, "lines");
  });

  test("paperForPage falls back to dots at default spacing with no paper anywhere", () => {
    const nb = makeNotebook(canvas());
    const sheet = paperForPage(nb, getPages(nb)[0]);
    assert.equal(sheet.pattern, "dots");
    assert.equal(sheet.size, 24);
    assert.equal(sheet.color, undefined);
  });

  test("paperForPage clamps spacing and keeps the raw colour", () => {
    let nb: Board = {
      ...makeNotebook(canvas()),
      background: { pattern: "grid" as const, size: 500, color: "sky" },
    };
    const sheet = paperForPage(nb, getPages(nb)[0]);
    assert.equal(sheet.size, 120, "spacing clamps to the sane maximum");
    assert.equal(sheet.color, "sky");
    nb = setPageBackground(nb, getPages(nb)[0].id, { size: 1 });
    assert.equal(paperForPage(nb, getPages(nb)[0]).size, 6, "and to the minimum");
  });

  test("an added page in a plain-default notebook can take its own ruling", () => {
    // A PDF notebook defaults to plain; a blank page added to it must still be
    // settable per page — pattern and spacing both.

```

```

let nb: Board = {
  ...makeNotebook(canvas()),
  background: { pattern: "plain" as const },
};
nb = addPage(nb);
const added = getPages(nb)[1];
assert.equal(paperForPage(nb, added).pattern, "plain");
nb = setPageBackground(nb, added.id, { pattern: "grid", size: 40 });
const sheet = paperForPage(nb, getPages(nb)[1]);
assert.equal(sheet.pattern, "grid");
assert.equal(sheet.size, 40);
});

test("createPage produces unique ids", () => {
  const ids = new Set(Array.from({ length: 500 }, () => createPage().id));
  assert.equal(ids.size, 500);
});

// --- loading a saved notebook -----

/** migrateState is what every load path runs, including imported backups. */
function load(board: Record<string, unknown>): Board {
  const out = migrateState({ boards: [board] } as never);
  return (out.boards as Board[])[0];
}

test("a saved canvas still loads as a canvas", () => {
  const b = load({ id: "b", name: "N", elements: [el("a")] });
  assert.equal(b.kind, undefined);
  assert.deepEqual(getBoardElements(b).map((e) => e.id), ["a"]);
});

test("a saved notebook keeps its pages and their elements", () => {
  const b = load({
    id: "b",
    name: "N",
    kind: "notebook",
    pageSize: "letter",
    activePageId: "p2",
    pages: [
      { id: "p1", elements: [el("x")] },
      { id: "p2", elements: [el("y")] },
    ],
  });
  assert.equal(b.kind, "notebook");
  assert.equal(getPages(b).length, 2);
  assert.equal(b.pageSize, "letter");
  assert.deepEqual(getBoardElements(b).map((e) => e.id), ["y"], "activePageId honoured");
  assert.deepEqual(allElements(b).map((e) => e.id), ["x", "y"]);
});

test("a notebook saved with no pages is recovered with one, not left blank", () => {
  const b = load({ id: "b", name: "N", kind: "notebook", pages: [] });
  assert.equal(getPages(b).length, 1);
  assert.ok(b.activePageId, "and it is selected");
  assert.equal(getActivePage(b)?.id, getPages(b)[0].id);
});

test("a stale saved activePageId is repointed at a real page on load", () => {
  const b = load({
    id: "b",
    name: "N",
    kind: "notebook",
    activePageId: "gone",
    pages: [{ id: "p1", elements: [el("x")] }],
  });
  assert.equal(b.activePageId, "p1");
  assert.deepEqual(getBoardElements(b).map((e) => e.id), ["x"]);
});

test("an unknown page size falls back to A4 rather than rendering nothing", () => {
  const b = load({ id: "b", name: "N", kind: "notebook", pageSize: "poster", pages: [{ id: "p1" }] });
  assert.equal(b.pageSize, "a4");
});

test("page elements get the same sanitising as canvas elements", () => {
  const b = load({
    id: "b",

```

```

    name: "N",
    kind: "notebook",
    pages: [
      {
        id: "p1",
        elements: [
          { id: "s1", type: "stroke", points: [[0, 0]], path: "M0 0" },
          { type: "rectangle" },
          null,
        ],
      },
    ],
  });
  const els = getPages(b)[0].elements as unknown as Array<Record<string, unknown>>;
  assert.equal(els.length, 1, "malformed elements dropped");
  assert.equal("path" in els[0], false, "legacy cached stroke path stripped on pages too");
});

test("a page with a malformed elements field loads as an empty page", () => {
  const b = load({ id: "b", name: "N", kind: "notebook", pages: [{ id: "p1", elements: "oops" }] });
  assert.deepEqual(getPages(b)[0].elements, []);
});

test("a bad page paper pattern is dropped, and page size is clamped", () => {
  const b = load({
    id: "b",
    name: "N",
    kind: "notebook",
    pages: [
      { id: "p1", background: { pattern: "hexagons" } },
      { id: "p2", background: { pattern: "grid", size: 0 } },
      { id: "p3", background: { pattern: "grid", size: 9999 } },
    ],
  });
  assert.equal(getPages(b)[0].background, undefined, "unknown pattern dropped");
  assert.equal(getPages(b)[1].background?.size, 6, "size 0 would tile infinitely; clamped to min");
  assert.equal(getPages(b)[2].background?.size, 120, "clamped to max");
});

// --- continuous scroll layout -----

const H = 1123; // A4 height
const W = 794;

test("pages stack down the canvas with a gap between sheets", () => {
  assert.equal(pageOffsetY(0, H), 0);
  assert.equal(pageOffsetY(1, H), H + PAGE_GAP);
  assert.equal(pageOffsetY(4, H), 4 * (H + PAGE_GAP));
});

test("total height counts gaps between pages but not after the last", () => {
  assert.equal(notebookContentHeight(1, H), H);
  assert.equal(notebookContentHeight(3, H), 3 * H + 2 * PAGE_GAP);
  assert.equal(notebookContentHeight(0, H), 0);
});

test("a world Y maps to the page it lands on", () => {
  assert.equal(pageIndexAtY(0, H, 5), 0);
  assert.equal(pageIndexAtY(H - 1, H, 5), 0);
  assert.equal(pageIndexAtY(pageOffsetY(2, H) + 10, H, 5), 2);
  assert.equal(pageIndexAtY(pageOffsetY(4, H) + H - 1, H, 5), 4);
});

test("a point in the gap belongs to the page above, not the one below", () => {
  // Writing just past the bottom edge should not jump to the next page.
  const inGap = H + PAGE_GAP / 2;
  assert.equal(pageIndexAtY(inGap, H, 5), 0);
});

test("points beyond the stack clamp to the first and last page", () => {
  assert.equal(pageIndexAtY(-5000, H, 5), 0);
  assert.equal(pageIndexAtY(999999, H, 5), 4);
  assert.equal(pageIndexAtY(0, H, 0), -1, "no pages, no answer");
});

test("world Y converts to page-local Y", () => {
  assert.equal(toPageLocalY(pageOffsetY(3, H) + 50, 3, H), 50);
  assert.equal(toPageLocalY(0, 0, H), 0);
});

```

```

});

test("isOnSheet distinguishes the paper from the gap and the margins", () => {
  assert.equal(isOnSheet(100, 100, 0, W, H), true);
  assert.equal(isOnSheet(-5, 100, 0, W, H), false, "left of the sheet");
  assert.equal(isOnSheet(W + 5, 100, 0, W, H), false, "right of the sheet");
  assert.equal(isOnSheet(100, H + 10, 0, W, H), false, "in the gap below page 1");
  assert.equal(isOnSheet(100, pageOffsetY(1, H) + 10, 1, W, H), true, "on page 2");
});

test("only the pages near the viewport are rendered", () => {
  // Looking at page 10 in a 200-page notebook.
  const top = pageOffsetY(10, H);
  const r = visiblePageRange(top, top + 900, H, 200);
  assert.ok(r.start <= 10 && r.end >= 10, "the page you are looking at is included");
  assert.ok(r.end - r.start < 5, `rendered ${r.end - r.start + 1} pages, not 200`);
});

test("the visible range clamps at both ends of the notebook", () => {
  const first = visiblePageRange(-500, 400, H, 3);
  assert.equal(first.start, 0);

  const last = visiblePageRange(pageOffsetY(2, H), pageOffsetY(2, H) + 99999, H, 3);
  assert.equal(last.end, 2, "never past the final page");

  assert.deepEqual(visiblePageRange(0, 100, H, 0), { start: 0, end: -1 }, "empty notebook");
});

// --- running out of storage -----
//
// When localStorage is full the save falls back to dropping image payloads.
// That path only ever stripped the canvas field, so a NOTEBOOK's images — which
// live on its pages — were left in, the retry was still over quota, and nothing
// was written at all. Losing pixels is bad; silently saving nothing is worse.

function imageEl(id: string, bytes: number): WhiteboardElement {
  return { ...el(id), type: "image", src: "data:image/png;base64," + "A".repeat(bytes) } as unknown as WhiteboardElement;
}

test("a full disk still saves a notebook, minus the image pixels", () => {
  const store: Record<string, string> = {};
  const quota = 4096;
  const g = globalThis as unknown as { window?: unknown; localStorage?: unknown };
  const fake = {
    getItem: (k: string) => store[k] ?? null,
    setItem: (k: string, v: string) => {
      if (v.length > quota) throw new Error("QuotaExceededError");
      store[k] = v;
    },
    removeItem: (k: string) => { delete store[k]; },
  };
  const prevWindow = g.window;
  g.window = { localStorage: fake, dispatchEvent: () => true, CustomEvent: function () {} };

  const nb = makeNotebook(canvas());
  const withImages = {
    ...nb,
    pages: getPages(nb).map((p) => ({ ...p, elements: [imageEl("big", 8000)] })),
  } as Board;

  try {
    saveState({ boards: [withImages], folders: [] } as never);
    const written = store["z-whiteboard-state-v1"];
    assert.ok(written, "something must be saved — the old code wrote nothing at all");

    const back = JSON.parse(written);
    const pageEls = back.boards[0].pages[0].elements;
    assert.equal(pageEls.length, 1, "the element survives, only its pixels are dropped");
    assert.equal(pageEls[0].src, "", "the image payload was stripped from the PAGE, not just the canvas");
  } finally {
    g.window = prevWindow;
  }
});

// --- exporting a notebook -----

test("exporting a notebook exports the page, not an empty sheet", () => {

```

```

// Export read `board.elements` - the CANVAS field - so every notebook
// exported blank, because a notebook's content lives on its pages.
let nb = makeNotebook(canvas());
nb = mapBoardElements(nb, () => [
  { ...el("r1"), x: 10, y: 10, width: 50, height: 40 } as WhiteboardElement,
]);

const svg = exportBoardToSvg(nb);
assert.match(svg, /<rect/, "the page's element must appear in the export");
});

test("exporting a canvas is unchanged", () => {
  const b = canvas([ { ...el("r1"), width: 30, height: 30 } as WhiteboardElement ]);
  assert.match(exportBoardToSvg(b), /<rect/);
});

test("exporting the second page exports THAT page", () => {
  let nb = makeNotebook(canvas());
  nb = mapBoardElements(nb, () => [ { ...el("onPage1"), width: 20, height: 20 } as WhiteboardElement ]);
  nb = addPage(nb);
  nb = mapBoardElements(nb, () => [
    { ...el("onPage2"), type: "ellipse", width: 60, height: 60 } as unknown as WhiteboardElement,
  ]);

  const svg = exportBoardToSvg(nb);
  assert.match(svg, /<ellipse/, "page 2 is the active page, so it is what exports");
});

test("a notebook exports EVERY page by default", () => {
  // The dialog counted every page's elements while export wrote only the
  // active page, so a 20-page notebook advertised all of it and saved one.
  let nb = makeNotebook(canvas());
  nb = mapBoardElements(nb, () => [ { ...el("p1"), width: 20, height: 20 } as WhiteboardElement ]);
  nb = addPage(nb);
  nb = mapBoardElements(nb, () => [
    { ...el("p2"), type: "ellipse", width: 60, height: 60 } as unknown as WhiteboardElement,
  ]);

  const { svg, report, height } = buildExportSvg(nb);
  assert.equal(report.pages, 2);
  assert.match(svg, /<ellipse/, "page 2's content is exported");
  // Page 1's rect sits inside the first sheet's clip group.
  assert.ok(svg.includes('clip-path="url(#pn-sheet-0)"') && svg.includes('clip-path="url(#pn-sheet-1)"'));
  const { height: h } = pageSizeOf(nb);
  assert.equal(height, 2 * h + PAGE_GAP, "sheets are stacked with the scroll view's gap");
});

test('pages: "current" exports only the page on screen', () => {
  let nb = makeNotebook(canvas());
  nb = mapBoardElements(nb, () => [ { ...el("p1"), width: 20, height: 20 } as WhiteboardElement ]);
  nb = addPage(nb);
  const { report, height, svg } = buildExportSvg(nb, { pages: "current" });
  assert.equal(report.pages, 1);
  assert.equal(height, pageSizeOf(nb).height, "exactly one sheet");
  assert.equal(svg.includes("pn-sheet-1"), false);
});

test("a notebook page exports as the SHEET, clipped - not the ink's bounding box", () => {
  // Ink from before sheets were enforced can sit far off the page. On screen it
  // is invisible past the paper edge; exporting its bounds produced a smear.
  let nb = makeNotebook(canvas());
  nb = mapBoardElements(nb, () => [
    { ...el("offsheet"), x: -900, y: -900, width: 30, height: 30 } as WhiteboardElement,
  ]);
  const { svg, width, height } = buildExportSvg(nb, { pages: "current" });
  const size = pageSizeOf(nb);
  assert.equal(width, size.width);
  assert.equal(height, size.height);
  assert.match(svg, new RegExp(`viewBox="0 0 ${size.width} ${size.height}"`));
  assert.match(svg, /<clipPath id="pn-sheet-0">/);
});

test("the notebook's ruling is exported with the page", () => {
  let nb = makeNotebook(canvas());
  nb = setPageBackground(nb, getActivePage(nb)!.id, { pattern: "lines" });
  const { svg } = buildExportSvg(nb);
  assert.match(svg, /<pattern id="pn-paper-0"/, "lined paper is part of the page");
});

```

```

test("a picture whose pixels are missing is COUNTED, not silently greyed", () => {
  const b = canvas([
    { ...el("i1"), type: "image", src: "asset:img_gone", width: 40, height: 40 } as unknown as WhiteboardElement,
  ]);
  const { report, svg } = buildExportSvg(b);
  assert.equal(report.missingImages, 1);
  assert.match(svg, /fill="#e4e4e7"/, "still drawn as a placeholder so the layout holds");
});

test("a PDF-backed page with no PDF loaded is reported, and exports as paper", () => {
  let nb = makeNotebook(canvas());
  const page = getActivePage(nb)!;
  nb = { ...nb, pages: [{ ...page, pdf: { docId: "gone", pageNumber: 1 } }] };
  const { report } = buildExportSvg(nb);
  assert.equal(report.missingPdfPages, 1);
});

test("an unrecognised element with no position does not drag bounds to the origin", () => {
  // The fallback used `?? 0`, so one corrupt element pinned export and
  // zoom-to-fit to 0,0 however far away the real content was.
  const real = { ...el("r"), x: 5000, y: 5000, width: 100, height: 100 } as WhiteboardElement;
  const junk = { id: "junk", type: "hologram" } as unknown as WhiteboardElement;
  const b = getUnionBounds([real, junk]);
  assert.equal(b.x, 5000);
  assert.equal(b.y, 5000);
});

test("an unrecognised element WITH a position still counts toward bounds", () => {
  const real = { ...el("r"), x: 100, y: 100, width: 10, height: 10 } as WhiteboardElement;
  const future = { id: "f", type: "hologram", x: 400, y: 400 } as unknown as WhiteboardElement;
  const b = getUnionBounds([real, future]);
  assert.equal(b.x + b.width, 410, "the placed unknown element is still inside the frame");
});

test("an unresolved image reference is omitted, never written as a broken href", () => {
  const b = canvas([
    { ...el("i1"), type: "image", src: "asset:img_missing", width: 40, height: 40 } as unknown as WhiteboardElement,
  ]);
  const svg = exportBoardToSvg(b);
  assert.equal(/asset:\/.test(svg), false, "an asset: href means nothing outside the app");
});

test("a resolved image reference is written as its real data URL", () => {
  const b = canvas([
    { ...el("i1"), type: "image", src: "asset:img_1", width: 40, height: 40 } as unknown as WhiteboardElement,
  ]);
  const svg = exportBoardToSvg(b, {}, [], new Map([["asset:img_1", ""]]));
  assert.match(svg, /data:image\/png;base64,AAAA/);
});

// -----
// Ink stays on the paper
// -----
// A notebook used to share the whiteboard's unbounded world, so you could write
// beside a sheet or in the dead gap between two of them. That ink was saved but
// never exported, never printed, and impossible to find again once you
// scrolled. These pin the rule that it cannot happen.

const SHEET_W = 794;
const SHEET_H = 1123;

test("a point on the page is left exactly where it is", () => {
  const p = clampToSheet(400, 500, SHEET_W, SHEET_H);
  assert.deepEqual(p, { x: 400, y: 500 }, "clamping must not nudge valid ink");
});

test("a point to the RIGHT of the page is pulled back onto it", () => {
  const p = clampToSheet(SHEET_W + 600, 500, SHEET_W, SHEET_H);
  assert.equal(p.x, SHEET_W - SHEET_INK_INSET, "lands on the margin, not off in space");
  assert.equal(p.y, 500, "the other axis is untouched");
});

test("a point in the gap BETWEEN two sheets is pulled onto the sheet", () => {
  const p = clampToSheet(300, SHEET_H + PAGE_GAP / 2, SHEET_W, SHEET_H);
  assert.equal(p.y, SHEET_H - SHEET_INK_INSET, "no ink may live in the gap");
}

```

```

});

test("a point far above and left of the page is pulled back", () => {
  const p = clampToSheet(-5000, -5000, SHEET_W, SHEET_H);
  assert.deepEqual(p, { x: SHEET_INK_INSET, y: SHEET_INK_INSET });
});

test("the inset keeps a stroke's width off the page border", () => {
  assert.ok(SHEET_INK_INSET > 0, "a point on the edge paints half its width off the page");
  const p = clampToSheet(SHEET_W, SHEET_H, SHEET_W, SHEET_H);
  assert.ok(p.x < SHEET_W && p.y < SHEET_H);
});

test("isOnActiveSheet answers for the page at the origin", () => {
  assert.equal(isOnActiveSheet(10, 10, SHEET_W, SHEET_H), true);
  assert.equal(isOnActiveSheet(SHEET_W + 10, 10, SHEET_W, SHEET_H), false, "beside the page");
  assert.equal(isOnActiveSheet(10, SHEET_H + 10, SHEET_W, SHEET_H), false, "in the gap below");
  assert.equal(
    isOnActiveSheet(SHEET_W + 10, 10, SHEET_W, SHEET_H, 24),
    true,
    "the grab margin forgives a press just outside the border"
  );
});

test("a dragged element is MOVED back onto the sheet, never resized", () => {
  const box = clampBoxToSheet(SHEET_W - 10, 50, 200, 100, SHEET_W, SHEET_H);
  assert.equal(box.x, SHEET_W - 200, "its right edge sits on the page edge");
  assert.equal(box.y, 50, "vertical position is untouched");
});

test("an element too big for the page is pinned, not squashed", () => {
  const box = clampBoxToSheet(-300, -300, SHEET_W + 500, SHEET_H + 500, SHEET_W, SHEET_H);
  assert.deepEqual(box, { x: 0, y: 0 }, "stays whole and reachable at the top-left");
});

test("clamping is idempotent", () => {
  const once = clampToSheet(99999, -99999, SHEET_W, SHEET_H);
  const twice = clampToSheet(once.x, once.y, SHEET_W, SHEET_H);
  assert.deepEqual(twice, once, "re-clamping a clamped point must not drift");
});

// -----
// Which sheet is under the pen (several are visible when zoomed out)
// -----

{
  const W = 794, H = 1123, STRIDE = H + PAGE_GAP;
  // Page 3 of 5 is live (index 2) and sits at y = 0.
  test("a point on the live sheet is the live sheet", () => {
    assert.equal(sheetIndexAt(400, 500, 2, 5, W, H), 2);
  });
  test("a point on the sheet ABOVE the live one finds that sheet", () => {
    assert.equal(sheetIndexAt(400, -STRIDE + 300, 2, 5, W, H), 1);
  });
  test("a point on the sheet BELOW the live one finds that sheet", () => {
    assert.equal(sheetIndexAt(400, STRIDE + 10, 2, 5, W, H), 3);
  });
  test("the gap between two sheets is no sheet", () => {
    assert.equal(sheetIndexAt(400, H + PAGE_GAP / 2, 2, 5, W, H), null);
  });
  test("above the first page and below the last are no sheet", () => {
    assert.equal(sheetIndexAt(400, -3 * STRIDE + 10, 2, 5, W, H), null, "there is no page -1");
    assert.equal(sheetIndexAt(400, 3 * STRIDE + 10, 2, 5, W, H), null, "there is no page 6");
  });
  test("beside the paper is no sheet, unless within the grab margin", () => {
    assert.equal(sheetIndexAt(W + 30, 500, 2, 5, W, H), null);
    assert.equal(sheetIndexAt(W + 20, 500, 2, 5, W, H, 24), 2);
  });
}

console.log(`\n ${passed} passed, ${failed} failed\n`);
if (failed > 0) process.exit(1);

=====
// === END FILE: tests/notebook.test.ts ===
=====

```

```

=====
// === START FILE: tests/notebook_camera.test.ts ===
// File 165 of 194 | Lines: 348 | Size: 15778 bytes
=====

/**
 * How much bare desk may surround the paper.
 *
 * The bug this pins: the camera guard only asked that 120 px of the active
 * sheet stay somewhere on screen. On a tablet in portrait, zoomed out, that let
 * the page be shoved against one edge with most of a screenful of empty grey
 * beside it - and the same above and below the stack. Measured on the device:
 * an A4 sheet 349 px wide sitting at the left of a 1250 px screen, with about
 * 900 px of nothing to its right.
 *
 * Every assertion below is written in SCREEN pixels of visible desk, because
 * that is the thing being complained about. Re-deriving the formula here would
 * only prove the formula equals itself.
 */

import assert from "node:assert/strict";

import { useWhiteboard } from "../src/store/whiteboard";
import {
  PAGE_GAP,
  DESK_MARGIN_PX,
  clampNotebookCamera,
  minNotebookZoom,
} from "../src/lib/whiteboard/notebook";

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(` ${err as Error}.message`);
  }
}

const A4 = { width: 794, height: 1123 };
const TABLET = { width: 1250, height: 1126 }; // Galaxy Tab S9, portrait
const DESKTOP = { width: 1000, height: 1400 };

/**
 * The strip of empty desk visible on each side, in screen pixels.
 * Negative means the paper runs off that edge, so no desk shows there.
 */
function desk(
  cam: { x: number; y: number; zoom: number },
  vp: { width: number; height: number },
  page: { width: number; height: number },
  doc: { pageCount: number; activeIndex: number }
) {
  const z = cam.zoom;
  const stride = page.height + PAGE_GAP;
  const docTop = -(doc.activeIndex * stride);
  const docBottom = (doc.pageCount - 1 - doc.activeIndex) * stride + page.height;
  const left = cam.x - vp.width / 2 / z;
  const right = cam.x + vp.width / 2 / z;
  const top = cam.y - vp.height / 2 / z;
  const bottom = cam.y + vp.height / 2 / z;
  return {
    left: (0 - left) * z,
    right: (right - page.width) * z,
    top: (docTop - top) * z,
    bottom: (bottom - docBottom) * z,
  };
}

const SLACK = 0.5; // sub-pixel rounding

/**
 * The whole rule, on one axis: either the desk past the paper is at most

```

```

* DESK_MARGIN_PX, or the paper is smaller than the screen on that axis and is
* therefore centred - which is the least wasteful position that exists. What
* is forbidden is the third case, the one being complained about: paper pushed
* against one edge with the waste piled up on the other.
*/
function assertTidy(
  near: number,
  far: number,
  paperPx: number,
  screenPx: number,
  where: string
) {
  if (paperPx >= screenPx) {
    assert.ok(near <= DESK_MARGIN_PX + SLACK, `${where}: ${near.toFixed(0)}px of desk`);
    assert.ok(far <= DESK_MARGIN_PX + SLACK, `${where}: ${far.toFixed(0)}px of desk`);
  } else {
    assert.ok(Math.abs(near - far) < SLACK, `${where}: off-centre by ${Math.abs(near - far).toFixed(0)}px`);
  }
}

const at = (x: number, y: number, zoom: number) => ({ x, y, zoom });

console.log("\n■■ Notebook camera: the desk around the paper ■■");

test("a page shoved to the left edge is pulled back", () => {
  // The camera measured on the tablet: page at the left, screenful of grey.
  const doc = { pageCount: 14, activeIndex: 8 };
  const c = clampNotebookCamera(at(1420, 0, 0.44), TABLET, A4, doc);
  const d = desk({ ...c, zoom: 0.44 }, TABLET, A4, doc);
  // Before the fix this camera was legal and put ~900px of grey on the right.
  assert.ok(d.right < 500, `right desk ${d.right.toFixed(0)}px is still a wasteland`);
  assertTidy(d.left, d.right, A4.width * 0.44, TABLET.width, "x");
});

test("a sheet narrower than the screen is centred, not parked on one side", () => {
  const doc = { pageCount: 14, activeIndex: 8 };
  // 794 * 0.44 = 349px of paper on a 1250px screen: desk on both sides is
  // unavoidable, but the two sides must match.
  for (const pushed of [-4000, -500, 0, 500, 4000]) {
    const c = clampNotebookCamera(at(pushed, 0, 0.44), TABLET, A4, doc);
    const d = desk({ ...c, zoom: 0.44 }, TABLET, A4, doc);
    assert.ok(
      Math.abs(d.left - d.right) < SLACK,
      `off-centre by ${Math.abs(d.left - d.right).toFixed(0)}px starting from x=${pushed}`
    );
  }
});

test("you cannot pan into an empty void above the first page", () => {
  const doc = { pageCount: 10, activeIndex: 0 };
  const c = clampNotebookCamera(at(397, -3000, 0.9), DESKTOP, A4, doc);
  const d = desk({ ...c, zoom: 0.9 }, DESKTOP, A4, doc);
  assert.ok(d.top <= DESK_MARGIN_PX + SLACK, `${d.top.toFixed(0)}px of grey above page 1`);
});

test("nor below the last page", () => {
  const doc = { pageCount: 10, activeIndex: 9 };
  const c = clampNotebookCamera(at(397, 9000, 0.9), DESKTOP, A4, doc);
  const d = desk({ ...c, zoom: 0.9 }, DESKTOP, A4, doc);
  assert.ok(d.bottom <= DESK_MARGIN_PX + SLACK, `${d.bottom.toFixed(0)}px of grey below the last page`);
});

test("the desk that is left is a real strip, not a hairline", () => {
  // "smaller, but not zero": there must still be something to see and to grab.
  const doc = { pageCount: 10, activeIndex: 0 };
  const c = clampNotebookCamera(at(397, -3000, 0.9), DESKTOP, A4, doc);
  const d = desk({ ...c, zoom: 0.9 }, DESKTOP, A4, doc);
  assert.ok(d.top > 40, `only ${d.top.toFixed(0)}px of desk above the page`);
});

test("the middle of the notebook still scrolls freely", () => {
  // The page turn fires when the camera passes the middle of the gap. If the
  // bound stopped short of that, the notebook would jam on one page forever.
  const doc = { pageCount: 10, activeIndex: 5 };
  const reanchorForward = A4.height + PAGE_GAP / 2;
  const reanchorBack = -PAGE_GAP / 2;
  for (const z of [0.3, 0.44, 0.9, 1.6, 2]) {
    const fwd = clampNotebookCamera(at(397, reanchorForward + 1, z), TABLET, A4, doc);
    assert.ok(fwd.y > reanchorForward, `zoom ${z}: cannot reach the next page (y=${fwd.y.toFixed(0)})`);
  }
});

```

```

    const back = clampNotebookCamera(at(397, reanchorBack - 1, z), TABLET, A4, doc);
    assert.ok(back.y < reanchorBack, `zoom ${z}: cannot reach the previous page (y=${back.y.toFixed(0)})`);
  }
});

test("zoomed in, both edges of the sheet are still reachable", () => {
  const doc = { pageCount: 3, activeIndex: 1 };
  const z = 2.5; // the page is 1985px wide, far wider than the screen
  const leftEnd = clampNotebookCamera(at(-9999, 0, z), TABLET, A4, doc);
  const dL = desk({ ...leftEnd, zoom: z }, TABLET, A4, doc);
  assert.ok(dL.left > 0 && dL.left <= DESK_MARGIN_PX + SLACK, `left edge desk ${dL.left.toFixed(0)}px`);
  const rightEnd = clampNotebookCamera(at(9999, 0, z), TABLET, A4, doc);
  const dR = desk({ ...rightEnd, zoom: z }, TABLET, A4, doc);
  assert.ok(dR.right > 0 && dR.right <= DESK_MARGIN_PX + SLACK, `right edge desk ${dR.right.toFixed(0)}px`);
});

test("a one-page notebook that fits is centred on both axes", () => {
  const doc = { pageCount: 1, activeIndex: 0 };
  const z = 0.4; // 318 x 449 of paper inside 1250 x 1126
  const c = clampNotebookCamera(at(-2000, 5000, z), TABLET, A4, doc);
  const d = desk({ ...c, zoom: z }, TABLET, A4, doc);
  assert.ok(Math.abs(d.left - d.right) < SLACK, `x off-centre by ${Math.abs(d.left - d.right).toFixed(0)}px`);
  assert.ok(Math.abs(d.top - d.bottom) < SLACK, `y off-centre by ${Math.abs(d.top - d.bottom).toFixed(0)}px`);
});

test("clamping twice changes nothing (two guards compose)", () => {
  // The store bounds the camera and the canvas bounds it again a frame later.
  // If this were not idempotent, those two would push each other forever.
  const doc = { pageCount: 14, activeIndex: 8 };
  for (const z of [0.2, 0.44, 1, 2.5]) {
    for (const start of [at(-5000, -5000, z), at(5000, 5000, z), at(397, 500, z)]) {
      const once = clampNotebookCamera(start, TABLET, A4, doc);
      const twice = clampNotebookCamera({ ...once, zoom: z }, TABLET, A4, doc);
      assert.ok(
        Math.abs(twice.x - once.x) < 1e-6 && Math.abs(twice.y - once.y) < 1e-6,
        `zoom ${z} from ${start.x},${start.y}`
      );
    }
  }
});

test("a camera already inside the bounds is left alone", () => {
  const doc = { pageCount: 14, activeIndex: 8 };
  const start = at(397, 560, 1.4);
  const c = clampNotebookCamera(start, TABLET, A4, doc);
  assert.equal(c.x, start.x);
  assert.equal(c.y, start.y);
});

test("nonsense input is returned untouched rather than producing NaN", () => {
  const doc = { pageCount: 5, activeIndex: 0 };
  assert.deepEqual(clampNotebookCamera(at(10, 20, 0), TABLET, A4, doc), { x: 10, y: 20 });
  assert.deepEqual(clampNotebookCamera(at(10, 20, 1), { width: 0, height: 0 }, A4, doc), { x: 10, y: 20 });
  assert.deepEqual(clampNotebookCamera(at(10, 20, 1), TABLET, { width: 0, height: 0 }, doc), { x: 10, y: 20 });
  ;
});

test("a PDF-sized page (landscape slides) obeys the same rule", () => {
  const SLIDE = { width: 1280, height: 720 };
  const doc = { pageCount: 40, activeIndex: 20 };
  for (const z of [0.8, 1.2]) {
    const c = clampNotebookCamera(at(-4000, 0, z), TABLET, SLIDE, doc);
    const d = desk({ ...c, zoom: z }, TABLET, SLIDE, doc);
    assertTidy(d.left, d.right, SLIDE.width * z, TABLET.width, `slides at ${z}`);
  }
});

console.log(`\n■■■ Notebook camera: how far out you may zoom ■■■`);

/** What fraction of the screen the paper covers at a given zoom. */
function coverage(z: number, vp: { width: number; height: number }, page: { width: number; height: number }) {
  return (Math.min(page.width * z, vp.width) * Math.min(page.height * z, vp.height)) / (vp.width * vp.height);
}

test("the floor always leaves the whole page visible", () => {
  // The point of stopping there: one more step out shows no more paper.
  for (const vp of [TABLET, DESKTOP, { width: 1250, height: 1980 }, { width: 375, height: 812 }]) {
    for (const page of [A4, { width: 1280, height: 720 }, { width: 900, height: 900 }]) {

```

```

    const z = minNotebookZoom(vp, page);
    assert.ok(page.width * z <= vp.width + SLACK, `${vp.width}x${vp.height}: page too wide at the floor`);
    assert.ok(page.height * z <= vp.height + SLACK, `${vp.width}x${vp.height}: page too tall at the floor`);
  }
}
});

test("and leaves the desk margin around it, not a hairline", () => {
  const z = minNotebookZoom(TABLET, A4);
  const spare = Math.min(TABLET.width - A4.width * z, TABLET.height - A4.height * z);
  assert.ok(spare >= DESK_MARGIN_PX * 2 - SLACK, `only ${spare.toFixed(0)}px spare around the page`);
});

test("the page cannot be shrunk to a stamp any more", () => {
  // The reported state: A4 at 44% on the tablet covered 12% of the screen.
  const vp = { width: 1250, height: 1980 };
  const before = coverage(0.44, vp, A4);
  const after = coverage(minNotebookZoom(vp, A4), vp, A4);
  assert.ok(before < 0.15, `sanity: the old state covered ${((before * 100).toFixed(0))}%`);
  // Not the whole screen: the floor deliberately allows a fifth past the fit
  // so the next sheet shows. What that extra room shows is paper, not desk -
  // vertically it is the neighbouring page, and only the sides are bare.
  assert.ok(after > 0.4, `the floor still only covers ${((after * 100).toFixed(0))}% of the screen`);
});

test("the tablet that was reported can reach 60%", () => {
  // 1600 x 2560 at density 340 is 753 x 1126 CSS pixels, which is what the
  // app actually measures. The ask was "let me reach 60%".
  const z = minNotebookZoom({ width: 753, height: 1126 }, A4);
  assert.ok(z > 0.57 && z < 0.63, `floor is ${((z * 100).toFixed(0))}%, not about 60%`);
});

test("at the floor you can see past the page you are on", () => {
  // The reason to zoom out at all. Stopping exactly at the fit hid the next
  // sheet, which made the floor feel like a cage.
  for (const vp of [{ width: 753, height: 1126 }, TABLET, DESKTOP]) {
    const z = minNotebookZoom(vp, A4);
    const visibleWorldH = vp.height / z;
    assert.ok(
      visibleWorldH > A4.height + PAGE_GAP,
      `${vp.width}x${vp.height}: only ${Math.round(visibleWorldH)} of ${A4.height + PAGE_GAP} world px visible`
    );
  }
});

test("the join between sheets is a seam, not a band", () => {
  // What the reference app does: pages run together with a hairline. A gap
  // you can measure in centimetres is what made zooming out feel empty.
  for (const vp of [{ width: 753, height: 1126 }, TABLET, DESKTOP]) {
    const seamPx = PAGE_GAP * minNotebookZoom(vp, A4);
    assert.ok(seamPx > 0, "the boundary must stay visible");
    assert.ok(seamPx < 12, `the seam is ${seamPx.toFixed(0)}px of dead space`);
  }
});

test("the floor follows the screen, which a fixed percentage cannot", () => {
  // "40%" is a full page on a phone and a stamp on a monitor. Same paper,
  // three screens, three different answers - all of them showing a whole page.
  const phone = minNotebookZoom({ width: 375, height: 812 }, A4);
  const tablet = minNotebookZoom({ width: 1250, height: 1980 }, A4);
  const monitor = minNotebookZoom({ width: 1800, height: 1000 }, A4);
  assert.ok(phone < tablet, `phone ${phone.toFixed(2)} should be below tablet ${tablet.toFixed(2)}`);
  assert.ok(monitor < tablet, `a short monitor runs out of height first`);
  assert.ok(coverage(0.4, { width: 1800, height: 1000 }, A4) < 0.2, "40% on a monitor is indeed a stamp");
});

test("a wide screen can still show a whole A4 page", () => {
  // The floor must not be taken from the width alone, or a landscape monitor
  // would be pinned so far in that the bottom of the page is unreachable.
  const vp = { width: 1800, height: 1000 };
  const z = minNotebookZoom(vp, A4);
  assert.ok(A4.height * z <= vp.height, `page bottom off screen at the floor (${z.toFixed(2)})`);
});

test("no floor when the screen is smaller than the margins", () => {
  assert.equal(minNotebookZoom({ width: 100, height: 100 }, A4), 0);
  assert.equal(minNotebookZoom(TABLET, { width: 0, height: 0 }), 0);
});

```

```

});

console.log("\n■■■ Notebook camera: the floor follows the canvas, not the window ■■■");

test("zoomTo clamps to the floor for the canvas size the store was told", () => {
  const S = () => useWhiteboard.getState();
  const id = S().createNotebook("Floor NB", null);
  S().setActiveBoard(id);

  // Narrow canvas (sidebar open): a low floor.
  S().setCanvasViewport({ width: 544, height: 1126 });
  S().zoomTo(0.05);
  const narrow = S().camera.zoom;
  assert.ok(Math.abs(narrow - minNotebookZoom({ width: 544, height: 1126 }, A4)) < 1e-6, `got ${narrow}`);

  // The sidebar is hidden and the canvas grows. The floor must grow with it -
  // this is the bug: the observer updated the component and not the store, so
  // the floor stayed at the narrow canvas and the page could still shrink.
  S().setCanvasViewport({ width: 800, height: 1126 });
  S().zoomTo(0.05);
  const wide = S().camera.zoom;
  assert.ok(wide > narrow, `floor did not follow the canvas: ${narrow} -> ${wide}`);
  assert.ok(Math.abs(wide - minNotebookZoom({ width: 800, height: 1126 }, A4)) < 1e-6, `got ${wide}`);
});

test("a whiteboard still zooms out freely", () => {
  const S = () => useWhiteboard.getState();
  const id = S().createBoard("Floor WB", null);
  S().setActiveBoard(id);
  S().setCanvasViewport({ width: 800, height: 1126 });
  S().zoomTo(0.05);
  assert.ok(S().camera.zoom <= 0.05 + 1e-9, `whiteboard was clamped to ${S().camera.zoom}`);
});

test("a nonsense canvas size is ignored rather than stored", () => {
  const S = () => useWhiteboard.getState();
  S().setCanvasViewport({ width: 800, height: 1126 });
  S().setCanvasViewport({ width: 0, height: 0 });
  assert.deepEqual(S().canvasViewport, { width: 800, height: 1126 });
});

console.log(`\n ${passed} passed, ${failed} failed\n`);
if (failed > 0) process.exit(1);

=====
// === END FILE: tests/notebook_camera.test.ts ===
=====

=====
// === START FILE: tests/notebook_store.test.ts ===
// File 166 of 194 | Lines: 1055 | Size: 37538 bytes
=====

/**
 * Notebook × store integration.
 *
 * notebook.test.ts proves the pure page model. This proves the *wiring*: 34
 * places in the store used to touch `board.elements` directly, and every one
 * of them now has to route to the active page instead. A site that was missed
 * would write a stroke onto the canvas field of a notebook, where nothing
 * renders it – invisible data loss. These tests exercise the real store.
 */

import assert from "node:assert/strict";

import { useWhiteboard } from "../src/store/whiteboard";
import { migrateState } from "../src/lib/whiteboard/persistence";
import {
  getBoardElements,
  getPages,
  allElements,
  getActivePageIndex,
  PAGE_GAP,
  bookmarkedPages,
  pageSizeOf,
} from "../src/lib/whiteboard/notebook";
import type { Board, WhiteboardElement } from "../src/lib/whiteboard/types";

```

```

import {
  effectiveSelectFilter,
  passesSelectFilter,
  selectCategoryOf,
} from "../src/lib/whiteboard/types";

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`      ${err as Error}.message`);
  }
}

const S = () => useWhiteboard.getState();
const board = (id: string): Board => S().boards.find((b) => b.id === id)!;

function rect(id: string, x = 0): WhiteboardElement {
  return {
    id,
    type: "rectangle",
    x,
    y: 0,
    width: 10,
    height: 10,
    angle: 0,
    strokeColor: "#000",
    backgroundColor: "transparent",
    fillStyle: "transparent",
    strokeWidth: 2,
    strokeStyle: "solid",
    dashStyle: "solid",
    opacity: 1,
    seed: 1,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  } as unknown as WhiteboardElement;
}

/** A fresh notebook, made active, with `pageCount` pages. */
function freshNotebook(pageCount = 1): string {
  const id = S().createNotebook(`NB ${Math.random().toString(36).slice(2, 7)}`, null);
  S().setActiveBoard(id);
  for (let i = 1; i < pageCount; i++) S().addPage();
  return id;
}

console.log(`\n■■ Notebook × store ■■`);

test("createNotebook produces a notebook with exactly one page", () => {
  const id = freshNotebook();
  assert.equal(board(id).kind, "notebook");
  assert.equal(getPages(board(id)).length, 1);
  assert.equal(board(id).pageSize, "a4");
});

test("a plain board is still a canvas", () => {
  const id = S().createBoard("Plain", null);
  assert.equal(board(id).kind, undefined);
});

test("addElement lands on the active page, never on the canvas field", () => {
  const id = freshNotebook(2); // page 2 is active after addPage
  S().addElement(rect("a"));

  assert.deepEqual(getBoardElements(board(id)).map((e) => e.id), ["a"]);
  assert.deepEqual(board(id).elements, [], "canvas field must stay empty");
  assert.deepEqual(getPages(board(id))[0].elements, [], "page 1 untouched");
  assert.deepEqual(getPages(board(id))[1].elements.map((e) => e.id), ["a"]);
});

test("ink drawn on one page does not appear on another", () => {

```

```
const id = freshNotebook(3);
S().addElement(rect("p3"));          // on page 3
S().stepPage(-1);
assert.deepEqual(getBoardElements(board(id)), [], "page 2 is blank");
S().addElement(rect("p2"));
S().stepPage(-1);
assert.deepEqual(getBoardElements(board(id)), [], "page 1 is blank");

assert.deepEqual(allElements(board(id)).map((e) => e.id), ["p2", "p3"].sort(), );
});

test("addElement (paste / multi-add) also lands on the active page", () => {
  const id = freshNotebook(2);
  S().addElement([rect("x"), rect("y")]);
  assert.deepEqual(getBoardElements(board(id)).map((e) => e.id), ["x", "y"]);
  assert.deepEqual(getPages(board(id))[0].elements, []);
});

test("updateElement edits the copy on the active page", () => {
  const id = freshNotebook();
  S().addElement(rect("a", 5));
  S().updateElement("a", { x: 99 } as never, false);
  assert.equal((getBoardElements(board(id))[0] as { x: number }).x, 99);
});

test("deleteElements removes from the active page only", () => {
  const id = freshNotebook();
  S().addElement(rect("keep"));
  S().addPage();
  S().addElement(rect("gone"));
  S().deleteElements(["gone"], false);

  assert.deepEqual(getBoardElements(board(id)), [], "page 2 emptied");
  assert.deepEqual(getPages(board(id))[0].elements.map((e) => e.id), ["keep", "page 1 kept"]);
});

test("z-order actions reorder within the active page", () => {
  const id = freshNotebook();
  S().addElement(rect("a"));
  S().addElement(rect("b"));
  S().addElement(rect("c"));

  S().bringToFront(["a"]);
  assert.deepEqual(getBoardElements(board(id)).map((e) => e.id), ["b", "c", "a"]);

  S().sendToBack(["c"]);
  assert.deepEqual(getBoardElements(board(id)).map((e) => e.id), ["c", "b", "a"]);

  S().sendBackward(["a"]);
  assert.deepEqual(getBoardElements(board(id)).map((e) => e.id), ["c", "a", "b"]);

  S().bringForward(["c"]);
  assert.deepEqual(getBoardElements(board(id)).map((e) => e.id), ["a", "c", "b"]);
});

test("duplicateElements clones onto the same page", () => {
  const id = freshNotebook(2);
  S().addElement(rect("orig"));
  const made = S().duplicateElements(["orig"]);
  assert.equal(made.length, 1);
  assert.equal(getBoardElements(board(id)).length, 2);
  assert.deepEqual(getPages(board(id))[0].elements, [], "page 1 not touched");
});

test("selectAll selects only what is on the current page", () => {
  const id = freshNotebook();
  S().addElement(rect("a"));
  S().addPage();
  S().addElement(rect("b"));
  S().addElement(rect("c"));
  S().selectAll();
  assert.deepEqual(S().selectedIds.sort(), ["b", "c"]);
  assert.ok(board(id));
});

test("undo restores onto the page the edit happened on", () => {
  const id = freshNotebook(2);
  S().addElement(rect("first"));          // page 2, commits history
```

```

S().addElement(rect("second")); // page 2
assert.equal(getBoardElements(board(id)).length, 2);

S().undo();
assert.deepEqual(
  getBoardElements(board(id)).map((e) => e.id),
  ["first"],
  "undo removed the second element from page 2"
);
assert.equal(getActivePageIndex(board(id)), 1, "and stayed on page 2");
assert.deepEqual(getPages(board(id))[0].elements, [], "page 1 was never involved");
});

test("undo after turning the page edits the page the snapshot came from", () => {
  const id = freshNotebook(2);
  S().addElement(rect("onPage2"));
  S().setActivePage(getPages(board(id))[0].id); // turn back to page 1

  S().undo();

  assert.deepEqual(getPages(board(id))[1].elements, [], "page 2's element was undone");
  assert.deepEqual(getPages(board(id))[0].elements, [], "page 1 was not overwritten with page 2 content");
});

test("redo puts it back on the right page too", () => {
  const id = freshNotebook(2);
  S().addElement(rect("x"));
  S().undo();
  assert.deepEqual(getPages(board(id))[1].elements, []);
  S().redo();
  assert.deepEqual(getPages(board(id))[1].elements.map((e) => e.id), ["x"]);
  assert.deepEqual(getPages(board(id))[0].elements, []);
});

test("duplicateBoard copies every page with fresh element ids", () => {
  const id = freshNotebook();
  S().addElement(rect("p1"));
  S().addPage();
  S().addElement(rect("p2"));

  const copyId = S().duplicateBoard(id)!;
  const copy = board(copyId);

  assert.equal(getPages(copy).length, 2, "both pages copied");
  assert.equal(allElements(copy).length, 2);
  assert.deepEqual(copy.elements, [], "nothing stranded on the canvas field");

  const originalIds = allElements(board(id)).map((e) => e.id);
  for (const el of allElements(copy)) {
    assert.ok(!originalIds.includes(el.id), `copied element ${el.id} must not reuse an original id`);
  }
  const pageIds = getPages(copy).map((p) => p.id);
  for (const p of getPages(board(id))) {
    assert.ok(!pageIds.includes(p.id), "copied pages must not reuse page ids");
  }
});

test("search finds text on any page, not just the visible one", () => {
  const id = freshNotebook();
  S().addElement({ ...rect("t1"), type: "text", text: "alpha marker" } as never);
  S().addPage();
  S().addElement({ ...rect("t2"), type: "text", text: "beta marker" } as never);
  S().setActivePage(getPages(board(id))[0].id);

  const hits = S().searchBoards("beta marker");
  assert.ok(
    hits.some((h) => h.boardId === id),
    "text on page 2 must be findable while page 1 is open"
  );
});

test("page turns clear the selection so handles do not float over nothing", () => {
  const id = freshNotebook();
  S().addElement(rect("a"));
  S().selectAll();
  assert.deepEqual(S().selectedIds, ["a"]);
  S().addPage();
  assert.deepEqual(S().selectedIds, [], "selection cleared on the new page");
});

```

```
    assert.ok(board(id));
  });

test("convertToNotebook turns a canvas into page 1, losing nothing", () => {
  const id = S().createBoard("Canvas with work", null);
  S().setActiveBoard(id);
  S().addElement(rect("drawn1"));
  S().addElement(rect("drawn2"));
  assert.equal(board(id).kind, undefined);

  S().convertToNotebook(id);

  assert.equal(board(id).kind, "notebook");
  assert.equal(getPages(board(id)).length, 1);
  assert.deepEqual(
    getBoardElements(board(id)).map((e) => e.id),
    ["drawn1", "drawn2"],
    "existing work became page 1"
  );
  assert.deepEqual(board(id).elements, [], "and is not left duplicated on the canvas field");
});

test("convertToNotebook is a no-op on something already a notebook", () => {
  const id = freshNotebook(3);
  S().addElement(rect("keep"));
  S().convertToNotebook(id);
  assert.equal(getPages(board(id)).length, 3, "pages not collapsed");
  assert.deepEqual(getBoardElements(board(id)).map((e) => e.id), ["keep"]);
});

test("converting clears history, since old entries name no page", () => {
  const id = S().createBoard("C", null);
  S().setActiveBoard(id);
  S().addElement(rect("a"));
  S().convertToNotebook(id);
  S().undo();
  assert.deepEqual(
    getBoardElements(board(id)).map((e) => e.id),
    ["a"],
    "a pre-conversion undo must not fire and blank page 1"
  );
});

test("setPageSize applies to the notebook without moving ink", () => {
  const id = freshNotebook();
  S().addElement(rect("a", 700));
  S().setPageSize("letter-landscape");
  assert.equal(board(id).pageSize, "letter-landscape");
  assert.equal((getBoardElements(board(id))[0] as { x: number }).x, 700, "ink untouched");
});

test("reanchorPage swaps the page and slides the camera so nothing jumps", () => {
  const id = freshNotebook(3);
  S().setActivePage(getPages(board(id))[0].id);
  const stride = 1123 + PAGE_GAP;

  // Scroll down past the bottom of page 1.
  S().setCamera({ x: 0, y: stride - 100, zoom: 1 });
  const before = S().camera.y;

  S().reanchorPage(1, stride);

  assert.equal(getActivePageIndex(board(id)), 1, "page 2 is now the anchored page");
  assert.equal(
    S().camera.y,
    before - stride,
    "camera slid by exactly one stride, so the view does not move"
  );
});

test("reanchorPage going back up is the exact inverse", () => {
  const id = freshNotebook(3);
  const stride = 1123 + PAGE_GAP;
  S().reanchorPage(1, stride);
  const midIndex = getActivePageIndex(board(id));
  const midY = S().camera.y;

  S().reanchorPage(-1, stride);
```

```

    assert.equal(getActivePageIndex(board(id)), midIndex - 1);
    assert.equal(S().camera.y, midY + stride, "returns to exactly where it was");
  });

test("reanchorPage does nothing at the ends, so the camera cannot drift", () => {
  const id = freshNotebook(2);
  S().setActivePage(getPages(board(id))[0].id);
  const stride = 1123 + PAGE_GAP;
  const y = S().camera.y;

  S().reanchorPage(-1, stride); // already on page 1
  assert.equal(getActivePageIndex(board(id)), 0);
  assert.equal(S().camera.y, y, "camera untouched when the page cannot change");

  S().reanchorPage(1, stride);
  S().reanchorPage(1, stride); // already on the last page
  assert.equal(getActivePageIndex(board(id)), 1);
});

test("reanchorPage is a no-op on a canvas", () => {
  const id = S().createBoard("C", null);
  S().setActiveBoard(id);
  const y = S().camera.y;
  S().reanchorPage(1, 1163);
  assert.equal(S().camera.y, y);
  assert.equal(board(id).kind, undefined);
});

// --- bookmarks, insert-before, templates -----

test("insert-before puts the new page ahead of the anchor and selects it", () => {
  const id = freshNotebook(3);
  const ids = getPages(board(id)).map((p) => p.id);

  S().addPageBefore(ids[1]);

  const after = getPages(board(id)).map((p) => p.id);
  assert.equal(after.length, 4);
  assert.equal(after[0], ids[0]);
  assert.equal(after[2], ids[1], "the anchor moved down one");
  assert.equal(S().boards.find((b) => b.id === id)!.activePageId, after[1], "new page is active");
});

test("insert-before on page 1 makes a new first page", () => {
  const id = freshNotebook(2);
  const first = getPages(board(id))[0].id;
  S().addPageBefore(first);
  assert.equal(getPages(board(id))[1].id, first);
  assert.equal(getActivePageIndex(board(id)), 0);
});

test("bookmarking a page shows it in the favourites list with its real number", () => {
  const id = freshNotebook(4);
  const ids = getPages(board(id)).map((p) => p.id);

  S().togglePageBookmark(ids[2]);

  const favs = bookmarkedPages(board(id));
  assert.equal(favs.length, 1);
  assert.equal(favs[0].index, 2, "keeps its position in the notebook, not in the filtered list");
  assert.equal(favs[0].page.id, ids[2]);
});

test("bookmarking toggles off and does not disturb the page", () => {
  const id = freshNotebook(2);
  S().addElement(rect("ink"));
  const pid = getPages(board(id))[1].id;

  S().togglePageBookmark(pid);
  assert.equal(bookmarkedPages(board(id)).length, 1);
  S().togglePageBookmark(pid);
  assert.equal(bookmarkedPages(board(id)).length, 0);
  assert.deepEqual(getBoardElements(board(id)).map((e) => e.id), ["ink"], "ink untouched");
});

test("bookmarking keeps the selection, since the page did not change", () => {
  const id = freshNotebook();

```

```

    S().addElement(rect("a"));
    S().selectAll();
    S().togglePageBookmark(getPages(board(id))[0].id);
    assert.deepEqual(S().selectedIds, ["a"]);
  });

test("saving a page as a template copies its ink and paper", () => {
  const id = freshNotebook();
  S().addElement(rect("drawn"));
  S().setPageBackground(getPages(board(id))[0].id, { pattern: "lines" });

  S().savePageAsTemplate(getPages(board(id))[0].id, "Lecture sheet");

  const t = S().pageTemplates.find((x) => x.name === "Lecture sheet");
  assert.ok(t, "template saved");
  assert.equal(t.elements.length, 1);
  assert.equal(t.background?.pattern, "lines");
  assert.equal(t.width, 794);
  assert.equal(t.height, 1123);
});

test("a page built from a template gets fresh element ids", () => {
  const id = freshNotebook();
  S().addElement(rect("orig"));
  S().savePageAsTemplate(getPages(board(id))[0].id, "T");
  const tid = S().pageTemplates.find((x) => x.name === "T")!.id;

  S().addPageFromTemplate(tid);

  const pages = getPages(board(id));
  assert.equal(pages.length, 2);
  assert.equal(pages[1].elements.length, 1);
  assert.notEqual(
    pages[1].elements[0].id,
    pages[0].elements[0].id,
    "shared ids would make the two pages delete and select as one"
  );
});

test("a template can be used twice without the two pages aliasing", () => {
  const id = freshNotebook();
  S().addElement(rect("orig"));
  S().savePageAsTemplate(getPages(board(id))[0].id, "Twice");
  const tid = S().pageTemplates.find((x) => x.name === "Twice")!.id;

  S().addPageFromTemplate(tid);
  S().addPageFromTemplate(tid);

  const [, a, b] = getPages(board(id));
  assert.notEqual(a.elements[0].id, b.elements[0].id);
  assert.notEqual(a.id, b.id);
});

test("an image template is fitted to the sheet, never cropped, and starts locked", () => {
  const id = freshNotebook();
  // A wide image on a portrait A4 page must fit by width.
  S().createImageTemplate("scan", "", 1588, 1123);

  const t = S().pageTemplates.find((x) => x.name === "scan");
  const el = t.elements[0] as unknown as { width: number; height: number; x: number; y: number; locked: boolean };
  assert.ok(el.width <= 794 + 0.01 && el.height <= 1123 + 0.01, "fits inside the page");
  assert.ok(Math.abs(el.width - 794) < 0.01, "scaled to the limiting dimension");
  assert.ok(el.y > 0, "centred vertically rather than pinned to the top");
  assert.equal(el.locked, true, "a backdrop you write on top of");
});

test("deleting a template leaves existing pages alone", () => {
  const id = freshNotebook();
  S().addElement(rect("orig"));
  S().savePageAsTemplate(getPages(board(id))[0].id, "Doomed");
  const tid = S().pageTemplates.find((x) => x.name === "Doomed")!.id;
  S().addPageFromTemplate(tid);
  const before = getPages(board(id)).length;

  S().deletePageTemplate(tid);

  assert.equal(S().pageTemplates.find((x) => x.id === tid), undefined);
});

```

```
    assert.equal(getPages(board(id)).length, before, "the page it made survives");
  });

test("adding from an unknown template id is a no-op", () => {
  const id = freshNotebook(2);
  const before = getPages(board(id)).length;
  S().addPageFromTemplate("nope");
  assert.equal(getPages(board(id)).length, before);
});

// --- split view -----

test("opening a board beside the current one sets the reference pane", () => {
  const a = freshNotebook();
  const b = S().createBoard("Reference", null);
  S().setActiveBoard(a);

  S().setSplitBoard(b);
  assert.equal(S().splitBoardId, b);
});

test("a board cannot be split against itself", () => {
  const a = freshNotebook();
  S().setActiveBoard(a);
  S().setSplitBoard(null);

  S().setSplitBoard(a);
  assert.equal(S().splitBoardId, null, "the pane would not follow your edits, so the halves would disagree");
});

test("swapping moves the documents to the other side, editability unchanged", () => {
  const a = freshNotebook();
  const b = S().createBoard("Other", null);
  S().setSplitPair(a, b);
  S().setSplitRatio(0.3);

  S().swapSplit();

  assert.equal(S().splitSide, "right", "the editable document moved to the right slot");
  assert.equal(S().activeBoardId, a, "and is still the same document – this is not a focus change");
  assert.equal(S().splitBoardId, b);
  assert.ok(
    Math.abs(S().splitRatio - 0.7) < 1e-9,
    "the left slot is always splitRatio, so flipping it keeps each document its own width"
  );
});

test("swapping twice returns everything exactly where it started", () => {
  const a = freshNotebook();
  const b = S().createBoard("Other-b", null);
  S().setSplitPair(a, b);
  S().setSplitRatio(0.35);

  S().swapSplit();
  S().swapSplit();

  assert.equal(S().splitSide, "left");
  assert.equal(S().activeBoardId, a);
  assert.equal(S().splitBoardId, b);
  assert.ok(Math.abs(S().splitRatio - 0.35) < 1e-9);
});

test("swapping is not the same action as focusing the other pane", () => {
  const a = freshNotebook();
  const b = S().createBoard("Other-c", null);
  S().setSplitPair(a, b);

  S().swapSplit();
  const afterSwap = { active: S().activeBoardId, side: S().splitSide };

  S().setSplitPair(a, b);
  S().focusSplitSide("right");
  const afterFocus = { active: S().activeBoardId, side: S().splitSide };

  assert.equal(afterSwap.side, afterFocus.side, "both end up with the right slot editable-side");
  assert.notEqual(
    afterSwap.active,
    afterFocus.active,
  );
});
```

```
    "but swap keeps the same document editable while focus hands over to the other"
  );
});

test("swapping with no split open does nothing", () => {
  const a = freshNotebook();
  S().setActiveBoard(a);
  S().setSplitBoard(null);
  S().swapSplit();
  assert.equal(S().activeBoardId, a);
  assert.equal(S().splitBoardId, null);
});

test("neither pane can be dragged away to nothing", () => {
  S().setSplitRatio(0.01);
  assert.equal(S().splitRatio, 0.2, "clamped, or the split looks broken with no way back");
  S().setSplitRatio(5);
  assert.equal(S().splitRatio, 0.8);
});

test("closing the split clears the reference board", () => {
  const a = freshNotebook();
  const b = S().createBoard("Other2", null);
  S().setActiveBoard(a);
  S().setSplitBoard(b);
  S().setSplitBoard(null);
  assert.equal(S().splitBoardId, null);
});

test("setSplitPair puts a chosen document on each side", () => {
  const a = S().createBoard("Left doc", null);
  const b = S().createBoard("Right doc", null);

  S().setSplitPair(a, b);

  assert.equal(S().activeBoardId, a, "the left document is the editable one");
  assert.equal(S().splitBoardId, b);
  assert.equal(S().splitSide, "left");
});

test("setSplitPair refuses the same document on both sides", () => {
  const a = S().createBoard("Only", null);
  S().setSplitBoard(null);
  S().setSplitPair(a, a);
  assert.equal(S().splitBoardId, null);
});

test("focusing the other pane makes it editable without moving either side", () => {
  const a = S().createBoard("L", null);
  const b = S().createBoard("R", null);
  S().setSplitPair(a, b);

  S().focusSplitSide("right");

  assert.equal(S().activeBoardId, b, "the right document is now editable");
  assert.equal(S().splitBoardId, a, "and the left one becomes the reference");
  assert.equal(S().splitSide, "right", "the editable pane is the right-hand slot");
});

test("focusing the side that is already editable does nothing", () => {
  const a = S().createBoard("L2", null);
  const b = S().createBoard("R2", null);
  S().setSplitPair(a, b);
  S().focusSplitSide("left");
  assert.equal(S().activeBoardId, a);
  assert.equal(S().splitBoardId, b);
});

test("each side keeps its own camera when focus moves between them", () => {
  const a = S().createBoard("CamA", null);
  const b = S().createBoard("CamB", null);
  S().setSplitPair(a, b);

  S().setCamera({ x: 111, y: 222, zoom: 1 }); // on A
  S().focusSplitSide("right");
  S().setCamera({ x: 333, y: 444, zoom: 1 }); // on B
  S().focusSplitSide("left");
```

```

    assert.equal(Math.round(S().camera.x), 111, "returning to A restores A's view");
    S().focusSplitSide("right");
    assert.equal(Math.round(S().camera.x), 333, "and B keeps its own");
  });

  test("focus does not switch while a text editor is open", () => {
    const a = S().createBoard("EditL", null);
    const b = S().createBoard("EditR", null);
    S().setSplitPair(a, b);
    S().addElement({ ...rect("t"), type: "text", text: "half typed" } as never);
    S().setEditingId("t");

    S().focusSplitSide("right");

    assert.equal(S().activeBoardId, a, "switching would discard the open edit");
    assert.equal(S().editingId, "t");
    S().setEditingId(null);
  });

  test("focus switches once the editor is closed", () => {
    const a = S().createBoard("EditL2", null);
    const b = S().createBoard("EditR2", null);
    S().setSplitPair(a, b);
    S().setEditingId(null);
    S().focusSplitSide("right");
    assert.equal(S().activeBoardId, b);
  });

  // --- components must read the same element list the store writes -----
  //
  // The store was funnelled through getBoardElements when notebooks landed, but
  // the COMPONENTS were not: they kept reading `board.elements`, which is empty
  // for a notebook. These pin the store-side contract those components rely on.

  test("a notebook's selection is findable through the funnel, not board.elements", () => {
    const id = freshNotebook();
    S().addElement(rect("sel"));
    S().setSelected(["sel"]);

    assert.deepEqual(board(id).elements, [], "the canvas field is empty for a notebook");
    assert.ok(
      getBoardElements(board(id)).find((e) => e.id === "sel"),
      "anything reading board.elements would show no properties for this selection"
    );
  });

  test("the editing target of a notebook is findable the same way", () => {
    const id = freshNotebook();
    S().addElement({ ...rect("t1"), type: "text", text: "hi" } as never);
    S().setEditingId("t1");

    assert.ok(
      getBoardElements(board(id)).find((e) => e.id === S().editingId),
      "otherwise double-clicking text in a notebook opens nothing"
    );
    S().setEditingId(null);
  });

  test("a document-wide count sees every page, so a notebook never reads as empty", () => {
    const id = freshNotebook();
    S().addElement(rect("p1"));
    S().addPage();
    S().addElement(rect("p2"));

    assert.equal(board(id).elements.length, 0, "the canvas field would report 0 elements");
    assert.equal(allElements(board(id)).length, 2);
  });

  // -----
  // The camera cannot lose the page
  // -----
  // Panning sideways used to carry on forever, leaving you in empty space with
  // the sheet nowhere on screen and no way back except zoom-to-fit. A whiteboard
  // should do that; a notebook should not.

  test("panning a notebook sideways forever cannot lose the page", () => {
    const id = freshNotebook();
    const { width } = pageSizeOf(board(id));

```

```

for (let i = 0; i < 50; i++) S().panCamera(-400, 0);
const far = S().camera.x;
assert.ok(
  far <= width * 1.5 + 0.001,
  `camera.x ran to ${far}; it must stop near the page (width ${width})`
);

for (let i = 0; i < 50; i++) S().panCamera(400, 0);
const near = S().camera.x;
assert.ok(
  near >= -width * 0.5 - 0.001,
  `camera.x ran to ${near} the other way`
);
});

test("the vertical camera is NOT bounded – that is how pages turn", () => {
  const id = freshNotebook(3);
  const { height } = pageSizeOf(board(id));
  S().setCamera({ y: height * 2 + PAGE_GAP * 2 });
  assert.ok(
    S().camera.y > height,
    "clamping Y would stop the notebook scrolling past the first sheet"
  );
});

test("a plain whiteboard stays unbounded", () => {
  const id = S().createBoard("Unbounded", null);
  S().setActiveBoard(id);
  for (let i = 0; i < 50; i++) S().panCamera(-400, 0);
  assert.ok(
    S().camera.x > 5000,
    `an infinite canvas must still pan forever, got ${S().camera.x}`
  );
});

test("panning DOWN past the last page cannot lose the notebook", () => {
  const id = freshNotebook(3);
  const { height } = pageSizeOf(board(id));
  const stride = height + PAGE_GAP;

  for (let i = 0; i < 60; i++) S().panCamera(0, -600); // drag upward, forever
  const far = S().camera.y;
  // Active page is index 0, so the document ends at 2 strides + one page.
  assert.ok(
    far <= 2 * stride + height + height * 0.5 + 0.001,
    `camera.y ran to ${far}: the sheets would be off screen with no way back`
  );
});

test("panning UP past the first page cannot lose the notebook", () => {
  const id = freshNotebook(3);
  const { height } = pageSizeOf(board(id));
  const stride = height + PAGE_GAP;
  // addPage selects the page it adds, so page 3 is the anchor here – the
  // document extends two strides ABOVE the origin, not zero.
  const activeIndex = getActivePageIndex(board(id));
  const top = -(activeIndex * stride) - height * 0.5;

  for (let i = 0; i < 60; i++) S().panCamera(0, 600);
  assert.ok(
    S().camera.y >= top - 0.001,
    `camera.y ran to ${S().camera.y}, past the top bound ${top}`
  );
});

test("the bound follows the ACTIVE page, so later pages stay reachable", () => {
  const id = freshNotebook(5);
  const { height } = pageSizeOf(board(id));
  const stride = height + PAGE_GAP;
  // On the last page the document extends UPWARD from the anchor instead.
  const pages = getPages(board(id));
  S().setActivePage(pages[pages.length - 1].id);
  for (let i = 0; i < 60; i++) S().panCamera(0, 600);
  assert.ok(
    S().camera.y >= -(4 * stride) - height * 0.5 - 0.001,
    "scrolling back to page 1 from the last page must still be possible"
  );
});

```

```

});

// -----
// One Select tool with a "pick up" filter
// -----

test("each element kind lands in exactly one pick-up category", () => {
  assert.equal(selectCategoryOf({ type: "stroke" }), "ink");
  assert.equal(selectCategoryOf({ type: "text" }), "text");
  assert.equal(selectCategoryOf({ type: "sticky" }), "text");
  assert.equal(selectCategoryOf({ type: "image" }), "images");
  for (const t of ["rectangle", "ellipse", "diamond", "triangle", "star", "cloud", "line", "arrow"]) {
    assert.equal(selectCategoryOf({ type: t }), "shapes", t);
  }
  assert.equal(selectCategoryOf({ type: "connection" }), null, "connections are never swept up");
});

test("a malformed saved filter falls back to everything, never to nothing", () => {
  for (const bad of [undefined, null, 42, "ink", [], ["nonsense"], [{}]]) {
    assert.equal(effectiveSelectFilter(bad).size, 4, `for ${JSON.stringify(bad)}`);
  }
  const f = effectiveSelectFilter(["ink", "from-a-newer-build"]);
  assert.deepEqual([...f], ["ink"], "unknown entries are dropped, known ones kept");
});

test("Select All obeys the filter", () => {
  const id = freshNotebook();
  const stroke = { ...rect("s1"), type: "stroke", points: [[0, 0, 0.5]] } as unknown as WhiteboardElement;
  const text = { ...rect("t1"), type: "text", text: "hi" } as unknown as WhiteboardElement;
  S().addElement(stroke, true);
  S().addElement(text, true);
  S().addElement(rect("r1"), true);

  S().setTouchSettings({ selectFilter: ["ink"] });
  S().selectAll();
  assert.deepEqual(S().selectedIds, ["s1"], "only the handwriting");

  S().setTouchSettings({ selectFilter: ["text", "shapes"] });
  S().selectAll();
  assert.deepEqual(new Set(S().selectedIds), new Set(["t1", "r1"]));

  S().setTouchSettings({ selectFilter: undefined });
  S().selectAll();
  assert.equal(S().selectedIds.length, 3, "no filter selects everything");
  assert.ok(getBoardElements(board(id)).length >= 3);
});

test("passesSelectFilter rejects kinds that are switched off", () => {
  const onlyInk = effectiveSelectFilter(["ink"]);
  assert.equal(passesSelectFilter({ type: "stroke" }, onlyInk), true);
  assert.equal(passesSelectFilter({ type: "image" }, onlyInk), false);
});

// -----
// The "+" between pages
// -----

/** Where page `j` is on screen (world y minus camera y), in the anchored frame. */
function screenYOf(id: string, j: number): number {
  const b = board(id);
  const stride = pageSizeOf(b).height + PAGE_GAP;
  return (j - getActivePageIndex(b)) * stride - S().camera.y;
}

test("insertPageAt(0) adds a page ABOVE the first one", () => {
  const id = freshNotebook(3);
  const first = getPages(board(id))[0].id;
  S().insertPageAt(0);
  const pages = getPages(board(id));
  assert.equal(pages.length, 4);
  assert.equal(pages[1].id, first, "old first page pushed down to 2nd");
  assert.equal(getActivePageIndex(board(id)), 0, "the new page is active");
});

test("insertPageAt(length) adds a page BELOW the last one", () => {
  const id = freshNotebook(3);
  const last = getPages(board(id))[2].id;
  S().insertPageAt(3);

```

```

    const pages = getPages(board(id));
    assert.equal(pages[2].id, last);
    assert.equal(getActivePageIndex(board(id)), 3);
  });

test("insertPageAt(k) puts it BETWEEN pages k-1 and k", () => {
  const id = freshNotebook(4);
  const [a, b, c, d] = getPages(board(id)).map((p) => p.id);
  S().insertPageAt(2);
  const ids = getPages(board(id)).map((p) => p.id);
  assert.deepEqual([ids[0], ids[1], ids[3], ids[4]], [a, b, c, d]);
  assert.equal(getActivePageIndex(board(id)), 2);
});

test("inserting between pages does not move anything above it on screen", () => {
  const id = freshNotebook(4); // page 4 active
  S().setCamera({ y: 300 });
  const before0 = screenYOf(id, 0);
  const before1 = screenYOf(id, 1);
  S().insertPageAt(2); // between pages 2 and 3
  assert.ok(Math.abs(screenYOf(id, 0) - before0) < 1e-6, "page 1 stayed put");
  assert.ok(Math.abs(screenYOf(id, 1) - before1) < 1e-6, "page 2 stayed put");
  // And the new page sits right where page 3 used to begin.
  const b = board(id);
  const stride = pageSizeOf(b).height + PAGE_GAP;
  assert.ok(Math.abs(screenYOf(id, 2) - (before1 + stride)) < 1e-6, "new sheet fills the gap");
});

test("goToPage shows the TOP of the page", () => {
  const id = freshNotebook(3);
  S().setCamera({ zoom: 0.5 });
  const target = getPages(board(id))[0].id;
  S().goToPage(target, 1000);
  assert.equal(getActivePageIndex(board(id)), 0);
  // Page top (world 0) sits 24 px below the top of a 1000 px screen.
  const z = S().camera.zoom;
  const screenTop = (0 - S().camera.y) * z + 1000 / 2;
  assert.ok(Math.abs(screenTop - 24) < 1e-6, `page top at ${screenTop}px`);
});

test("movePage moves the page you DRAGGED to where you dropped it", () => {
  const id = freshNotebook(5);
  const ids = getPages(board(id)).map((p) => p.id);
  S().movePage(ids[0], 3); // drag page 1 to position 4
  const after = getPages(board(id)).map((p) => p.id);
  assert.equal(after[3], ids[0], "page 1 landed at position 4");
  assert.deepEqual(after.filter((x) => x !== ids[0]), ids.slice(1), "others keep their order");
});

test("zoomToFit on a notebook fits the sheet width, not the ink", () => {
  // Fitting ink bounds on a sparse page lands at a random zoom per page; the
  // sheet is the stable frame.
  const id = freshNotebook();
  S().addElement(rect("dot"));
  S().zoomToFit({ width: 800, height: 1200 });
  const dim = pageSizeOf(board(id));
  const cam = S().camera;
  assert.ok(
    Math.abs(cam.zoom - (800 - 48) / dim.width) < 1e-9,
    `zoom fills the width, got ${cam.zoom}`
  );
  assert.equal(cam.x, dim.width / 2);
  assert.equal(cam.y, dim.height / 2, "stays anchored on the active page");
});

test("link legend visibility persists like the minimap", () => {
  S().setLegendOpen(true);
  try {
    const snap = S().getPersistedState() as unknown as Record<string, unknown>;
    assert.equal(snap.legendOpen, true, "saved with the workspace");
    const back = migrateState({ boards: [], ...snap, version: 1 } as never);
    assert.equal(
      (back as unknown as Record<string, unknown>).legendOpen,
      true,
      "survives the load path"
    );
  } finally {
    S().setLegendOpen(false);
  }
});

```

```

    }
  });

test("zoom controls visibility persists and defaults to shown", () => {
  assert.equal(S().zoomDockOpen, true, "shown by default");
  S().setZoomDockOpen(false);
  try {
    const snap = S().getPersistedState() as unknown as Record<string, unknown>;
    assert.equal(snap.zoomDockOpen, false, "saved with the workspace");
    const back = migrateState({ boards: [], ...snap, version: 1 } as never);
    assert.equal(
      (back as unknown as Record<string, unknown>).zoomDockOpen,
      false,
      "survives the load path"
    );
  } finally {
    S().setZoomDockOpen(true);
  }
});

test("new boards start with the app default paper", () => {
  const id = S().createBoard("DefTest", null);
  assert.equal(board(id).background?.pattern, "plain");
});

test("set as app default changes future boards, never past ones", () => {
  const before = S().createBoard("Before", null);
  S().setDefaultPaper({ pattern: "grid", size: 40 });
  try {
    const after = S().createBoard("After", null);
    assert.equal(board(after).background?.pattern, "grid");
    assert.equal(board(after).background?.size, 40);
    assert.equal(board(before).background?.pattern, "plain", "past boards keep theirs");
  } finally {
    S().setDefaultPaper({ pattern: "plain", size: 24 });
  }
});

test("board paper rewrites all pages only when asked", () => {
  const id = freshNotebook(2);
  S().setBoardBackground(id, { pattern: "grid" });
  assert.equal(board(id).background?.pattern, "grid");
  assert.equal(
    getPages(board(id))[0].background?.pattern,
    "plain",
    "pages keep their own paper by default"
  );
  S().setBoardBackground(id, { pattern: "lines" }, { allPages: true });
  assert.equal(getPages(board(id))[0].background?.pattern, "lines");
  assert.equal(getPages(board(id))[1].background?.pattern, "lines");
});

test("stamping the current board paper onto all pages matches every page", () => {
  // What the Settings "Apply to all pages" toggle does when flipped on.
  const id = freshNotebook(3);
  S().setBoardBackground(id, { pattern: "grid", size: 40 });
  const bg = board(id).background!;
  S().setBoardBackground(id, { ...bg }, { allPages: true });
  for (const p of getPages(board(id))) {
    assert.equal(p.background?.pattern, "grid");
    assert.equal(p.background?.size, 40);
  }
});

test("sidebar filing: boards and folders move, cycles and ghosts rejected", () => {
  const fa = S().createFolder("FileA", null);
  const fb = S().createFolder("FileB", fa);
  const id = S().createBoard("Filing", null);
  S().moveBoard(id, fa);
  assert.equal(board(id).folderId, fa, "board files into the folder");
  S().moveBoard(id, null);
  assert.equal(board(id).folderId, null, "board files back to the top level");
  // Into its own descendant: rejected while the nesting holds.
  S().moveFolder(fa, fb);
  assert.equal(
    S().folders.find((f) => f.id === fa)?.parentId,
    null,
    "a folder cannot move into its own subtree"
  );
});

```

```

    );
    S().moveFolder(fa, fa);
    assert.equal(
      S().folders.find((f) => f.id === fa)!.parentId,
      null,
      "a folder cannot move into itself"
    );
    S().moveFolder(fb, null);
    assert.equal(S().folders.find((f) => f.id === fb)!.parentId, null);
    // Ghosts: no throw, nothing changes.
    S().moveBoard("ghost-board", fa);
    S().moveBoard(id, "ghost-folder");
    S().moveFolder("ghost-folder", fa);
    assert.equal(board(id).folderId, null);
  });

console.log(`\n ${passed} passed, ${failed} failed\n`);
if (failed > 0) process.exit(1);

=====
// === END FILE: tests/notebook_store.test.ts ===
=====

=====
// === START FILE: tests/office_import.test.ts ===
// File 167 of 194 | Lines: 150 | Size: 7318 bytes
=====

/**
 * Word / PowerPoint import: the parts that must hold without a browser.
 * - Which files are Word, PowerPoint, or the old binary formats.
 * - The slide clean-up that fixed a real Hebrew deck: a text box whose
 *   position lives on the layout gets it written in, and two boxes sharing a
 *   layout slot are separated (the renderer drew the main text as a column
 *   one letter wide).
 */

import assert from "node:assert/strict";
import {
  DOCUMENT_ACCEPT,
  hasRtl,
  normalizeSlideXml,
  officeKindOf,
  placeholderGeometry,
  restoreGlobals,
  pictureFromFilledBox,
  preferFallbackContent,
} from "../src/lib/whiteboard/officeImport";

const NL = String.fromCharCode(10);
let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`   ${err as Error}.message`);
  }
}

console.log(NL + "■■■ Office import ■■■");

test("recognises Word, PowerPoint and the old formats by name or type", () => {
  assert.equal(officeKindOf({ name: "Lecture 3.pptx" }), "pptx");
  assert.equal(officeKindOf({ name: "show.PPSX" }), "pptx");
  assert.equal(officeKindOf({ name: "notes.docx" }), "docx");
  assert.equal(
    officeKindOf({ name: "x", type: "application/vnd.openxmlformats-officedocument.presentationml.presentation" }),
    "pptx"
  );
  assert.equal(officeKindOf({ name: "old.doc" }), "legacy");
  assert.equal(officeKindOf({ name: "old.ppt" }), "legacy");
});

```

```

    assert.equal(officeKindOf({ name: "paper.pdf", type: "application/pdf" }), null);
    assert.equal(officeKindOf({ name: "photo.jpg", type: "image/jpeg" }), null);
  });

test("the file picker offers PDF, Word and PowerPoint", () => {
  for (const ext of [".pdf", ".docx", ".pptx"]) assert.ok(DOCUMENT_ACCEPT.includes(ext), ext);
});

test("Hebrew and Arabic are detected; Latin text is not", () => {
  assert.equal(hasRtl("■■■■■ ■■■■■"), true);
  assert.equal(hasRtl("■■■■■"), true);
  assert.equal(hasRtl("O(log n) and AVL"), false);
});

const LAYOUT = `

```

```
// The Office-2010 maths version drew as a one-letter column; the fallback
// is PowerPoint's rendering of the same box.
const slide =
  '<p:sld><mc:AlternateContent><mc:Choice Requires="a14"><p:sp><p:txBody><a14:m>0(n)</a14:m></p:txBody></p:s
  p></mc:Choice>' +
  '<mc:Fallback><p:sp><p:spPr><a:blipFill><a:blip r:embed="rId2"/></a:blipFill></p:spPr></p:sp></mc:Fallback
  ></mc:AlternateContent></p:sld>';
const out = preferFallbackContent(slide);
assert.ok(!out.includes("AlternateContent") && !out.includes("a14:m"), out);
assert.match(out, /<a:blip r:embed="rId2"/>/);
// No fallback: the choice is kept rather than losing the shape.
const onlyChoice = '<mc:AlternateContent><mc:Choice Requires="x"><p:sp>kept</p:sp></mc:Choice></mc:Alternate
  Content>';
assert.equal(preferFallbackContent(onlyChoice), "<p:sp>kept</p:sp>");
assert.equal(preferFallbackContent("<p:sld/>"), "<p:sld/>");
});
```

```
test("an equation box's fallback picture becomes a real picture, in place", () => {
  const box =
    '<p:sp><p:nvSpPr><p:cNvPr id="3" name="Content Placeholder 2"><p:nvPr><p:ph idx="1"/></p:nvPr></p:nvSpPr>
    ' +
    '<p:spPr><p:txBody><a:p><a:r><a:t> </a:t></a:r></a:p></p:txBody></p:sp>';
  const fallback = box.replace("<p:spPr>", '<p:spPr><a:blipFill><a:blip r:embed="rId2"/><a:stretch><a:fillRec
    t l="-1273"/></a:stretch></a:blipFill></p:spPr>');
  const out = normalizeSlideXml('<p:sld>${fallback}</p:sld>', LAYOUT);
  assert.match(out, /<p:pic>/);
  assert.match(out, /<p:blipFill><a:blip r:embed="rId2"/><a:stretch><a:fillRect l="-1273"/></a:stretch></p
    :blipFill>/);
  assert.match(out, /<a:off x="100" y="600"/>/, "at the layout slot's position");
  // A box with real text is never turned into a picture.
  const texty = fallback.replace("<a:t> </a:t>", "<a:t>hello</a:t>");
  assert.equal(pictureFromFilledBox(texty.replace("<p:spPr>", '<p:spPr><a:xfrm><a:off x="1" y="1"/><a:ext cx="
    1" cy="1"/></a:xfrm>')), null);
});
```

```
console.log(` ${passed} passed, ${failed} failed`);
if (failed > 0) process.exit(1);
```

```
=====
// === END FILE: tests/office_import.test.ts ===
=====
```

```
=====
// === START FILE: tests/onedrive.test.ts ===
// File 168 of 194 | Lines: 214 | Size: 6655 bytes
=====
```

```
/**
 * OneDrive sign-in storage behaviour – no network.
 *
 * Fakes `window` / `localStorage` / `sessionStorage` / `fetch` on globalThis
 * (the module reads the bare globals, exactly like a browser) and covers:
 *
 * 1. A verifier that cannot be saved fails BEFORE the Microsoft redirect,
 *    with an actionable message, and never navigates.
 * 2. Working storage navigates to Microsoft and stores the verifier.
 * 3. signOut clears the verifier from sessionStorage (where it lives), not
 *    localStorage (where it never was), plus tokens and the return URL.
 * 4. A failing token write does not reject sign-in completion.
 */
```

```
import assert from "node:assert/strict";
```

```
import {
  beginSignIn,
  completeSignInIfRedirected,
  signOut,
} from "../src/lib/whiteboard/onedrive";
```

```
const LS_TOKENS = "pronotes.ms.tokens";
const LS_VERIFIER = "pronotes.ms.pkce";
const LS_RETURN = "pronotes.ms.return";
```

```
let passed = 0;
let failed = 0;
async function testAsync(name: string, fn: () => Promise<void>): Promise<void> {
  try {
```

```

    await fn();
    passed++;
    console.log(`  ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(`  ✗ ${name}\n      ${err as Error}.message`);
  }
}

// --- fakes -----

function makeStore(throwOnSet: boolean) {
  const map = new Map<string, string>();
  const removed: string[] = [];
  return {
    map,
    removed,
    getItem: (k: string) => (map.has(k) ? (map.get(k) as string) : null),
    setItem: (k: string, v: string) => {
      if (throwOnSet) throw new Error("QuotaExceededError: storage is blocked");
      map.set(k, String(v));
    },
    removeItem: (k: string) => {
      removed.push(k);
      map.delete(k);
    },
  };
}

type Store = ReturnType<typeof makeStore>;

const g = globalThis as unknown as Record<string, unknown>;
let assignedUrl: string | null = null;

interface Installed {
  prev: Record<string, unknown>;
  local: Store;
  session: Store;
}

function install(opts: {
  localThrowOnSet?: boolean;
  sessionThrowOnSet?: boolean;
  href?: string;
}): Installed {
  const prev: Record<string, unknown> = {
    window: g.window,
    localStorage: g.localStorage,
    sessionStorage: g.sessionStorage,
    fetch: g.fetch,
  };
  assignedUrl = null;
  const local = makeStore(opts.localThrowOnSet ?? false);
  const session = makeStore(opts.sessionThrowOnSet ?? false);
  const href = opts.href ?? "https://localhost/";
  g.localStorage = local;
  g.sessionStorage = session;
  g.window = {
    location: {
      href,
      origin: "https://localhost",
      pathname: "/",
      assign: (url: string) => {
        assignedUrl = url;
      },
    },
  };
  history: { replaceState: () => undefined },
};
process.env.NEXT_PUBLIC_MS_CLIENT_ID = "test-client-id";
return { prev, local, session };
}

function restore(installed: Installed): void {
  g.window = installed.prev.window;
  g.localStorage = installed.prev.localStorage;
  g.sessionStorage = installed.prev.sessionStorage;
  g.fetch = installed.prev.fetch;
  delete process.env.NEXT_PUBLIC_MS_CLIENT_ID;
}

```

```

}

// --- tests -----

async function main(): Promise<void> {
  await testAsync(
    "beginSignIn rejects with the storage message and never navigates when the verifier cannot be saved",
    async () => {
      const installed = install({ sessionThrowOnSet: true });
      try {
        await assert.rejects(beginSignIn(), /blocking storage/);
        assert.equal(
          assignedUrl,
          null,
          "must not send the user to Microsoft when the return trip cannot succeed"
        );
      } finally {
        restore(installed);
      }
    }
  );

  await testAsync(
    "beginSignIn with working storage navigates to Microsoft and stores the verifier",
    async () => {
      const installed = install({});
      try {
        await beginSignIn();
        assert.ok(
          assignedUrl?.startsWith("https://login.microsoftonline.com/"),
          `expected a Microsoft authorize URL, got ${assignedUrl}`
        );
        assert.ok(
          assignedUrl?.includes("code_challenge="),
          "the PKCE challenge must travel with the redirect"
        );
        const verifier = installed.session.map.get(LS_VERIFIER);
        assert.ok(verifier, "the verifier must be stored before redirecting");
        assert.equal(
          installed.session.map.get(LS_RETURN),
          "https://localhost/",
          "the return URL is stored alongside the verifier"
        );
      } finally {
        restore(installed);
      }
    }
  );

  await testAsync(
    "signOut clears tokens from localStorage and the verifier from sessionStorage",
    async () => {
      const installed = install({});
      try {
        installed.local.map.set(LS_TOKENS, JSON.stringify({ accessToken: "a" }));
        installed.session.map.set(LS_VERIFIER, "verifier");
        installed.session.map.set(LS_RETURN, "https://localhost/");
        signOut();
        assert.equal(installed.local.map.has(LS_TOKENS), false, "tokens are gone");
        assert.equal(installed.session.map.has(LS_VERIFIER), false, "verifier is gone");
        assert.equal(installed.session.map.has(LS_RETURN), false, "return URL is gone");
        assert.ok(
          !installed.local.removed.includes(LS_VERIFIER),
          "must not clear the verifier from localStorage – it was never there"
        );
      } finally {
        restore(installed);
      }
    }
  );

  await testAsync(
    "a failing token write does not reject sign-in completion",
    async () => {
      const installed = install({
        localThrowOnSet: true,
        href: "https://localhost/?code=test-code",
      });
    }
  );
}

```

```

    try {
      installed.session.map.set(LS_VERIFIER, "verifier");
      g.fetch = async (url: unknown) => {
        if (String(url).includes("/oauth2/")) {
          return {
            ok: true,
            json: async () => ({ access_token: "a", refresh_token: "r", expires_in: 3600 }),
          };
        }
        return {
          ok: true,
          json: async () => ({ displayName: "Tester", mail: "t@example.com" }),
        };
      };
      // storeTokens throws on both writes inside; completion must still resolve.
      const completed = await completeSignInIfRedirected();
      assert.equal(completed, true, "sign-in completes even though tokens could not be persisted");
    } finally {
      restore(installed);
    }
  }
};

console.log(`\n  ${passed} passed, ${failed} failed\n`);
if (failed > 0) process.exit(1);
}

void main();

=====
// === END FILE: tests/onedrive.test.ts ===
=====

=====
// === START FILE: tests/pdf_text.test.ts ===
// File 169 of 194 | Lines: 289 | Size: 12793 bytes
=====

/**
 * Selectable, searchable text of PDF / Word / PowerPoint pages - the parts that
 * must hold without a browser.
 * - Reading order: lines kept in document order (two columns never interleave),
 *   runs of a Hebrew line read right to left with Latin/numbers kept LTR,
 *   glyph-per-item and whole-string visual-order Hebrew turned back.
 * - Hit testing, word / range geometry, the strokes a highlight makes.
 * - Search folding (case, accents, niqqud, harakat, presentation forms) and
 *   snippets whose match index is right.
 * - pdf.js items -> boxes; PowerPoint letter-by-letter drawing glued back.
 */

import assert from "node:assert/strict";
import {
  caretAt,
  dirOf,
  findAll,
  foldForSearch,
  groupLines,
  layoutPage,
  lineSegments,
  looksVisualOrder,
  markStrokes,
  mergeFragments,
  offsetAt,
  pageBoxOnSheet,
  rangeRects,
  runsFromPdfItems,
  selectedText,
  snippetAround,
  visualToLogical,
  wordRangeAt,
  type TextRun,
} from "../src/lib/whiteboard/pdfText";
import {
  mergeFragments as mergeFragmentsX,
  orderLine as orderLineX,
  queryTerms as queryTermsX,
  visualToLogical as visualToLogicalX,

```

```

} from "../src/lib/whiteboard/pdfText";
import { PEN_TYPE_PRESETS } from "../src/lib/whiteboard/types";

const NL = String.fromCharCode(10);
let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`   ${err as Error}.message`);
  }
}

const run = (t: string, x: number, y: number, w: number, h = 0.02): TextRun => ({ t, x, y, w, h });
const page = (runs: TextRun[]) => ({ w: 1, h: 1, runs });
const near = (a: number, b: number, eps = 1e-6) => assert.ok(Math.abs(a - b) < eps, `${a} != ${b}`);

console.log(NL + "■■■ PDF / Office page text ■■■");

test("direction of a run: Hebrew, Arabic, Latin, digits, punctuation", () => {
  assert.equal(dirOf("■■■■"), "r");
  assert.equal(dirOf("■■■■■"), "r");
  assert.equal(dirOf("hello"), "l");
  assert.equal(dirOf("2024"), "l");
  assert.equal(dirOf("-:"), "n");
});

test("words on one line join with a space; touching pieces of a word do not", () => {
  const l = layoutPage(page([run("Hel", 0.1, 0.1, 0.03), run("lo", 0.13, 0.1, 0.02), run("world", 0.17, 0.1, 0.05)]));
  assert.equal(l.text, "Hello world");
});

test("two columns keep document order: the left column is not interleaved with the right", () => {
  const runs = [
    run("left1", 0.1, 0.1, 0.2),
    run("left2", 0.1, 0.13, 0.2),
    run("right1", 0.6, 0.1, 0.2),
    run("right2", 0.6, 0.13, 0.2),
  ];
  assert.equal(groupLines(runs).length, 4);
  assert.equal(layoutPage(page(runs)).text, ["left1", "left2", "right1", "right2"].join(NL));
});

test("Hebrew stored one glyph per item, left to right, reads right to left", () => {
  // ■■■■ drawn as ■ ■ ■ ■ from left to right, which is how it looks.
  const glyphs = ["■", "■", "■", "■"].map((g, i) => run(g, 0.5 + i * 0.01, 0.2, 0.01));
  assert.equal(layoutPage(page(glyphs)).text, "■■■■");
});

test("a Hebrew line with English words and a number reads in the right order", () => {
  // Reading order: "■■■ ■■ Figure 3 ■■■■■"; on screen (left to right): ■■■■■ | Figure 3 | ■■ ■■■
  const runs = [run("■■■■■", 0.1, 0.3, 0.08), run("Figure", 0.2, 0.3, 0.08), run("3", 0.29, 0.3, 0.01), run("■", 0.32, 0.3, 0.04), run("■■■", 0.38, 0.3, 0.05)];
  assert.equal(layoutPage(page(runs)).text, "■■■ ■■ Figure 3 ■■■■■");
});

test("Office text keeps the order it was written in, whatever the renderer's word positions", () => {
  // The slide renderer placed Hebrew words left to right in reading order.
  const runs = [run("■■■■■", 0.05, 0.1, 0.1), run("-", 0.16, 0.1, 0.01), run("■■■■■", 0.2, 0.1, 0.1)];
  assert.equal(layoutPage({ w: 1, h: 1, runs, ordered: true }).text, "■■■■■ - ■■■■■");
  assert.equal(layoutPage(page(runs)).text, "■■■■■ - ■■■■■");
});

test("a number ending a Hebrew sentence keeps its full stop after it", () => {
  // Drawn ". 12 ■■■■ ■■■"; pdf.js hands the number over as ".12".
  const runs = [run(".12", 0.1, 0.4, 0.03), run("■■■■", 0.14, 0.4, 0.05), run("■■■", 0.2, 0.4, 0.04)];
  assert.equal(layoutPage(page(runs)).text, "■■■ ■■■■ 12.");
});

test("Arabic presentation forms read (and copy) as plain letters", () => {
  const l = layoutPage(page([run("\uFEDF\uFEDE\uFEDE", 0.1, 0.1, 0.05)]));
  assert.equal(l.text, "■■■■");
}

```

```

    assert.equal(findAll(l.text, "■").length, 1);
  });

test("visual-order strings are turned back, numbers keep their order", () => {
  assert.equal(visualToLogical("12 ■■■■"), "■■■■ 12");
  assert.equal(visualToLogical("3.14 = ■■"), "■■ = 3.14");
  assert.equal(looksVisualOrder("■■■■ ■■■■ ■■■"), true);
  assert.equal(looksVisualOrder("■■■■ ■■■■ ■■■"), false);
  // A page of reversed words is repaired on layout.
  const l = layoutPage(page([run("■■■■", 0.1, 0.1, 0.1), run("■■■■", 0.25, 0.1, 0.1)]));
  assert.equal(l.text, "■■■■ ■■■■");
});

test("hit testing: left-to-right from the left edge, right-to-left from the right edge", () => {
  const l = layoutPage(page([run("abcd", 0.1, 0.1, 0.4), run("■■■■", 0.1, 0.3, 0.4)]));
  assert.equal(offsetAt(l, 0.11, 0.11), 0);
  assert.equal(offsetAt(l, 0.49, 0.11), 4);
  const heb = l.runs[1].start;
  assert.equal(offsetAt(l, 0.49, 0.31), heb); // right edge = the first letter
  assert.equal(offsetAt(l, 0.11, 0.31), heb + 4);
  // Beside a line lands on that line, not on the nearest one diagonally.
  assert.equal(offsetAt(l, 0.9, 0.105), 4);
});

test("a range's boxes follow the text direction", () => {
  const l = layoutPage(page([run("abcd", 0.1, 0.1, 0.4), run("■■■■", 0.1, 0.3, 0.4)]));
  const [ltr] = rangeRects(l, 1, 3);
  near(ltr.x, 0.2);
  near(ltr.w, 0.2);
  const s = l.runs[1].start;
  const [rtl] = rangeRects(l, s, s + 1); // first Hebrew letter = rightmost quarter
  near(rtl.x, 0.4);
  near(rtl.w, 0.1);
  const c = caretAt(l, s, "start");
  near(c!.x, 0.5);
});

test("double tap picks the word; selected text drops invisible marks", () => {
  const text = "hello, world";
  assert.deepEqual(wordRangeAt(text, 2), [0, 5]);
  assert.deepEqual(wordRangeAt(text, 5), [0, 5]); // just after the word
  assert.deepEqual(wordRangeAt(text, 9), [7, 12]);
  assert.deepEqual(wordRangeAt("■■■■ ■■■■", 6), [5, 9]);
  const l = layoutPage(page([run("אֵלֶּיךָ", 0.1, 0.1, 0.1)]));
  assert.equal(selectedText(l, 0, 3), "ab");
});

test("line segments: pieces of a line merge, separate lines stay apart", () => {
  const segs = lineSegments([
    { x: 0.1, y: 0.1, w: 0.1, h: 0.02, line: 0 },
    { x: 0.205, y: 0.1, w: 0.1, h: 0.02, line: 0 },
    { x: 0.1, y: 0.2, w: 0.1, h: 0.02, line: 1 },
  ]);
  assert.equal(segs.length, 2);
  near(segs[0].w, 0.205);
});

test("highlight strokes are as tall as the line and exactly as long as the text", () => {
  const [hl] = markStrokes("highlight", [{ x: 100, y: 200, w: 150, h: 18 }], { fill: "#fde047", ink: "#ca8a04" }, () => "id");
  assert.equal(hl.strokeStyle, "highlighter");
  near(hl.strokeWidth * (PEN_TYPE_PRESETS.highlighter.sizeScale ?? 1), 18);
  assert.equal(hl.x, 100);
  assert.equal(hl.y, 209);
  assert.equal(hl.points[hl.points.length - 1][0], 150);
  assert.equal(hl.color, "#fde047");
  const [ul] = markStrokes("underline", [{ x: 0, y: 0, w: 50, h: 20 }], { fill: "#fde047", ink: "#ca8a04" }, () => "u");
  assert.equal(ul.strokeStyle, "fineliner");
  assert.equal(ul.color, "#ca8a04");
  assert.ok(ul.y > 15 && ul.y < 20);
  const [st] = markStrokes("strike", [{ x: 0, y: 0, w: 50, h: 20 }], { fill: "#fde047", ink: "#ca8a04" }, () => "s");
  assert.ok(st.y > 8 && st.y < 14);
});

test("search ignores case, accents, niqqud, harakat and Arabic letter forms", () => {
  assert.deepEqual(findAll("Café au lait", "CAFE"), [[0, 4]]);

```

```

assert.deepEqual(findAll("\u05B8\u05C1\u05B9", "████"), [[0, 7]]);
assert.equal(findAll("\u064F\u064E\u064E\u0651", "████").length, 1);
assert.equal(findAll("\uFEE3\uFEA4\uFEE4\uFEAA", "████").length, 1); // presentation forms
assert.equal(findAll("████", "████").length, 1); // alef with hamza
assert.equal(findAll("hello" + NL + "world", "hello world").length, 1);
assert.equal(findAll("████████", "████████").length, 1); // Persian yeh / heh look-alikes

assert.equal(findAll("aaa", "").length, 0);
const folded = foldForSearch("A\u00Adb").folded;
assert.equal(folded, "ab");
});

test("a snippet is about 80 characters and its match index points at the match", () => {
  const text = "x ".repeat(100) + "needle" + " y".repeat(100);
  const [[s, e]] = findAll(text, "NEEDLE");
  const sn = snippetAround(text, s, e);
  assert.equal(sn.snippet.slice(sn.matchStart, sn.matchStart + sn.matchLength), "needle");
  assert.ok(sn.snippet.length <= 84, `length ${sn.snippet.length}`);
  const short = snippetAround("find me", 5, 7);
  assert.equal(short.snippet, "find me");
  assert.equal(short.matchStart, 5);
});

test("pdf.js items become page-fraction boxes above their baseline", () => {
  // A 612 x 792 pt page; pdf.js's scale-1 viewport flips y.
  const vp = { transform: [1, 0, 0, -1, 0, 792], width: 612, height: 792 };
  const [r] = runsFromPdfItems(
    [{ str: "Hello", transform: [12, 0, 0, 12, 72, 700], width: 30, fontName: "f1" }, { str: " ", transform: [
      12, 0, 0, 12, 102, 700], width: 3 }],
    { f1: { ascent: 0.8, descent: -0.2 } },
    vp
  );
  near(r.x, 72 / 612);
  near(r.w, 30 / 612);
  near(r.y, (92 - 9.6) / 792);
  near(r.h, 12 / 792);
});

test("PowerPoint letters drawn one by one are glued back; a repeat is dropped", () => {
  const frags = ["H", "e", "y"].map((c, i) => run(c, 0.1 + i * 0.01, 0.1, 0.01));
  frags.push(run("H", 0.1, 0.1, 0.01)); // drawn twice
  frags.push(run("next", 0.1, 0.2, 0.04));
  const merged = mergeFragments(frags);
  assert.deepEqual(merged.map((m) => m.t), ["Hey", "next"]);
  near(merged[0].w, 0.03);
});

test("a page letterboxed onto a sheet of another shape is centred", () => {
  const b = pageBoxOnSheet({ w: 200, h: 100 }, 400, 400);
  assert.deepEqual(b, { x: 0, y: 100, w: 400, h: 200 });
});

// █ Found in review █
{
  const words = (xs: Array<string, number>) => xs.map(([t, x]) => ({ t, x, y: 0, w: t.length * 0.01, h: 0.02
    }));
  const read = (runs: ReturnType<typeof words>) => orderLineX(runs).map((r) => r.t).join(" ");

  test("a Hebrew line with an English term reads from its first Hebrew word", () => {
    // Visual, left to right: █ █ Binary Search Tree :████ (more Latin letters than Hebrew)
    const line = words(["█", 0.1], ["█", 0.14], ["Binary", 0.2], ["Search", 0.28], ["Tree", 0.36], ["████
      ", 0.44]);
    assert.equal(read(line), "████: Binary Search Tree █ █");
  });

  test("a Hebrew phrase inside an English line keeps its own order", () => {
    // Visual: The phrase █ █ means (█ is to the right of █)
    const line = words(["The", 0.1], ["phrase", 0.15], ["█", 0.25], ["█", 0.31], ["means", 0.4]);
    assert.equal(read(line), "The phrase █ █ means");
  });

  test("slide words drawn one at a time stay separate words", () => {
    // pptx draws word by word and skips the spaces; a space's gap must not glue.
    const h = 0.02;
    const frags = ["This", "is", "a", "test", "of", "it"].reduce<Array<{ t: string; x: number; y: number; w: n
      umber; h: number }>>({
      acc, t) => {
        const x = acc.length ? acc[acc.length - 1].x + acc[acc.length - 1].w + 0.25 * h : 0.1;

```

```

        acc.push({ t, x, y: 0.5, w: t.length * 0.5 * h, h });
        return acc;
    },
    []
);
const merged = mergeFragmentsX(frag);
assert.ok(!merged.some((r) => /isa|testof|ofit/.test(r.t)), merged.map((r) => r.t).join("|"));
// Letters drawn one by one, touching, are still glued.
const letters = ["a", "b", "c"].map((t, i) => ({ t, x: 0.1 + i * 0.01, y: 0.5, w: 0.01, h }));
assert.equal(mergeFragmentsX(letters).map((r) => r.t).join("|"), "abc");
});

test("brackets in visually stored Hebrew read the right way round", () => {
    // "████ (3)" stored in visual order as its logical characters: ")3( █████"
    assert.equal(visualToLogicalX(")3( █████"), "████ (3)");
});

test("search words: quotes dropped, each word once", () => {
    assert.deepEqual(queryTerms('"exam" physics exam'), ["exam", "physics"]);
    assert.deepEqual(queryTerms("  "), []);
});
}

console.log(` ${passed} passed, ${failed} failed`);
if (failed > 0) process.exit(1);

=====
// === END FILE: tests/pdf_text.test.ts ===
=====

=====
// === START FILE: tests/pen_dynamics_and_stabilization.test.ts ===
// File 170 of 194 | Lines: 321 | Size: 11702 bytes
=====

/**
 * S Pen Dynamics, Pressure Calibration, & Stabilization Test Suite
 * =====
 * Validates the inking engine capabilities:
 * 1. S Pen pressure calibration curves (soft, standard, firm, off) and dynamic sensitivity.
 * 2. 0-100% stabilization and streamline scaling across all 8 pen types.
 * 3. 8 distinct pen presets (ballpoint, gel, fineliner, fountain, brush, marker, highlighter, pencil).
 * 4. Draw-and-hold auto-straightening geometry detection and point interpolation.
 * 5. Terminal anti-hook filter on pen lift-off.
 * 6. buildStrokePath geometry generation with smoothing override.
 */

import assert from "node:assert/strict";
import {
    calibratePressure,
    getStrokeOptionsForStyle,
    applyPenDynamics,
    buildStrokePath,
    isStrokeStraight,
    straightenStrokePoints,
    pruneTerminalHooks,
} from "../src/lib/whiteboard/stroke";
import {
    PEN_TYPE_PRESETS,
    PEN_TYPES,
    type StrokeStyle,
    type StrokeElement,
} from "../src/lib/whiteboard/types";

let passed = 0;
let failed = 0;

function test(name: string, fn: () => void) {
    try {
        fn();
        passed++;
        console.log(` ✓ ${name}`);
    } catch (err) {
        failed++;
        console.error(` ✗ ${name}`);
        console.error(`   ${err as Error}.message`);
    }
}

```

```
}

console.log("\n■■■ S Pen Dynamics & Pressure Calibration ■■■");

test("pressure calibration: 'off' curve always returns uniform 0.5", () => {
  assert.equal(calibratePressure(0.1, 80, "off"), 0.5);
  assert.equal(calibratePressure(0.5, 80, "off"), 0.5);
  assert.equal(calibratePressure(0.9, 80, "off"), 0.5);
});

test("pressure calibration: 0 sensitivity always returns uniform 0.5", () => {
  assert.equal(calibratePressure(0.1, 0, "standard"), 0.5);
  assert.equal(calibratePressure(0.9, 0, "soft"), 0.5);
});

test("pressure calibration: 'soft' curve reaches higher pressure for light touch", () => {
  const lightRaw = 0.25;
  const softCalibrated = calibratePressure(lightRaw, 50, "soft");
  const standardCalibrated = calibratePressure(lightRaw, 50, "standard");
  assert.ok(
    softCalibrated > standardCalibrated,
    `Soft curve (${softCalibrated}) should exceed standard (${standardCalibrated}) for light touch`
  );
});

test("pressure calibration: 'firm' curve requires heavier touch for thick ink", () => {
  const mediumRaw = 0.4;
  const firmCalibrated = calibratePressure(mediumRaw, 50, "firm");
  const standardCalibrated = calibratePressure(mediumRaw, 50, "standard");
  assert.ok(
    firmCalibrated < standardCalibrated,
    `Firm curve (${firmCalibrated}) should be lower than standard (${standardCalibrated}) for medium touch`
  );
});

test("pressure calibration: clamps extreme raw inputs safely into [0.02, 1.0]", () => {
  const low = calibratePressure(-0.5, 100, "standard");
  const high = calibratePressure(2.5, 100, "standard");
  assert.ok(low >= 0.02, "Low pressure must clamp at or above 0.02");
  assert.ok(high <= 1.0, "High pressure must clamp at or below 1.0");
});

test("applyPenDynamics: applies speed dynamics to stroke points", () => {
  const rawPoints: Array<[number, number, number]> = [
    [0, 0, 0.5],
    [5, 0, 0.5],
    [50, 0, 0.5], // fast movement -> velocity response for ballpoint
  ];
  const dynamicPoints = applyPenDynamics(rawPoints, "ballpoint");
  assert.equal(dynamicPoints.length, rawPoints.length);
  // Point 2 moved 45px (fast) -> speed response reduces effective pressure/width
  assert.ok(dynamicPoints[2][2] < dynamicPoints[1][2], "Fast movement should thin stroke in ballpoint");
});

console.log("\n■■■ Stroke Stabilization & Smoothing (0-100%) ■■■");

test("smoothing: default 50% exactly matches preset default parameters", () => {
  for (const st of PEN_TYPES) {
    const preset = PEN_TYPE_PRESETS[st];
    const opts = getStrokeOptionsForStyle(st, 10, true, 50);
    assert.equal(
      opts.smoothing,
      preset.smoothing,
      `${st} smoothing at 50% must equal preset default (${preset.smoothing})`
    );
    assert.equal(
      opts.streamline,
      preset.streamline,
      `${st} streamline at 50% must equal preset default (${preset.streamline})`
    );
  }
});

test("smoothing: 0% scales down smoothing and streamline to minimal boundary", () => {
  const opts = getStrokeOptionsForStyle("gel", 10, true, 0);
  assert.equal(opts.smoothing, 0.02);
  assert.equal(opts.streamline, 0.02);
});
```

```
test("smoothing: 100% doubles smoothing and streamline up to safety clamps", () => {
  const gelPreset = PEN_TYPE_PRESETS.gel;
  const opts = getStrokeOptionsForStyle("gel", 10, true, 100);
  const expectedSmoothing = Math.min(0.98, gelPreset.smoothing * 2.0);
  const expectedStreamline = Math.min(0.95, gelPreset.streamline * 2.0);
  assert.equal(opts.smoothing, expectedSmoothing);
  assert.equal(opts.streamline, expectedStreamline);
});

console.log("\n■■■ 8 Distinct Pen Engines Characteristics ■■■");

test("all 8 core pen engines are registered and defined", () => {
  const requiredPens: StrokeStyle[] = [
    "ballpoint",
    "gel",
    "fineliner",
    "fountain",
    "brush",
    "marker",
    "highlighter",
    "pencil",
  ];
  for (const pen of requiredPens) {
    assert.ok(PEN_TYPE_PRESETS[pen], `Preset for ${pen} must exist`);
    // Ballpoint and fineliner keep their presets (old strokes still draw)
    // but are not offered in the picker: they looked like the gel pen.
    if (pen !== "ballpoint" && pen !== "fineliner") {
      assert.ok(PEN_TYPES.includes(pen), `PEN_TYPES must include ${pen}`);
    }
  }
});

test("pen characteristics: ballpoint has speed thinning and round cap", () => {
  const p = PEN_TYPE_PRESETS.ballpoint;
  assert.equal(p.cap, "round");
  assert.ok(p.speedResponse && p.speedResponse > 0, "Ballpoint must respond to speed");
  assert.equal(p.sizeScale, 0.7, "Ballpoint sizeScale should be 0.7 for fine notes");
});

test("pen characteristics: fineliner is monoline technical drafting pen", () => {
  const p = PEN_TYPE_PRESETS.fineliner;
  assert.equal(p.thinning, 0, "Fineliner must have zero thinning");
  assert.equal(p.startTaper, 0, "Fineliner must have zero start taper");
  assert.equal(p.endTaper, 0, "Fineliner must have zero end taper");
  assert.equal(p.sizeScale, 0.5, "Fineliner must be fine 0.5 scale");
});

test("pen characteristics: fountain pen has 40° chisel nib and taper", () => {
  const p = PEN_TYPE_PRESETS.fountain;
  assert.equal(p.nibAngle, 40, "Fountain pen must have 40 deg nib angle");
  assert.ok(p.endTaper > 0, "Fountain pen must taper at end of stroke");
  assert.ok(p.thinning >= 0.5, "Fountain pen must have dynamic thinning");
});

test("pen characteristics: brush pen has fluid taper and pressure response", () => {
  const p = PEN_TYPE_PRESETS.brush;
  assert.equal(p.simulatePressure, true);
  assert.ok(p.startTaper > 0 && p.endTaper > 0, "Brush must taper at both ends");
  assert.ok(p.sizeScale >= 1.2, "Brush should be bold");
});

test("pen characteristics: highlighter has multiply blend mode and flat chisel cap", () => {
  const p = PEN_TYPE_PRESETS.highlighter;
  assert.equal(p.mixBlendMode, "multiply", "Highlighter must multiply so text stays sharp");
  assert.equal(p.cap, "flat", "Highlighter must have flat chisel cap");
  assert.ok(p.opacity <= 0.4, "Highlighter must be translucent");
  assert.equal(p.sizeScale, 3.0, "Highlighter must be broad 3x scale");
});

test("pen characteristics: pencil has graphite grain texture and multiply blend", () => {
  const p = PEN_TYPE_PRESETS.pencil;
  assert.equal(p.grain, true, "Pencil must have grain texture flag");
  assert.equal(p.mixBlendMode, "multiply");
  assert.ok(p.opacity < 1.0, "Pencil has slight tooth translucency");
});

console.log("\n■■■ Draw-and-Hold Auto-Straighten Detection ■■■");
```

```
test("isStrokeStraight: detects intentional straight line", () => {
  // Horizontal straight line with 10 points
  const straightPoints: Array<[number, number, number]> = [
    [10, 50, 0.5],
    [30, 50, 0.5],
    [50, 50, 0.5],
    [70, 50, 0.5],
    [90, 50, 0.5],
    [110, 50, 0.5],
    [130, 50, 0.5],
    [150, 50, 0.5],
  ];
  assert.ok(isStrokeStraight(straightPoints), "Horizontal line must be classified as straight");
});

test("isStrokeStraight: rejects curved or zigzag strokes", () => {
  // Zigzag stroke
  const zigzagPoints: Array<[number, number, number]> = [
    [10, 10, 0.5],
    [30, 50, 0.5],
    [50, 10, 0.5],
    [70, 50, 0.5],
    [90, 10, 0.5],
    [110, 50, 0.5],
  ];
  assert.ok(!isStrokeStraight(zigzagPoints), "Zigzag stroke must not be classified as straight");
});

test("isStrokeStraight: rejects short strokes under 28px chord threshold", () => {
  const shortPoints: Array<[number, number, number]> = [
    [10, 10, 0.5],
    [12, 10, 0.5],
    [14, 10, 0.5],
    [16, 10, 0.5],
    [18, 10, 0.5],
    [20, 10, 0.5],
  ];
  assert.ok(!isStrokeStraight(shortPoints), "Stroke with chord < 28px must return false");
});

test("straightenStrokePoints: generates straight interpolated line preserving endpoints and average pressure",
  () => {
    const input: Array<[number, number, number]> = [
      [0, 0, 0.4],
      [20, 5, 0.6],
      [40, -5, 0.8],
      [100, 0, 0.6],
    ];
    const straightened = straightenStrokePoints(input, 5);
    assert.equal(straightened.length, 6);
    assert.deepEqual(straightened[0], [0, 0, 0.6]); // start point with average pressure (0.4+0.6+0.8+0.6)/4 = 0.6
    assert.deepEqual(straightened[5], [100, 0, 0.6]); // end point
    // Midpoint x must be 40% of chord at segment 2
    assert.equal(straightened[2][0], 40);
    assert.equal(straightened[2][1], 0);
  });

console.log("\n■■■ Anti-Hook Lift-Off Filter ■■■");

test("pruneTerminalHooks: trims sharp reverse hook flick at stroke end", () => {
  // A forward stroke that sharply hooks back on lift-off
  const hookedPoints: Array<[number, number, number]> = [
    [10, 100, 0.5],
    [30, 100, 0.5],
    [50, 100, 0.5],
    [70, 100, 0.5],
    [90, 100, 0.5],
    [110, 100, 0.5],
    [95, 95, 0.2], // sharp terminal hook backward on lift-off
  ];
  const pruned = pruneTerminalHooks(hookedPoints);
  assert.ok(pruned.length < hookedPoints.length, "Terminal hook should be pruned");
  assert.equal(pruned[pruned.length - 1][0], 110, "Last point should be before the hook turn");
});

test("pruneTerminalHooks: preserves smooth natural strokes without hooks", () => {
```

```

const smoothPoints: Array<[number, number, number]> = [
  [10, 100, 0.5],
  [30, 105, 0.5],
  [50, 112, 0.5],
  [70, 120, 0.5],
  [90, 130, 0.5],
];
const result = pruneTerminalHooks(smoothPoints);
assert.equal(result.length, smoothPoints.length, "Smooth strokes must not be pruned");
});

console.log("\n■■ buildStrokePath with Smoothing Override ■■");

test("buildStrokePath generates valid SVG path data", () => {
  const testStroke: StrokeElement = {
    id: "test-stroke",
    type: "stroke",
    x: 10,
    y: 10,
    rotation: 0,
    opacity: 1,
    createdAt: Date.now(),
    updatedAt: Date.now(),
    points: [
      [10, 10, 0.5],
      [20, 15, 0.5],
      [30, 25, 0.5],
      [40, 40, 0.5],
    ],
    color: "#3b82f6",
    strokeWidth: 4,
    strokeStyle: "gel",
  };
  const pathNormal = buildStrokePath(testStroke, false);
  assert.ok(pathNormal.startsWith("M"), "SVG path must start with M");

  const pathSmoothed = buildStrokePath(testStroke, false, 85);
  assert.ok(pathSmoothed.startsWith("M"), "Overridden smoothing path must be valid SVG");
});

console.log(`\n=====`);
console.log(`Pen Dynamics & Stabilization: ${passed} passed, ${failed} failed`);
console.log(`=====`);

if (failed > 0) {
  process.exit(1);
}

=====
// === END FILE: tests/pen_dynamics_and_stabilization.test.ts ===
=====

=====
// === START FILE: tests/pens_bar_placement.test.ts ===
// File 171 of 194 | Lines: 92 | Size: 3318 bytes
=====

/**
 * Favorite pens bar: one home shared by every board – left edge, centred.
 *
 * Reported on startup: the bar opened in the middle of the notebook page
 * instead of docking to the left side. The default only applies when there is
 * no saved drag (a fresh workspace); a persisted position must keep winning.
 * These pin the default so it cannot drift back to the top corner.
 */

import assert from "node:assert/strict";
import {
  defaultPensBarPosition,
  PENS_BAR_EDGE_GAP,
  PENS_BAR_HEIGHT_ESTIMATE,
} from "../src/lib/whiteboard/placement";

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {

```

```
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`${(err as Error).message}`);
  }
}

const near = (a: number, b: number, eps = 1) => Math.abs(a - b) <= eps;

// A typical phone-sized canvas pane.
const AREA = { left: 80, top: 0, right: 880, bottom: 1700 };
const BAR_H = 320;

console.log(`\n■■ Pens bar default placement ■■`);

test("parks on the left edge of the canvas pane", () => {
  const p = defaultPensBarPosition(AREA, BAR_H);
  assert.equal(p.x, AREA.left + PENS_BAR_EDGE_GAP, `x ${p.x}, expected ${AREA.left + PENS_BAR_EDGE_GAP}`);
});

test("is vertically centred in the pane, not near the top", () => {
  const p = defaultPensBarPosition(AREA, BAR_H);
  const barMiddle = p.y + BAR_H / 2;
  const areaMiddle = (AREA.top + AREA.bottom) / 2;
  assert.ok(
    near(barMiddle, areaMiddle),
    `bar middle ${barMiddle}, pane middle ${areaMiddle}`
  );
});

test("stays clear of the top toolbar zone", () => {
  const p = defaultPensBarPosition(AREA, BAR_H);
  assert.ok(p.y > 120, `y ${p.y} sits inside the top toolbar/zoom-pill zone`);
});

test("same inputs give the same spot on any board (one global home)", () => {
  const a = defaultPensBarPosition(AREA, BAR_H);
  const b = defaultPensBarPosition(AREA, BAR_H);
  assert.deepEqual(a, b, "default must be deterministic, not per-board");
});

test("a short pane still keeps the bar inside instead of centring off-screen", () => {
  const short = { left: 0, top: 0, right: 800, bottom: 200 };
  const p = defaultPensBarPosition(short, BAR_H);
  // The bar (320) is taller than this pane (200): it cannot fit, so it pins
  // to the top and the render-time clamp owns it from there – never centred
  // off-screen, never NaN.
  assert.equal(p.y, short.top + 8, `y ${p.y} should pin to the top`);
  assert.equal(p.x, short.left + PENS_BAR_EDGE_GAP, "still on the left edge");
});

test("garbage heights and degenerate areas never yield NaN", () => {
  for (const h of [0, -50, NaN, Infinity]) {
    const p = defaultPensBarPosition(AREA, h);
    assert.ok(Number.isFinite(p.x) && Number.isFinite(p.y), `height ${h} gave ${p.x},${p.y}`);
  }
  const p = defaultPensBarPosition({ left: 0, top: 0, right: 0, bottom: 0 }, BAR_H);
  assert.ok(Number.isFinite(p.x) && Number.isFinite(p.y), "zero area gave NaN");
});

test("height estimate constant is sane", () => {
  assert.ok(
    PENS_BAR_HEIGHT_ESTIMATE >= 150 && PENS_BAR_HEIGHT_ESTIMATE <= 600,
    `estimate ${PENS_BAR_HEIGHT_ESTIMATE} is not bar-shaped`
  );
});

console.log(`\n${passed} passed, ${failed} failed.`);
if (failed > 0) process.exit(1);

=====
// === END FILE: tests/pens_bar_placement.test.ts ===
=====
```

```

=====
// === START FILE: tests/pointer_handoff.test.ts ===
// File 172 of 194 | Lines: 313 | Size: 10925 bytes
=====

/**
 * Split-view pointer hand-off: every replayed press gets exactly one release.
 *
 * The bug this pins: the canvas counts fingers (in on pointerdown, out on
 * pointerup). The split pane replays a press a couple of frames after focus
 * switches sides, so a quick tap had usually lifted already - its real release
 * reached the canvas FIRST and removed nothing (Map.delete of an unknown id
 * is a silent no-op), then the late replay added a finger that was never
 * lifted. Every later one-finger touch then counted as the second finger of a
 * pinch: scrolling zoomed (measured 0.5 -> 0.355 on one drag), drawing and
 * typing did nothing, and closing the split did not help.
 *
 * These tests drive handOff/handOffPress against a stub DOM (separate node
 * process per test file, so stubbing globals is safe) and assert the pairing:
 * a press released before the replay is never dispatched; a held press gets
 * exactly one release, real if it lands inside the canvas, synthetic if not.
 */

import assert from "node:assert/strict";

import { handOff, handOffPress } from "../src/lib/whiteboard/pointerHandoff";

// -----
// Stub DOM: just enough for the hand-off (listeners, hit-testing, dispatch).
// Synthetic events do NOT bubble back to window here; in a real browser the
// re-dispatched release would, but it can never loop: the pairing check only
// synthesises when the real target is OUTSIDE the canvas, and
// Node.contains(itself) is true, so a second pass always sees "inside".
// -----

class FakeNode {}

class FakeEvent {
  readonly type: string;
  target: FakeNode | null = null;
  constructor(type: string, init: Record<string, unknown> = {}) {
    this.type = type;
    for (const [k, v] of Object.entries(init)) {
      (this as unknown as Record<string, unknown>)[k] = v;
    }
  }
}

class FakePointerEvent extends FakeEvent {}

class FakeElement extends FakeNode {
  readonly name: string;
  readonly dispatched: FakeEvent[] = [];
  readonly seenSelectors: string[] = [];
  private readonly contained = new Set<FakeNode>();
  closestResult: FakeElement | null = null;

  constructor(name: string) {
    super();
    this.name = name;
    this.contained.add(this);
    this.closestResult = this;
  }

  addContained(n: FakeNode): this {
    this.contained.add(n);
    return this;
  }

  dispatchEvent(e: FakeEvent): boolean {
    if (!e.target) e.target = this;
    this.dispatched.push(e);
    return true;
  }

  closest(selector: string): FakeElement | null {
    this.seenSelectors.push(selector);
    return this.closestResult;
  }
}

```

```

    }

    contains(n: FakeNode | null): boolean {
        return n !== null && this.contained.has(n);
    }
}

type Listener = (e: FakeEvent) => void;

interface Harness {
    listeners: Map<string, Listener[]>;
    fire: (type: string, e: FakeEvent) => void;
    listenerCount: (type: string) => number;
    rafQueue: Array<() => void>;
    timerQueue: Array<{ cb: () => void; delay: number }>;
    drainRaf: () => void;
    drainTimers: (delay: number) => void;
    setElementAtPoint: (el: FakeElement | null) => void;
    lastHitPoint: () => { x: number; y: number } | null;
}

function installHarness(): Harness {
    const listeners = new Map<string, Listener[]>();
    const rafQueue: Array<() => void> = [];
    const timerQueue: Array<{ cb: () => void; delay: number }> = [];
    let elementAtPoint: FakeElement | null = null;
    let hitPoint: { x: number; y: number } | null = null;
    const g = globalThis as unknown as Record<string, unknown>;

    g.window = {
        addEventListener: (t: string, fn: Listener) => {
            const list = listeners.get(t) ?? [];
            list.push(fn);
            listeners.set(t, list);
        },
        removeEventListener: (t: string, fn: Listener) => {
            listeners.set(t, (listeners.get(t) ?? []).filter((f) => f !== fn));
        },
    };
    g.document = {
        elementFromPoint: (x: number, y: number) => {
            hitPoint = { x, y };
            return elementAtPoint;
        },
    };
    g.PointerEvent = FakePointerEvent;
    g.Node = FakeNode;
    g.requestAnimationFrame = (cb: () => void) => {
        rafQueue.push(cb);
        return rafQueue.length;
    };
    g.setTimeout = ((cb: () => void, delay?: number) => {
        timerQueue.push({ cb, delay: delay ?? 0 });
        return timerQueue.length;
    }) as unknown as typeof setTimeout;

    return {
        listeners,
        fire: (type, e) => {
            [...(listeners.get(type) ?? [])].forEach((f) => f(e));
        },
        listenerCount: (type) => listeners.get(type)?.length ?? 0,
        rafQueue,
        timerQueue,
        drainRaf: () => {
            while (rafQueue.length > 0) rafQueue.splice(0).forEach((cb) => cb());
        },
        drainTimers: (delay) => {
            const due = timerQueue.filter((t) => t.delay === delay);
            for (const t of due) {
                const i = timerQueue.indexOf(t);
                if (i >= 0) timerQueue.splice(i, 1);
                t.cb();
            }
        },
        setElementAtPoint: (el) => {
            elementAtPoint = el;
        },
    };
}

```

```

    lastHitPoint: () => hitPoint,
  };
}

function field(e: FakeEvent, key: string): unknown {
  return (e as unknown as Record<string, unknown>)[key];
}

const PRESS = {
  pointerId: 7,
  pointerType: "touch",
  clientX: 100,
  clientY: 100,
  button: 0,
  buttons: 1,
  pressure: 0.5,
  isPrimary: true,
};

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`   ${err as Error}.message`);
  }
}

test("a tap released before the replay dispatches nothing (the ghost, pinned)", () => {
  const h = installHarness();
  const canvas = new FakeElement("canvas");
  const elsewhere = new FakeElement("elsewhere");
  h.setElementAtPoint(canvas);

  handOffPress({ ...PRESS }, 100, 100);
  // The real release lands first, before React has swapped the panes.
  const up = new FakePointerEvent("pointerup", { pointerId: 7, clientX: 100, clientY: 100, buttons: 0 });
  up.target = elsewhere;
  h.fire("pointerup", up);
  // Now let the replay moment pass.
  h.drainRaf();
  h.drainTimers(50);

  assert.equal(canvas.dispatched.length, 0, "no press may be replayed after its release");
  assert.equal(elsewhere.dispatched.length, 0, "nothing dispatched anywhere");
  assert.equal(h.listenerCount("pointerup"), 0, "listeners cleaned up");
  assert.equal(h.listenerCount("pointercancel"), 0, "listeners cleaned up");
});

test("a held press replays, and a release outside gets exactly one synthetic release", () => {
  const h = installHarness();
  const canvas = new FakeElement("canvas");
  const outside = new FakeElement("outside");
  h.setElementAtPoint(canvas);

  handOffPress({ ...PRESS }, 100, 100);
  h.drainRaf();
  assert.equal(canvas.dispatched.length, 1, "the held press replays once");
  const press = canvas.dispatched[0];
  assert.equal(press.type, "pointerdown");
  assert.equal(field(press, "pointerId"), 7);
  assert.equal(field(press, "clientX"), 100);
  assert.equal(field(press, "bubbles"), true);
  assert.deepEqual(h.lastHitPoint(), { x: 100, y: 100 }, "replay targets the press point");

  // The finger lifts off the canvas (sidebar, toolbar, another app surface).
  const up = new FakePointerEvent("pointerup", { pointerId: 7, clientX: 140, clientY: 130, buttons: 0 });
  up.target = outside;
  h.fire("pointerup", up);

  assert.equal(canvas.dispatched.length, 2, "exactly one release follows the press");
  const syn = canvas.dispatched[1];
  assert.equal(syn.type, "pointerup", "release type preserved");

```

```
assert.equal(field(syn, "pointerId"), 7, "paired to the same finger");
assert.equal(field(syn, "buttons"), 0, "a release carries no buttons");
assert.equal(field(syn, "pressure"), 0, "a release carries no pressure");
assert.equal(field(syn, "clientX"), 140, "at the release point, not the press point");
assert.equal(field(syn, "clientY"), 130, "at the release point, not the press point");
assert.equal(field(syn, "pointerType"), "touch", "original contact kind preserved");
assert.ok(
  canvas.seenSelectors.includes("[data-wb-canvas-area]"),
  "pairing resolves through the canvas area, not the raw target"
);
assert.equal(h.listenerCount("pointerup"), 0, "listeners cleaned up");
});

test("a release inside the canvas after the replay sends nothing extra", () => {
  const h = installHarness();
  const canvas = new FakeElement("canvas");
  const inner = new FakeElement("inner");
  canvas.addContained(inner);
  h.setElementAtPoint(canvas);

  handOffPress({ ...PRESS }, 100, 100);
  h.drainRaf();
  assert.equal(canvas.dispatched.length, 1, "press replayed");

  // The canvas got the real release itself: one press, one release, done.
  const up = new FakePointerEvent("pointerup", { pointerId: 7, clientX: 110, clientY: 105, buttons: 0 });
  up.target = inner;
  h.fire("pointerup", up);

  assert.equal(canvas.dispatched.length, 1, "no synthetic duplicate");
  assert.equal(h.listenerCount("pointerup"), 0, "listeners cleaned up");
});

test("pointercancel pairs exactly like pointerup", () => {
  const h = installHarness();
  const canvas = new FakeElement("canvas");
  const outside = new FakeElement("outside");
  h.setElementAtPoint(canvas);

  handOffPress({ ...PRESS }, 100, 100);
  h.drainRaf();
  assert.equal(canvas.dispatched.length, 1, "press replayed");

  const cancel = new FakePointerEvent("pointercancel", { pointerId: 7, clientX: 100, clientY: 100, buttons: 0 });
  cancel.target = outside;
  h.fire("pointercancel", cancel);

  assert.equal(canvas.dispatched.length, 2, "cancel paired");
  assert.equal(canvas.dispatched[1].type, "pointercancel", "cancel type preserved, not rewritten as up");
});

test("another finger's release is ignored", () => {
  const h = installHarness();
  const canvas = new FakeElement("canvas");
  h.setElementAtPoint(canvas);

  handOffPress({ ...PRESS }, 100, 100);
  const other = new FakePointerEvent("pointerup", { pointerId: 9, clientX: 0, clientY: 0, buttons: 0 });
  other.target = canvas;
  h.fire("pointerup", other);
  h.drainRaf();

  assert.equal(canvas.dispatched.length, 1, "finger 7 still replays; finger 9 changed nothing");
  assert.equal(canvas.dispatched[0].type, "pointerdown");
});

test("handOff dispatches made events and skips nulls", () => {
  const h = installHarness();
  const el = new FakeElement("el");
  h.setElementAtPoint(el);

  handOff(() => new FakeEvent("wheel", {}) as unknown as Event, 5, 5);
  h.drainRaf();
  assert.equal(el.dispatched.length, 1, "made event dispatched");
  assert.equal(el.dispatched[0].type, "wheel");

  handOff(() => null, 5, 5);
```

```

    h.drainRaf();
    h.drainTimers(50);
    assert.equal(el.dispatched.length, 1, "cancelled replay dispatches nothing");
  });

console.log(`\n ${passed} passed, ${failed} failed\n`);
if (failed > 0) process.exit(1);

=====

// === END FILE: tests/pointer_handoff.test.ts ===
=====

=====

// === START FILE: tests/pro_text_tool.test.ts ===
// File 173 of 194 | Lines: 319 | Size: 11825 bytes
=====

/**
 * Test Suite for Pro Text Tool & Typography Architecture
 * =====
 * Tests:
 * 1. Markdown block parsing (Headings, Checklists, Lists, Quotes, Codeblocks)
 * 2. Inline tokenization (Bold, Italic, Underline, Strikethrough, Code, Highlight, LaTeX, WikiLinks, Tags)
 * 3. Interactive canvas checklist toggling (toggleChecklistAtLine)
 * 4. Intelligent list auto-continuation on Enter (getNextLinePrefix)
 * 5. Voice dictation controller initialization & safety
 * 6. Card mode properties integrity on TextElement
 */

import assert from "node:assert/strict";
import {
  parseMarkdownBlocks,
  tokenizeInline,
  toggleChecklistAtLine,
  getNextLinePrefix,
} from "../src/lib/whiteboard/textParser";
import {
  VoiceDictationController,
  isSpeechRecognitionSupported,
  requestMicrophonePermission,
} from "../src/lib/whiteboard/speechRecognition";
import { useWhiteboard } from "../src/store/whiteboard";
import { getBoardElements } from "../src/lib/whiteboard/notebook";
import type { TextElement } from "../src/lib/whiteboard/types";

let passed = 0;
let failed = 0;
async function test(name: string, fn: () => void | Promise<void>) {
  try {
    await fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(` ${(err as Error).message}`);
  }
}

async function runTests() {
  console.log("\n■ Professional Text Tool & Markdown Engine Tests ■");

  // 1. Markdown Block Tokenization
  await test("parseMarkdownBlocks: correctly parses headings, checklists, lists, quotes, and code blocks", ()
    => {
    const markdown = [
      "# Heading 1",
      "### Heading 2",
      "#### Heading 3",
      "- [ ] Pending task",
      "- [x] Completed task",
      "- Bullet item",
      "1. Numbered item 1",
      "> Blockquote text",
      "```",
      "const x = 42;",
      "```",
    ];
  });
}

```

```
    "Normal paragraph",
  ].join("\n");

const blocks = parseMarkdownBlocks(markdown);
assert.equal(blocks.length, 10);

assert.equal(blocks[0].type, "h1");
assert.equal(blocks[0].tokens[0].text, "Heading 1");

assert.equal(blocks[1].type, "h2");
assert.equal(blocks[1].tokens[0].text, "Heading 2");

assert.equal(blocks[2].type, "h3");
assert.equal(blocks[2].tokens[0].text, "Heading 3");

assert.equal(blocks[3].type, "checkboxlist");
assert.equal(blocks[3].checked, false);
assert.equal(blocks[3].tokens[0].text, "Pending task");

assert.equal(blocks[4].type, "checkboxlist");
assert.equal(blocks[4].checked, true);
assert.equal(blocks[4].tokens[0].text, "Completed task");

assert.equal(blocks[5].type, "bullet");
assert.equal(blocks[5].tokens[0].text, "Bullet item");

assert.equal(blocks[6].type, "numbered");
assert.equal(blocks[6].number, 1);
assert.equal(blocks[6].tokens[0].text, "Numbered item 1");

assert.equal(blocks[7].type, "quote");
assert.equal(blocks[7].tokens[0].text, "Blockquote text");

assert.equal(blocks[8].type, "codeblock");
assert.equal(blocks[8].rawText, "const x = 42;");

assert.equal(blocks[9].type, "paragraph");
assert.equal(blocks[9].tokens[0].text, "Normal paragraph");
});

// 2. Inline Tokenization
await test("tokenizeInline: parses bold, italic, underline, strike, code, highlight, math, wikilinks, tags",
  () => {
    const line =
      "Plain bold italic <u>under</u> ~~strike~~ `code` ==highlight== $E=mc^2$ [[Target Board|My Link]]"
      #tag https://pronotes.app";
    const tokens = tokenizeInline(line);

    const boldTok = tokens.find((t) => t.type === "bold");
    assert.ok(boldTok);
    assert.equal(boldTok.text, "bold");

    const italicTok = tokens.find((t) => t.type === "italic");
    assert.ok(italicTok);
    assert.equal(italicTok.text, "italic");

    const underTok = tokens.find((t) => t.type === "underline");
    assert.ok(underTok);
    assert.equal(underTok.text, "under");

    const strikeTok = tokens.find((t) => t.type === "strike");
    assert.ok(strikeTok);
    assert.equal(strikeTok.text, "strike");

    const codeTok = tokens.find((t) => t.type === "code");
    assert.ok(codeTok);
    assert.equal(codeTok.text, "code");

    const highlightTok = tokens.find((t) => t.type === "highlight");
    assert.ok(highlightTok);
    assert.equal(highlightTok.text, "highlight");

    const mathTok = tokens.find((t) => t.type === "math");
    assert.ok(mathTok);
    assert.equal(mathTok.text, "E=mc^2");

    const linkTok = tokens.find((t) => t.type === "link");
    assert.ok(linkTok);
```

```

    assert.equal(linkTok.text, "My Link");
    assert.equal(linkTok.value, "Target Board");

    const tagTok = tokens.find((t) => t.type === "tag");
    assert.ok(tagTok);
    assert.equal(tagTok.text, "#tag");

    const urlTok = tokens.find((t) => t.type === "url");
    assert.ok(urlTok);
    assert.equal(urlTok.text, "https://pronotes.app");
  });

  // 3. Interactive Checklist Toggling
  await test("toggleChecklistAtLine: toggles check state between [ ] and [x] at target line", () => {
    const text = [
      "- [ ] Item 1",
      "- [ ] Item 2",
      "- [x] Item 3",
    ].join("\n");

    // Toggle item 2 to checked
    const toggled2 = toggleChecklistAtLine(text, 1);
    const lines2 = toggled2.split("\n");
    assert.equal(lines2[0], "- [ ] Item 1");
    assert.equal(lines2[1], "- [x] Item 2");
    assert.equal(lines2[2], "- [x] Item 3");

    // Toggle item 3 back to unchecked
    const toggled3 = toggleChecklistAtLine(toggled2, 2);
    const lines3 = toggled3.split("\n");
    assert.equal(lines3[2], "- [ ] Item 3");

    // Non-target or invalid line is safe no-op
    assert.equal(toggleChecklistAtLine(text, 99), text);
  });

  // 4. Intelligent List Auto-continuation on Enter
  await test("getNextLinePrefix: handles continuation and clean exit on empty list items", () => {
    // Checklists
    assert.deepEqual(getNextLinePrefix("- [ ] Task 1"), { prefix: "- [ ] ", shouldExit: false });
    assert.deepEqual(getNextLinePrefix("- [ ] "), { prefix: "", shouldExit: true });
    assert.deepEqual(getNextLinePrefix("- [x] Done"), { prefix: "- [ ] ", shouldExit: false });
    assert.deepEqual(getNextLinePrefix("- [x] "), { prefix: "", shouldExit: true });

    // Bullet lists
    assert.deepEqual(getNextLinePrefix("- Apple"), { prefix: "- ", shouldExit: false });
    assert.deepEqual(getNextLinePrefix("- "), { prefix: "", shouldExit: true });
    assert.deepEqual(getNextLinePrefix("-* Banana"), { prefix: "* ", shouldExit: false });
    assert.deepEqual(getNextLinePrefix("-* "), { prefix: "", shouldExit: true });

    // Numbered lists
    assert.deepEqual(getNextLinePrefix("1. First step"), { prefix: "2. ", shouldExit: false });
    assert.deepEqual(getNextLinePrefix("2. Second step"), { prefix: "3. ", shouldExit: false });
    assert.deepEqual(getNextLinePrefix("3. "), { prefix: "", shouldExit: true });

    // Blockquotes
    assert.deepEqual(getNextLinePrefix("> Wise words"), { prefix: "> ", shouldExit: false });
    assert.deepEqual(getNextLinePrefix("> "), { prefix: "", shouldExit: true });

    // Ordinary line: no prefix
    assert.deepEqual(getNextLinePrefix("Ordinary sentence."), { prefix: "", shouldExit: false });
  });

  // 5. Voice Dictation Controller Lifecycle & Permission
  await test("VoiceDictationController: instantiates safely, verifies permissions and reports status", async () => {
    const controller = new VoiceDictationController();
    assert.equal(controller.getStatus(), "idle");
    assert.equal(typeof isSpeechRecognitionSupported(), "boolean");

    const perm = await requestMicrophonePermission();
    assert.equal(typeof perm.granted, "boolean");

    const started = await controller.start();
    assert.equal(typeof started, "boolean");

    // Stopping or aborting while idle is safe no-op
    controller.stop();
  });

```

```
    assert.equal(controller.getStatus(), "idle");
    controller.abort();
    assert.equal(controller.getStatus(), "idle");
  });

// 5b. Bug fix: stop() and abort() clear interim text via onInterim("")
await test("VoiceDictationController: stop/abort clears interim text callback", async () => {
  const interimCalls: string[] = [];
  const controller = new VoiceDictationController({
    onInterim: (text) => interimCalls.push(text),
  });

  // stop() should call onInterim("") to clear lingering preview text
  controller.stop();
  assert.ok(interimCalls.includes(""), "stop() should emit onInterim('') to clear preview");

  interimCalls.length = 0;

  // abort() should also clear
  controller.abort();
  assert.ok(interimCalls.includes(""), "abort() should emit onInterim('') to clear preview");
});

// 5c. Bug fix: multiple start() calls reset internal state cleanly
await test("VoiceDictationController: start() resets state for fresh session", async () => {
  const statusChanges: string[] = [];
  const controller = new VoiceDictationController({
    onStatusChange: (s) => statusChanges.push(s),
  });

  // In Node.js, window is undefined so start() returns false immediately.
  // We still verify that stop() after a failed start is safe, and that
  // a second start() doesn't throw or leave stale state.
  const started1 = await controller.start();
  assert.equal(started1, false, "start() returns false in Node (no window)");

  // Stop explicitly – should be safe even after a failed start
  controller.stop();
  assert.equal(controller.getStatus(), "idle");

  statusChanges.length = 0;

  // Second start – should not throw or leave stale error state
  const started2 = await controller.start();
  assert.equal(started2, false, "second start() also returns false in Node");
  // After stop(), status should be clean idle
  controller.stop();
  assert.equal(controller.getStatus(), "idle");
  assert.equal(controller.getErrorMessage(), "", "No lingering error after clean stop");
});

// 5d. Bug fix: getErrorMessage() returns empty string after fresh start
await test("VoiceDictationController: getErrorMessage() is empty on fresh controller", () => {
  const controller = new VoiceDictationController();
  assert.equal(controller.getErrorMessage(), "", "Fresh controller should have no error message");
});

// 6. Card Mode & TextElement Properties Integrity
await test("TextElement: supports cardMode, backgroundColor, borderRadius, and padding in store", () => {
  const store = useWhiteboard.getState();
  const boardId = store.createBoard("Text Test Board", null);
  store.setActiveBoard(boardId);

  const cardElement: TextElement = {
    id: "card_test_123",
    type: "text",
    x: 100,
    y: 100,
    width: 280,
    height: 120,
    rotation: 0,
    opacity: 1,
    text: "# Card Note\n- [ ] Task A\n- [ ] Task B",
    color: "#1e293b",
    fontSize: 16,
    fontFamily: "Inter",
    fontWeight: 400,
    align: "left",
  };
});
```

```
        cardMode: true,
        backgroundColor: "#eff6ff",
        borderColor: "#bfbdbfe",
        borderRadius: 12,
        padding: 16,
        createdAt: Date.now(),
        updatedAt: Date.now(),
    });

    store.addElement(cardElement, false);
    const activeBoard = useWhiteboard.getState().boards.find((b) => b.id === boardId!);
    const elements = getBoardElements(activeBoard);
    const saved = elements.find((e) => e.id === "card_test_123") as TextElement;

    assert.ok(saved, "Card element was saved");
    assert.equal(saved.cardMode, true);
    assert.equal(saved.backgroundColor, "#eff6ff");
    assert.equal(saved.borderColor, "#bfbdbfe");
    assert.equal(saved.borderRadius, 12);
    assert.equal(saved.padding, 16);
  });

  console.log(`\n${passed} passed, ${failed} failed.\n`);
  if (failed > 0) {
    process.exit(1);
  }
}

runTests();

=====
// === END FILE: tests/pro_text_tool.test.ts ===
=====

=====
// === START FILE: tests/regression_fix_pass.test.ts ===
// File 174 of 194 | Lines: 1447 | Size: 66989 bytes
=====

/**
 * Regression suite for the 2026-08-17 audit & fix pass (CHANGELOG_FIXES.md).
 * Each test pins a specific defect that was found and fixed – if any of these
 * fail, the fix regressed.
 */

import assert from "node:assert/strict";

// ---- libraries under test -----
import {
  parseText,
  buildGraph,
  elementNodeId,
  parseElementNodeId,
} from "../src/lib/whiteboard/links";
import { getSvgPathFromStroke, eraseFromStroke } from "../src/lib/whiteboard/stroke";
import { migrateState } from "../src/lib/whiteboard/persistence";
import {
  looksLikeMouseWheel,
  normalizeWheelDelta,
  resolveWheelAction,
  type RawWheel,
} from "../src/lib/whiteboard/input";
import { exportBoardToSvg } from "../src/lib/whiteboard/export";
import {
  classifyTouch,
  contactLooksLikePalm,
  exceedsLongPressSlop,
  isRejectedTouch,
  LONG_PRESS_SLOP_PX,
  PALM_CONTACT_PX,
  PEN_SUPPRESSION_MS,
} from "../src/lib/whiteboard/palm";
import {
  hitTestElement,
  isElementInPolygon,
  screenSpaceOverlayTransform,
} from "../src/lib/whiteboard/geometry";
```

```

import type {
  Board,
  Folder,
  ToolType,
  StrokeElement,
  TextElement,
  StickyElement,
  ConnectionElement,
  RectangleElement,
  WhiteboardElement,
} from "../src/lib/whiteboard/types";
import { useWhiteboard } from "../src/store/whiteboard";
import { getToolColor, getToolSize, setToolInk, PEN_TYPES } from "../src/lib/whiteboard/types";
import { isUndoCombo, isRedoCombo, type ComboEvent } from "../src/lib/whiteboard/shortcuts";
import {
  mergeWorkspaces,
  parseWorkspace,
  serializeWorkspace,
  WorkspaceParseError,
} from "../src/lib/whiteboard/workspace";

console.log("=== Regression Suite: 2026-08-17 Fix Pass ===");

function makeBoard(id: string, elements: WhiteboardElement[] = []): Board {
  return {
    id,
    name: `Board ${id}`,
    folderId: null,
    elements,
    camera: { x: 0, y: 0, zoom: 1 },
    createdAt: 1,
    updatedAt: 1,
    tags: [],
    starred: false,
    accessedAt: 1,
  };
}

function makeRect(id: string, x = 0, y = 0, w = 100, h = 80): RectangleElement {
  return {
    id, type: "rectangle", x, y, width: w, height: h, rotation: 0, opacity: 1,
    strokeColor: "#000", fillColor: "transparent", fillStyle: "solid", dashStyle: "solid",
    strokeWidth: 2, createdAt: 1, updatedAt: 1,
  };
}

// =====
// B1: moveFolder must reject cycles (folder into its own descendant)
// =====
console.log("\n[REG-B1] moveFolder cycle rejection...");
{
  const s = useWhiteboard.getState();
  const a = useWhiteboard.getState().createFolder("A", null);
  const b = useWhiteboard.getState().createFolder("B", a);
  const c = useWhiteboard.getState().createFolder("C", b);
  // Move A into C (its own grandchild) -> must be rejected
  useWhiteboard.getState().moveFolder(a, c);
  const folders = useWhiteboard.getState().folders;
  const folderA = folders.find((f) => f.id === a);
  assert.equal(folderA.parentId, null, "A must stay at root (cycle rejected)");
  // Legit move still works
  useWhiteboard.getState().moveFolder(c, a);
  assert.equal(useWhiteboard.getState().folders.find((f) => f.id === c).parentId, a);
}

// =====
// B5: deleteFolder removes the FULL descendant chain
// =====
console.log("[REG-B5] deleteFolder full-descendant removal...");
{
  const st = useWhiteboard.getState();
  const p = st.createFolder("P", null);
  const child = st.createFolder("child", p);
  const grand = st.createFolder("grand", child);
  const boardInGrand = st.createBoard("kept-board", grand);
  useWhiteboard.getState().deleteFolder(p);
  const s2 = useWhiteboard.getState();
  assert.ok(!s2.folders.some((f) => f.id === child), "child deleted");
}

```

```

    assert.ok(!s2.folders.some((f) => f.id === grand), "grandchild deleted too (was orphaned before)");
    const movedBoard = s2.boards.find((b) => b.id === boardInGrand!);
    assert.equal(movedBoard.folderId, null, "board inside deleted subtree moves to root");
  }

  // =====
  // B4: deleting a NON-active board must not hijack the active board
  // =====
  console.log("[REG-B4] deleteBoard non-active no-hijack...");
  {
    const st = useWhiteboard.getState();
    const active = st.activeBoardId!;
    const other = st.createBoard("background", null);
    useWhiteboard.getState().setActiveBoard(active);
    const before = useWhiteboard.getState();
    const camBefore = { ...before.camera };
    useWhiteboard.getState().deleteBoard(other);
    const after = useWhiteboard.getState();
    assert.equal(after.activeBoardId, active, "active board unchanged");
    assert.deepEqual(after.camera, camBefore, "camera unchanged");
  }

  // =====
  // B4b: cross-board connections referencing a deleted board are cleaned
  // =====
  console.log("[REG-B4b] dangling cross-board connections cleaned...");
  {
    const st = useWhiteboard.getState();
    const boardX = st.createBoard("X", null);
    const boardY = st.createBoard("Y", null);
    const r1 = makeRect("xr1", 0, 0);
    const r2 = makeRect("yr1", 400, 0);
    useWhiteboard.getState().addElementToBoard(boardX, r1, false);
    useWhiteboard.getState().addElementToBoard(boardY, r2, false);
    const conn: ConnectionElement = {
      id: "xconn", type: "connection", x: 0, y: 0, rotation: 0, opacity: 1,
      source: { boardId: boardX, elementId: "xr1", anchor: "auto" },
      target: { boardId: boardY, elementId: "yr1", anchor: "auto" },
      linkType: "relates", routing: "straight", dashStyle: "solid",
      startArrowhead: "none", endArrowhead: "arrow", strokeWidth: 2, createdAt: 1, updatedAt: 1,
    };
    useWhiteboard.getState().addElementToBoard(boardX, conn, false);
    useWhiteboard.getState().deleteBoard(boardY);
    const bx = useWhiteboard.getState().boards.find((b) => b.id === boardX!);
    assert.ok(!bx.elements.some((e) => e.id === "xconn"), "connection to deleted board removed");
  }

  // =====
  // B8: history snapshots must be DEEP clones
  // =====
  console.log("[REG-B8] history deep-clone isolation...");
  {
    const st = useWhiteboard.getState();
    const boardId = st.createBoard("history-board", null);
    const stroke: StrokeElement = {
      id: "s1", type: "stroke", x: 0, y: 0, rotation: 0, opacity: 1,
      points: [[0, 0, 0.5], [10, 0, 0.5], [20, 5, 0.5]],
      color: "#000", strokeWidth: 3, strokeStyle: "pen", createdAt: 1, updatedAt: 1,
    };
    useWhiteboard.getState().addElements([stroke], true);
    useWhiteboard.getState().commitHistory();
    // Mutate the LIVE element's nested points in place (simulating a rogue writer)
    const live = useWhiteboard.getState().boards.find((b) => b.id === boardId!).elements[0] as StrokeElement;
    live.points.push([999, 999, 0.5]);
    // Undo must restore the snapshot WITHOUT the in-place mutation leaking in
    useWhiteboard.getState().undo();
    const restored = useWhiteboard.getState().boards.find((b) => b.id === boardId!).elements[0] as StrokeElement;
    assert.equal(restored.points.length, 3, "snapshot must not share points array with live state");
  }

  // =====
  // D3: locked elements cannot be deleted via deleteElements / excluded by selectAll
  // =====
  console.log("[REG-D3] lock protection...");
  {
    const st = useWhiteboard.getState();
    const boardId = st.createBoard("lock-board", null);
  }

```

```

const locked = makeRect("locked-1");
locked.locked = true;
const free = makeRect("free-1", 300, 0);
useWhiteboard.getState().addElements([locked, free], false);
useWhiteboard.getState().setSelected(["locked-1", "free-1"]);
useWhiteboard.getState().deleteElements(useWhiteboard.getState().selectedIds, true);
const els = useWhiteboard.getState().boards.find((b) => b.id === boardId)!.elements;
assert.ok(els.some((e) => e.id === "locked-1"), "locked element survives delete");
assert.ok(!els.some((e) => e.id === "free-1"), "unlocked element deleted");
useWhiteboard.getState().selectAll();
assert.ok(
  !useWhiteboard.getState().selectedIds.includes("locked-1"),
  "selectAll excludes locked"
);
}

// =====
// D6: undo/redo are blocked while the inline editor is open
// =====
console.log("[REG-D6] undo/redo blocked during editing...");
{
  const st = useWhiteboard.getState();
  const boardId = st.createBoard("edit-board", null);
  const txt: TextElement = {
    id: "t1", type: "text", x: 0, y: 0, width: 200, height: 50, rotation: 0, opacity: 1,
    text: "before", color: "#000", fontSize: 18, fontFamily: "Inter", fontWeight: 400,
    align: "left", createdAt: 1, updatedAt: 1,
  };
  useWhiteboard.getState().addElements([txt], true);
  useWhiteboard.getState().commitHistory();
  useWhiteboard.getState().setEditingId("t1");
  useWhiteboard.getState().undo(); // must be a no-op mid-edit
  const stillThere = useWhiteboard.getState().boards.find((b) => b.id === boardId)!.elements.find((e) => e.id
    === "t1");
  assert.ok(stillThere, "undo mid-edit must not revert the board");
  useWhiteboard.getState().setEditingId(null);
}

// =====
// B15: bringForward/sendBackward are single-layer steps
// =====
console.log("[REG-B15] single-step layer ordering...");
{
  const st = useWhiteboard.getState();
  const boardId = st.createBoard("zorder-board", null);
  const a = makeRect("z-a");
  const b = makeRect("z-b", 10, 10);
  const c = makeRect("z-c", 20, 20);
  useWhiteboard.getState().addElements([a, b, c], false);
  useWhiteboard.getState().bringForward(["z-a"]);
  let order = useWhiteboard.getState().boards.find((x) => x.id === boardId)!.elements.map((e) => e.id);
  assert.deepEqual(order, ["z-b", "z-a", "z-c"], "bringForward swaps one layer, not to front");
  useWhiteboard.getState().sendBackward(["z-c"]);
  order = useWhiteboard.getState().boards.find((x) => x.id === boardId)!.elements.map((e) => e.id);
  assert.deepEqual(order, ["z-b", "z-c", "z-a"], "sendBackward swaps one layer, not to back");
}

// =====
// B7/D13: camera is persisted into boards and survives switching
// =====
console.log("[REG-B7] camera persistence across board switches...");
{
  const st = useWhiteboard.getState();
  const b1 = st.createBoard("cam1", null);
  const b2 = st.createBoard("cam2", null);
  useWhiteboard.getState().setActiveBoard(b1);
  useWhiteboard.getState().panCamera(120, 60);
  useWhiteboard.getState().setActiveBoard(b2);
  const persisted = useWhiteboard.getState().getPersistedState();
  const cam1 = persisted.boards.find((b) => b.id === b1)!.camera;
  assert.equal(cam1.x, -120, "outgoing board camera captured on switch");
  assert.equal(cam1.y, -60, "outgoing board camera captured on switch (y)");
}

// =====
// B10: saved state with ZERO boards is honored (no demo re-injection logic here;
// we verify getPersistedState round-trips an empty board list intact)
// =====

```

```

console.log("[REG-B10] empty-boards state integrity...");
{
  const persisted = useWhiteboard.getState().getPersistedState();
  assert.ok(Array.isArray(persisted.boards), "boards always an array");
  assert.ok(Array.isArray(persisted.folders), "folders always an array");
}

// =====
// B11: WikiLink alias edge cases
// =====
console.log("[REG-B11] wikilink alias semantics...");
{
  const parsed = parseText("see [[Target|]] and [[A|b|c]] and [[Plain|]]");
  assert.equal(parsed.links.length, 3, "all three links match");
  assert.equal(parsed.links[0].display, "Target", "[[Target|]] displays target");
  assert.equal(parsed.links[1].display, "b", "[[A|b|c]] displays first alias segment");
  assert.equal(parsed.links[1].target, "A", "[[A|b|c]] targets A");
  assert.equal(parsed.links[2].display, "Plain", "plain link unchanged");
}

// =====
// B9/B12/D10: export escaping, alignment anchors, custom link colors, blend
// =====
console.log("[REG-B9/B12/D10] export serialization...");
{
  const txt: TextElement = {
    id: "t", type: "text", x: 100, y: 100, width: 300, height: 50, rotation: 0, opacity: 1,
    text: "centered [[Link|shown]]", color: "#111", fontSize: 20, fontFamily: "Inter",
    fontWeight: 400, align: "center", createdAt: 1, updatedAt: 1,
  };
  const hostile = makeRect("hostile");
  hostile.fillColor = `red` onload="alert(1)";
  hostile.fillStyle = "solid";
  const board = makeBoard("exp", [txt, hostile]);
  const svg = exportBoardToSvg(board, { background: "white" });
  assert.ok(!svg.includes(`red` onload=), "fill attribute injection escaped");
  // Center-aligned text anchors at x + width/2
  assert.ok(svg.includes('x="250.00"'), "center anchor uses x + width/2");
  // Wikilink rendered as its alias display text, not raw brackets
  assert.ok(svg.includes(">centered shown<"), "wikilink alias text exported");
  assert.ok(!svg.includes("[[Link|shown]]"), "raw wikilink markup not exported");

  // Custom link type color flows through export
  const r1 = makeRect("ca", 0, 0);
  const r2 = makeRect("cb", 400, 0);
  const conn: ConnectionElement = {
    id: "cc", type: "connection", x: 0, y: 0, rotation: 0, opacity: 1,
    source: { boardId: "exp", elementId: "ca", anchor: "auto" },
    target: { boardId: "exp", elementId: "cb", anchor: "auto" },
    linkType: "custom:xyz", routing: "straight", dashStyle: "solid",
    startArrowhead: "arrow", endArrowhead: "arrow", strokeWidth: 2, createdAt: 1, updatedAt: 1,
  };
  const board2 = makeBoard("exp2", [r1, r2, conn]);
  const svg2 = exportBoardToSvg(board2, [{ id: "custom:xyz", name: "My Link", color: "#ff00aa" }]);
  assert.ok(svg2.includes("#ff00aa"), "custom link color used in export");
  // Start arrowhead rendered (both heads present -> at least 2 filled arrow paths)
  const arrowPaths = (svg2.match(/fill="#ff00aa"/g) || []).length;
  assert.ok(arrowPaths >= 2, "start AND end arrowheads exported");

  // Highlighter blend mode preserved
  const hl: StrokeElement = {
    id: "hl", type: "stroke", x: 0, y: 0, rotation: 0, opacity: 1,
    points: [[0, 0, 0.5], [50, 10, 0.5], [100, 20, 0.5]],
    color: "#ffff00", strokeWidth: 20, strokeStyle: "highlighter", createdAt: 1, updatedAt: 1,
  };
  const svg3 = exportBoardToSvg(makeBoard("exp3", [hl]));
  assert.ok(svg3.includes("mix-blend-mode: multiply"), "highlighter multiply blend exported");
}

// =====
// A10/A10b: rotated stroke & line hit-testing follows visible geometry
// =====
console.log("[REG-A10] rotated stroke/line hit-test...");
{
  // Stroke along y=0 from x=0..100, rotated 90° about its center -> vertical
  const stroke: StrokeElement = {
    id: "rot", type: "stroke", x: 0, y: 0, rotation: 0, opacity: 1,
    points: [[0, 0, 0.5], [50, 0, 0.5], [100, 0, 0.5]],
  };

```

```

    color: "#000", strokeWidth: 6, strokeStyle: "pen", createdAt: 1, updatedAt: 1,
  };
  assert.ok(hitTestElement(stroke, 50, 0, 6), "unrotated stroke hit at midpoint");
  assert.ok(!hitTestElement(stroke, 50, 60, 6), "unrotated stroke miss far away");
  const rotated = { ...stroke, rotation: Math.PI / 2 };
  // After 90° rotation about center (50,0): the stroke runs vertically at x=50
  assert.ok(hitTestElement(rotated, 50, -40, 6), "rotated stroke hit along rotated axis");
  assert.ok(hitTestElement(rotated, 50, 40, 6), "rotated stroke hit other direction");
  assert.ok(!hitTestElement(rotated, 10, 0, 6), "rotated stroke misses its OLD position");

  const line = {
    id: "rl", type: "line" as const, x: 0, y: 0, rotation: 0, opacity: 1,
    x2: 100, y2: 0, strokeColor: "#000", strokeWidth: 4, dashStyle: "solid" as const,
    createdAt: 1, updatedAt: 1,
  };
  const rotatedLine = { ...line, rotation: Math.PI / 2 };
  assert.ok(hitTestElement(rotatedLine, 50, -30, 5), "rotated line hit on rotated axis");
  assert.ok(!hitTestElement(rotatedLine, 10, 0, 5), "rotated line misses old position");
}

// =====
// A11: lasso polygon containment uses ROTATED corners
// =====
console.log("[REG-A11] rotated lasso containment...");
{
  const rect = makeRect("r", 0, 0, 100, 100);
  const rotated = { ...rect, rotation: Math.PI / 2 }; // same AABB, same corners for 90°
  // Use 45° so rotated corners differ from AABB corners
  const diag = { ...rect, rotation: Math.PI / 4 };
  // Lasso a small polygon around the top corner of the rotated square.
  // Rotated 45°, the top corner sits at (50, 50 - 70.71) ≈ (50, -20.7)
  const lasso = [
    { x: 40, y: -30 },
    { x: 60, y: -30 },
    { x: 60, y: -10 },
    { x: 40, y: -10 },
  ];
  assert.ok(isElementInPolygon(diag, lasso), "lasso catches rotated corner region");
  // A polygon far from the shape must not contain it
  const far = [{ x: 500, y: 500 }, { x: 600, y: 500 }, { x: 550, y: 600 }];
  assert.ok(!isElementInPolygon(diag, far), "far lasso misses");
  void rotated;
}

// =====
// B2-equivalent: corrupt persisted payload sanitization happens in migrate()
// (indirect test: the store must never expose a non-array elements list)
// =====
console.log("[REG-B2] store invariant: elements always an array...");
{
  for (const b of useWhiteboard.getState().boards) {
    assert.ok(Array.isArray(b.elements), `board ${b.id} elements is an array`);
    assert.ok(typeof b.camera.zoom === "number" && b.camera.zoom > 0, `board ${b.id} camera valid`);
  }
}

// =====
// B1-store: addElementToBoard writes without switching the active board
// =====
console.log("[REG-D2] addElementToBoard isolation...");
{
  const st = useWhiteboard.getState();
  const keepActive = st.activeBoardId!;
  const target = st.createBoard("bg-write", null);
  useWhiteboard.getState().setActiveBoard(keepActive);
  const before = useWhiteboard.getState();
  const histBefore = before.history.past.length;
  useWhiteboard.getState().addElementToBoard(target, makeRect("bg-el", false));
  const after = useWhiteboard.getState();
  assert.equal(after.activeBoardId, keepActive, "active board unchanged by background write");
  assert.equal(after.camera, before.camera, "camera untouched");
  assert.ok(after.boards.find((b) => b.id === target)!?.elements.some((e) => e.id === "bg-el"), "element written to target board");
  assert.equal(after.history.past.length, histBefore, "history untouched");
}

// =====
// F1: Link Manager links whose endpoint is a FOLDER must appear in the graph.

```

```

// buildGraph used to key edges off endpoint.boardId only, so a board→folder
// link collapsed to a self-loop and was dropped, and a folder→board link was
// drawn from the wrong (board) node. Endpoints now resolve through elementId.
// =====
console.log("[REG-F1] graph edges resolve folder endpoints...");
{
  const folders: Folder[] = [
    { id: "fRoot", name: "Root", createdAt: 1, parentId: null, expanded: true },
    { id: "fKid", name: "Kid", createdAt: 1, parentId: "fRoot", expanded: true },
  ];
  const mkLink = (id: string, filedOn: string, sourceId: string, targetId: string): ConnectionElement => ({
    id,
    type: "connection",
    x: 0,
    y: 0,
    rotation: 0,
    opacity: 1,
    source: { boardId: filedOn, elementId: sourceId, anchor: "auto" },
    // This mirrors what LinkManager writes: only a BOARD target lands in
    // boardId; a folder (or an on-canvas shape) leaves the storage board's id
    // there, so boardId alone cannot identify the endpoint.
    target: { boardId: targetId.startsWith("b") ? targetId : filedOn, elementId: targetId, anchor: "auto" },
    linkType: "leads_to",
    routing: "curved",
    dashStyle: "solid",
    startArrowhead: "none",
    endArrowhead: "arrow",
    strokeWidth: 2,
    createdAt: 1,
    updatedAt: 1,
  });

  const bA = makeBoard("bA");
  const bB = makeBoard("bB");

  // board -> folder (used to vanish entirely)
  bA.elements = [mkLink("l1", "bA", "bA", "fKid")];
  const g1 = buildGraph([bA, bB], [], folders);
  const e1 = g1.edges.find((e) => e.id === "conn-l1");
  assert.ok(e1, "board→folder link produces a graph edge");
  assert.equal(e1!.source, "bA", "edge starts at the source board");
  assert.equal(e1!.target, "fKid", "edge ends at the FOLDER node");

  // folder -> board (used to start from an arbitrary board inside the folder)
  bA.elements = [mkLink("l2", "bA", "fRoot", "bB")];
  const g2 = buildGraph([bA, bB], [], folders);
  const e2 = g2.edges.find((e) => e.id === "conn-l2");
  assert.ok(e2, "folder→board link produces a graph edge");
  assert.equal(e2!.source, "fRoot", "edge starts at the FOLDER node, not its first board");
  assert.equal(e2!.target, "bB", "edge ends at the target board");

  // folder -> folder
  bA.elements = [mkLink("l3", "bA", "fRoot", "fKid")];
  const g3 = buildGraph([bA, bB], [], folders);
  const e3 = g3.edges.find((e) => e.id === "conn-l3");
  assert.ok(e3, "folder→folder link produces a graph edge");
  assert.equal(e3!.source, "fRoot", "folder→folder source");
  assert.equal(e3!.target, "fKid", "folder→folder target");

  // A plain on-canvas arrow between two shapes on ONE board is NOT a
  // board-level relationship and must stay out of the global graph.
  bA.elements = [mkLink("l4", "bA", "shape-1", "shape-2")];
  const g4 = buildGraph([bA, bB], [], folders);
  assert.equal(
    g4.edges.find((e) => e.id === "conn-l4"),
    undefined,
    "same-board shape-to-shape arrow stays out of the global graph"
  );

  // A custom label overrides the link-type name on the edge.
  const labelled = mkLink("l5", "bA", "bA", "bB");
  labelled.label = "blocks release";
  bA.elements = [labelled];
  const g5 = buildGraph([bA, bB], [], folders);
  assert.equal(
    g5.edges.find((e) => e.id === "conn-l5")!.label,
    "blocks release",
    "custom label wins over the type name"
  );
}

```

```

    });
}

// =====
// F2: Favorite pens bar – pins are independent (same pen type, different
// colors), reorder is a real move, and applying a pin writes to the ACTIVE
// tool's style fields (not penColor, which the canvas ignores for other pens).
// =====
console.log("[REG-F2] favorite pens: pin, reorder, apply...");
{
  useWhiteboard.getState().pinnedPens.forEach((p) => useWhiteboard.getState().removePinnedPen(p.id));

  const redHl = useWhiteboard.getState().addPinnedPen({ tool: "highlighter", color: "#ef4444", size: 20 });
  const blueHl = useWhiteboard.getState().addPinnedPen({ tool: "highlighter", color: "#3b82f6", size: 8 });
  const pen = useWhiteboard.getState().addPinnedPen({ tool: "pen", color: "#1e293b", size: 3 });

  assert.equal(useWhiteboard.getState().pinnedPens.length, 3, "three independent pins");
  assert.equal(
    useWhiteboard.getState().pinnedPens.filter((p) => p.tool === "highlighter").length,
    2,
    "same pen type can be pinned twice in different colors"
  );

  // Reorder: move the last pin to the front.
  useWhiteboard.getState().reorderPinnedPens(2, 0);
  assert.deepEqual(
    useWhiteboard.getState().pinnedPens.map((p) => p.id),
    [pen, redHl, blueHl],
    "reorder moves the pin, it does not swap or duplicate"
  );

  // Out-of-range reorders must be no-ops, not corruptions.
  const beforeOrder = useWhiteboard.getState().pinnedPens.map((p) => p.id);
  useWhiteboard.getState().reorderPinnedPens(0, 99);
  useWhiteboard.getState().reorderPinnedPens(-1, 0);
  assert.deepEqual(
    useWhiteboard.getState().pinnedPens.map((p) => p.id),
    beforeOrder,
    "bad indices are ignored"
  );

  // Applying a highlighter pin must set the HIGHLIGHTER's ink – writing the
  // pen's would leave the highlighter drawing its old colour. Asserted through
  // the accessor rather than a named field, so this keeps testing the behaviour
  // if the storage shape changes again.
  useWhiteboard.getState().applyPinnedPen(redHl);
  const applied = useWhiteboard.getState();
  assert.equal(applied.tool, "highlighter", "pin activates its own tool");
  assert.equal(getToolColor(applied.style, "highlighter"), "#ef4444", "colour lands on the highlighter");
  assert.equal(getToolSize(applied.style, "highlighter"), 20, "size lands on the highlighter");
  // ...and must NOT have leaked onto another pen.
  assert.notEqual(getToolColor(applied.style, "gel"), "#ef4444", "another pen's ink is untouched");

  useWhiteboard.getState().applyPinnedPen(blueHl);
  assert.equal(
    getToolColor(useWhiteboard.getState().style, "highlighter"),
    "#3b82f6",
    "second same-type pin switches color"
  );

  // Deleting a pin leaves the rest intact.
  useWhiteboard.getState().removePinnedPen(redHl);
  assert.deepEqual(
    useWhiteboard.getState().pinnedPens.map((p) => p.id),
    [pen, blueHl],
    "removing a pin leaves the others in order"
  );
}

// =====
// F3: Selection overlay drew its box and resize handles in the wrong place at
// any zoom !== 1. The overlay computes SCREEN coordinates inside the canvas'
// world group, so it must apply the exact inverse of that group's transform.
// The three operations were listed in forward order instead of reversed, which
// composes to the identity only at zoom 1; elsewhere it offsets everything by
// (camera + viewport/2) * (1 - zoom) – handles far from the image they resize.
// =====
console.log("[REG-F3] selection overlay transform is a true inverse...");

```

```

{
  // Parse the transform string and apply it, then apply the canvas' world
  // transform: the round trip must return the original screen point exactly.
  const applyOverlayThenWorld = (
    screen: { x: number; y: number },
    camera: { x: number; y: number; zoom: number },
    viewport: { width: number; height: number }
  ) => {
    const t = screenSpaceOverlayTransform(camera, viewport);
    const nums = t.match(/-?\d+(\.\d+)?(e-?\d+)?/g)?.map(Number);
    const [t1x, t1y, scale, t2x, t2y] = nums;
    // SVG applies a transform list right-to-left.
    const afterT2 = { x: screen.x + t2x, y: screen.y + t2y };
    const afterScale = { x: afterT2.x * scale, y: afterT2.y * scale };
    const local = { x: afterScale.x + t1x, y: afterScale.y + t1y };
    // Canvas world group: translate(vw/2, vh/2) scale(z) translate(-cx, -cy)
    return {
      x: (local.x - camera.x) * camera.zoom + viewport.width / 2,
      y: (local.y - camera.y) * camera.zoom + viewport.height / 2,
    };
  };
};

const viewport = { width: 1600, height: 900 };
const point = { x: 640, y: 380 };
for (const zoom of [0.25, 0.5, 0.75, 1, 1.5, 2.4, 6]) {
  for (const camera of [
    { x: 0, y: 0, zoom },
    { x: 320, y: -180, zoom },
    { x: -2100, y: 940, zoom },
  ]) {
    const out = applyOverlayThenWorld(point, camera, viewport);
    const drift = Math.hypot(out.x - point.x, out.y - point.y);
    assert.ok(
      drift < 1e-6,
      `overlay transform must round-trip at zoom ${zoom}, camera (${camera.x},${camera.y}) - drift was ${drift}
      ft.toFixed(2))px`
    );
  }
}
}

// =====
// F4: Undo/redo combos matched on event.key, which carries the SHIFTED/caps
// character. Ctrl+Shift+Z therefore arrived as "Z" and never matched "z", so
// the documented redo combo was dead, and Ctrl+Z stopped undoing with Caps
// Lock on. Matching moved to event.code (the physical key).
// =====
console.log("[REG-F4] undo/redo key combos (case + layout safe)...");
{
  const ev = (code: string, mods: Partial<ComboEvent> = {}): ComboEvent => ({
    code,
    ctrlKey: true,
    metaKey: false,
    shiftKey: false,
    ...mods,
  });

  assert.ok(isUndoCombo(ev("KeyZ")), "Ctrl+Z undoes");
  assert.ok(isUndoCombo({ ...ev("KeyZ"), ctrlKey: false, metaKey: true }), "Cmd+Z undoes");
  assert.ok(!isUndoCombo(ev("KeyZ", { shiftKey: true })), "Ctrl+Shift+Z is NOT undo");
  assert.ok(!isUndoCombo({ ...ev("KeyZ"), ctrlKey: false }), "bare Z is not undo");

  assert.ok(isRedoCombo(ev("KeyY")), "Ctrl+Y redoes");
  assert.ok(isRedoCombo({ ...ev("KeyY"), ctrlKey: false, metaKey: true }), "Cmd+Y redoes");
  assert.ok(
    isRedoCombo(ev("KeyZ", { shiftKey: true })),
    "Ctrl+Shift+Z redoes - this is the combo that was dead"
  );
  assert.ok(!isRedoCombo(ev("KeyZ")), "plain Ctrl+Z is not redo");
  assert.ok(!isRedoCombo({ ...ev("KeyY"), ctrlKey: false }), "bare Y is not redo");

  // Undo and redo must never both claim the same event.
  for (const code of ["KeyZ", "KeyY"]) {
    for (const shiftKey of [true, false]) {
      const e = ev(code, { shiftKey });
      assert.ok(
        !(isUndoCombo(e) && isRedoCombo(e)),
        `${code}${shiftKey ? "+Shift" : ""} must not match both undo and redo`
      );
    }
  }
}

```

```

    });
  }
}

// =====
// F5: Workspace backup/restore/merge – the rules that decide whether work
// survives syncing the same account on two devices.
// =====
console.log("[REG-F5] workspace backup + per-board sync merge...");
{
  const snap = (boards: Board[], folders: Folder[] = []) => ({
    folders,
    boards,
    activeBoardId: boards[0]?.id ?? null,
    activeFolderId: null,
    customLinkTypes: [] as Array<{ id: string; name: string; color: string }>,
  });
  const board = (id: string, updatedAt: number, elements: WhiteboardElement[] = []): Board => ({
    ...makeBoard(id, elements),
    updatedAt,
  });

  // --- round trip -----
  const original = snap([board("b1", 100, [makeRect("r1", 5, 6)])]);
  const restored = parseWorkspace(serializeWorkspace(original, "test device"));
  assert.equal(restored.boards.length, 1, "round trip keeps the board");
  assert.equal(restored.boards[0].elements.length, 1, "round trip keeps elements");
  assert.equal((restored.boards[0].elements[0] as RectangleElement).x, 5, "element data intact");

  // --- rejects foreign / corrupt files -----
  assert.throws(() => parseWorkspace("not json"), WorkspaceParseError, "rejects non-JSON");
  assert.throws(
    () => parseWorkspace(JSON.stringify({ app: "something-else", format: 1, boards: [] })),
    WorkspaceParseError,
    "rejects a file from another app"
  );
  assert.throws(
    () => parseWorkspace(JSON.stringify({ app: "pronotes", format: 99, boards: [] })),
    WorkspaceParseError,
    "rejects a backup from a newer version"
  );

  // --- the merge that matters -----
  // Board A edited only locally, board B edited only remotely: BOTH must
  // survive. Whole-document last-write-wins would destroy one of them.
  const local = snap([board("A", 200), board("B", 100)]);
  const remote = snap([board("A", 100), board("B", 300), board("C", 50)]);
  const { merged, report } = mergeWorkspaces(local, remote);

  const byId = new Map(merged.boards.map((b) => [b.id, b]));
  assert.equal(byId.size, 3, "union of both devices' boards");
  assert.equal(byId.get("A")!.updatedAt, 200, "locally-newer board keeps the LOCAL version");
  assert.equal(byId.get("B")!.updatedAt, 300, "remotely-newer board takes the REMOTE version");
  assert.ok(byId.has("C"), "board that exists only remotely is pulled in");
  assert.equal(report.added, 1, "one board added");
  assert.equal(report.updatedFromRemote, 1, "one board updated from remote");
  assert.equal(report.keptLocal, 1, "one board kept local");

  // The device keeps looking at whatever it was looking at.
  assert.equal(merged.activeBoardId, "A", "active board preserved across a merge");

  // Custom link types are unioned – losing one would render its connections
  // as unlabelled grey lines on the device that lost it.
  const l2 = { ...snap([board("A", 1)]), customLinkTypes: [{ id: "c1", name: "L", color: "#111" }] };
  const r2 = { ...snap([board("A", 1)]), customLinkTypes: [{ id: "c2", name: "R", color: "#222" }] };
  const merged2 = mergeWorkspaces(l2, r2).merged;
  assert.equal(merged2.customLinkTypes!.length, 2, "custom link types are unioned, never dropped");

  // A merge must never lose a board that only one side has, in either direction.
  const onlyLocal = mergeWorkspaces(snap([board("X", 1)]), snap([])).merged;
  assert.ok(onlyLocal.boards.some((b) => b.id === "X"), "empty remote does not wipe local boards");
}

// =====
// F6: A tapered stroke end is emitted by perfect-freehand as SEVERAL
// COINCIDENT vertices. Zero-length edges have no defined angle, so the
// sharp-corner test bailed out and the tip was smoothed away – the stroke

```

```
// finished in a flat chisel cut instead of a point. Duplicates are now
// collapsed before the path is built.
// =====
console.log("[REG-F6] stroke ends finish in a point, not a flat cut...");
{
  // An outline converging to a point via three coincident vertices, exactly as
  // perfect-freehand emits for a tapered end.
  const tip: number[] = [100, 0];
  const outline: number[][] = [
    [0, 20],
    [50, 14],
    tip,
    [...tip],
    [...tip],
    [50, -14],
    [0, -20],
  ];
  const path = getSvgPathFromStroke(outline, true);
  assert.ok(
    path.includes(`L ${tip[0].toFixed(2)},${tip[1].toFixed(2)}`),
    "the tip must be an on-curve point, not rounded off into a flat end"
  );

  // A smooth outline must still be fully curved, so collapsing duplicates
  // cannot regress into faceting ordinary strokes.
  const circle: number[][] = [];
  for (let i = 0; i < 40; i++) {
    circle.push([Math.cos((i / 40) * Math.PI * 2) * 60, Math.sin((i / 40) * Math.PI * 2) * 60]);
  }
  assert.equal(
    (getSvgPathFromStroke(circle, true).match(/ L /g) || []).length,
    0,
    "a smooth outline still uses no straight segments"
  );

  // Degenerate input (every point identical) must not crash or emit NaN.
  const degenerate = [
    [5, 5],
    [5, 5],
    [5, 5],
    [5, 5],
  ];
  assert.ok(
    !getSvgPathFromStroke(degenerate, true).includes("NaN"),
    "coincident-only outline produces no NaN"
  );
}

// =====
// F7: Tool setup and preferences were never persisted. Pen colours/sizes, font
// size, sticky colour, snap-to-grid, the minimap, the favourites bar and – on a
// tablet, most painfully – the stylus mode all reset to defaults on every
// launch, because getPersistedState() omitted them and their setters never
// scheduled a save.
// =====
console.log("[REG-F7] tool setup and preferences survive a restart...");
{
  const st = useWhiteboard.getState();

  st.setStyle({ fontSize: 42, penColor: "#ff00aa", penSize: 19, stickyColor: "green" });
  st.setTouchSettings({ mode: "stylus-only", palmRejection: true });
  if (!useWhiteboard.getState().snapToGrid) useWhiteboard.getState().toggleSnapToGrid();

  const persisted = useWhiteboard.getState().getPersistedState();

  // Everything the user configured must be in the saved payload.
  assert.equal(persisted.style?.fontSize, 42, "font size is saved");
  assert.equal(persisted.style?.penColor, "#ff00aa", "pen colour is saved");
  assert.equal(persisted.style?.penSize, 19, "pen size is saved");
  assert.equal(persisted.style?.stickyColor, "green", "sticky colour is saved");
  assert.equal(persisted.touchSettings?.mode, "stylus-only", "stylus mode is saved");
  assert.equal(persisted.touchSettings?.palmRejection, true, "palm rejection is saved");
  assert.equal(persisted.snapToGrid, true, "snap-to-grid is saved");
  assert.equal(typeof persisted.minimapOpen, "boolean", "minimap visibility is saved");
  assert.equal(typeof persisted.pinnedBarOpen, "boolean", "favourites bar visibility is saved");

  // A save produced now must round-trip through the loader's validation.
  const reloaded = migrateState(persisted as never) as typeof persisted;
}
```

```

assert.equal(reloaded.style?.fontSize, 42, "font size survives migrate()");
assert.equal(reloaded.touchSettings?.mode, "stylus-only", "stylus mode survives migrate()");
assert.equal(reloaded.snapToGrid, true, "snap-to-grid survives migrate()");

// Corrupt preferences must be dropped, not merged in – init() falls back to
// the defaults for anything that comes back undefined.
const corruptInput = {
  ...(persisted as unknown as Record<string, unknown>),
  style: "not an object",
  touchSettings: 42,
  tool: "definitely-not-a-tool",
  snapToGrid: "yes",
};
const corrupt = migrateState(corruptInput as never) as typeof persisted;
assert.equal(corrupt.style, undefined, "a non-object style is dropped");
assert.equal(corrupt.touchSettings, undefined, "non-object touch settings are dropped");
assert.equal(corrupt.tool, undefined, "an unknown tool is dropped");
assert.equal(corrupt.snapToGrid, undefined, "a non-boolean toggle is dropped");
}

// =====
// F8: Pixel eraser. Point-sampling failed in ways that made the eraser feel
// broken, all measured on a stroke with 30px point spacing:
// - an eraser passing BETWEEN two recorded points erased nothing;
// - a 24px eraser removed 60px of stroke, because a cut could only land on
//   an existing point;
// - only the CURRENT position was tested, so a fast drag skipped gaps.
// The eraser is now a swept capsule and each segment is clipped analytically.
// =====
console.log("[REG-F8] pixel eraser clips segments along the swept path...");
{
  // Deliberately sparse, like a quickly drawn stroke.
  const sparse: Array<number, number, number> = [];
  for (let x = 0; x <= 300; x += 30) sparse.push([x, 0, 0.5]);

  // 1. An eraser BETWEEN two recorded points must still cut.
  assert.ok(
    eraseFromStroke(sparse, 15, 0, 15, 0, 12),
    "an eraser between two recorded points must still erase"
  );

  // 2. The cut must land at the eraser's EDGE, not on a recorded point.
  const cut = eraseFromStroke(sparse, 150, 0, 150, 0, 12);
  assert.equal(cut!.length, 2, "erasing the middle splits the stroke in two");
  const leftEnd = cut![0][cut![0].length - 1][0];
  const rightStart = cut![1][0][0];
  assert.ok(
    Math.abs(leftEnd - 138) < 0.5 && Math.abs(rightStart - 162) < 0.5,
    `cut must land on the eraser edge (138/162), got ${leftEnd}/${rightStart}`
  );
  assert.ok(
    rightStart - leftEnd < 25,
    `a 24px eraser must not remove ${rightStart - leftEnd.toFixed(0)}px of stroke`
  );

  // 3. A fast drag erases the whole swept path, not just its end.
  assert.deepEqual(
    eraseFromStroke(sparse, 0, 0, 300, 0, 12),
    [],
    "a drag along the stroke erases all of it"
  );

  // Erasing one end trims it and leaves a single piece.
  const trimmed = eraseFromStroke(sparse, 0, 0, 0, 0, 25);
  assert.equal(trimmed!.length, 1, "erasing an end leaves one piece");
  assert.ok(trimmed![0][0][0] >= 24.9, "the erased end is gone up to the eraser edge");

  // A miss reports no change, so an erase drag over empty canvas does no work.
  assert.equal(
    eraseFromStroke(sparse, 0, 400, 300, 400, 12),
    null,
    "a miss reports no change"
  );

  // A perpendicular sweep across the stroke cuts it, even between points.
  const across = eraseFromStroke(sparse, 15, -50, 15, 50, 5);
  assert.ok(across && across.length === 2, "a sweep across the line splits it");
}

```

```

// Surviving fragments must never be a single stray point.
const short: Array<number, number, number> = [
  [0, 0, 0.5],
  [10, 0, 0.5],
  [100, 0, 0.5],
];
assert.ok(
  (eraseFromStroke(short, 5, 0, 5, 0, 12) ?? []).every((run) => run.length >= 2),
  "single-point fragments are discarded, not left as dots"
);

// Direction independence: the cut width must depend on the eraser SIZE and
// how far it travelled, never on the angle between eraser and stroke.
const gapOf = (runs: Array<Array<number, number, number>> | null): number => {
  if (!runs || runs.length < 2) return -1;
  const a = runs[0][runs[0].length - 1];
  const b = runs[1][0];
  return Math.hypot(b[0] - a[0], b[1] - a[1]);
};
const R = 12;
const horiz: Array<number, number, number> = [];
for (let x = 0; x <= 300; x += 30) horiz.push([x, 0, 0.5]);
const vert: Array<number, number, number> = [];
for (let y = 0; y <= 300; y += 30) vert.push([0, y, 0.5]);
const diag: Array<number, number, number> = [];
for (let i = 0; i <= 10; i++) diag.push([i * 30, i * 30, 0.5]);

// A STATIONARY eraser cuts exactly its own diameter, whatever the stroke's
// orientation – this is what "direction must not matter" means.
for (const [name, pts, x, y] of [
  ["horizontal", horiz, 150, 0],
  ["vertical", vert, 0, 150],
  ["diagonal", diag, 150, 150],
] as const) {
  const g = gapOf(eraseFromStroke(pts, x, y, x, y, R));
  assert.ok(
    Math.abs(g - 2 * R) < 1.5,
    `${name} stroke: a stationary eraser must cut its own diameter (${2 * R}px), got ${g.toFixed(1)}px`
  );
}

// A sweep ACROSS a stroke also cuts one diameter – the travel is
// perpendicular, so it must not widen the cut.
assert.ok(
  Math.abs(gapOf(eraseFromStroke(horiz, 150, -40, 150, 40, R)) - 2 * R) < 1.5,
  "a perpendicular sweep cuts exactly the eraser's diameter"
);

// A sweep ALONG a stroke removes travel + diameter, because that is the area
// the eraser actually passed over.
const along = gapOf(eraseFromStroke(horiz, 140, 0, 160, 0, R));
assert.ok(
  Math.abs(along - (20 + 2 * R)) < 1.5,
  `a 20px sweep along the stroke removes 20+${2 * R}px, got ${along.toFixed(1)}px`
);

// A dot (single-point stroke) is removed only when actually touched.
const dot: Array<number, number, number> = [[50, 50, 0.5]];
assert.deepEqual(eraseFromStroke(dot, 50, 50, 50, 50, 10), [], "a touched dot is erased");
assert.equal(
  eraseFromStroke(dot, 200, 200, 200, 200, 10),
  null,
  "an untouched dot is left alone"
);
}

// =====
// F9: Wheel input. deltaMode was never read, so Firefox's LINE units (3 per
// notch) were treated as 3 pixels – the wheel felt inert and pinch-zoom crawled
// at 0.2% per event. Pinch also has to be recognised: browsers deliver a
// trackpad pinch as a wheel event with ctrlKey set.
// =====
console.log("[REG-F9] wheel input: units, pinch, and mouse vs trackpad...");
{
  const ev = (o: Partial<RawWheel>): RawWheel => ({
    deltaX: 0,
    deltaY: 0,
    deltaMode: 0,
  });
}

```

```

    ctrlKey: false,
    metaKey: false,
    shiftKey: false,
    ...0,
  });

  // Line and page units are converted to pixels.
  assert.equal(normalizeWheelDelta(ev({ deltaY: 3, deltaMode: 1 })).dy, 48, "3 lines = 48px");
  assert.equal(normalizeWheelDelta(ev({ deltaY: 1, deltaMode: 2 })).dy, 800, "1 page = 800px");
  assert.equal(normalizeWheelDelta(ev({ deltaY: 100, deltaMode: 0 })).dy, 100, "pixels pass through");

  // A Firefox notch must move the canvas comparably to a Chrome notch, rather
  // than 3px vs 100px.
  const chromeNotch = resolveWheelAction(ev({ deltaY: 100 }), "trackpad");
  const firefoxNotch = resolveWheelAction(ev({ deltaY: 3, deltaMode: 1 }), "trackpad");
  assert.equal(chromeNotch.kind, "pan");
  assert.equal(firefoxNotch.kind, "pan");
  const ratio =
    (firefoxNotch as { dy: number }).dy / (chromeNotch as { dy: number }).dy;
  assert.ok(ratio > 0.3 && ratio < 3, `notch sizes must be comparable, got ratio ${ratio}`);

  // A pinch always zooms, in every mode.
  for (const mode of ["auto", "trackpad", "mouse"] as const) {
    assert.equal(
      resolveWheelAction(ev({ deltaY: -8, ctrlKey: true }), mode).kind,
      "zoom",
      `pinch zooms in ${mode} mode`
    );
  }

  // Zooming is symmetric: in then out returns exactly to the start, or the
  // canvas drifts every time the user scrubs the wheel.
  const zin = resolveWheelAction(ev({ deltaY: -120, ctrlKey: true }), "auto");
  const zout = resolveWheelAction(ev({ deltaY: 120, ctrlKey: true }), "auto");
  const roundTrip =
    (zin as { factor: number }).factor * (zout as { factor: number }).factor;
  assert.ok(Math.abs(roundTrip - 1) < 1e-9, `zoom in then out must return to 1, got ${roundTrip}`);

  // Mode behaviour.
  assert.equal(
    resolveWheelAction(ev({ deltaY: 100 }), "mouse").kind,
    "zoom",
    "in mouse mode the wheel zooms"
  );
  assert.equal(
    resolveWheelAction(ev({ deltaY: 4.2, deltaX: 1.1 }), "trackpad").kind,
    "pan",
    "in trackpad mode two-finger scroll pans"
  );

  // Auto-detection: a quantised vertical-only step is a wheel; small
  // fractional deltas with sideways drift are a trackpad.
  assert.ok(looksLikeMouseWheel(ev({ deltaY: 100 })), "100px vertical step is a wheel");
  assert.ok(!looksLikeMouseWheel(ev({ deltaY: 4.2, deltaX: 1.1 })), "fractional drift is a trackpad");
  assert.ok(looksLikeMouseWheel(ev({ deltaY: 3, deltaMode: 1 })), "line units only come from a wheel");

  // Shift scrolls sideways on a device with only a vertical wheel.
  const sideways = resolveWheelAction(ev({ deltaY: 100, shiftKey: true }), "trackpad");
  assert.equal(sideways.kind, "pan");
  assert.equal((sideways as { dx: number }).dx, 100, "shift moves the vertical delta to X");
  assert.equal((sideways as { dy: number }).dy, 0, "shift leaves no vertical movement");
}

// =====
// F10: Paper size is per board and must survive a reload – and be clamped. A
// size of 0 or NaN would make the SVG pattern tile infinitely small and lock
// the renderer, so a corrupt value must never reach the canvas.
// =====
console.log("[REG-F10] board paper size persists and is clamped...");
{
  const st = useWhiteboard.getState();
  const boardId = st.createBoard("paper-size-board", null);

  useWhiteboard.getState().setBoardBackground(boardId, { pattern: "grid", size: 64 });
  const saved = useWhiteboard.getState().getPersistedState();
  const savedBoard = saved.boards.find((b) => b.id === boardId!);
  assert.equal(savedBoard.background?.size, 64, "paper size is saved");
  assert.equal(savedBoard.background?.pattern, "grid", "pattern is saved alongside it");
}

```

```

// Each board keeps its own paper: setting one board must not leak into
// another. (New boards start with the app default paper, so "own paper"
// now means the untouched default, not undefined.)
const other = useWhiteboard.getState().createBoard("other-paper", null);
const otherSaved = useWhiteboard
  .getState()
  .getPersistedState()
  .boards.find((b) => b.id === other!);
assert.equal(otherSaved.background?.pattern, "plain", "a different board keeps its own paper");
assert.equal(
  otherSaved.background?.size,
  undefined,
  "untouched by the other board's 64px spacing"
);

// Round trip through the loader's validation.
const reloaded = migrateState(saved as never) as typeof saved;
assert.equal(
  reloaded.boards.find((b) => b.id === boardId)!.background?.size,
  64,
  "paper size survives migrate()"
);

// Values that would break the renderer are clamped, not passed through.
const corrupt = migrateState({
  ...(saved as unknown as Record<string, unknown>),
  boards: saved.boards.map((b) =>
    b.id === boardId ? { ...b, background: { pattern: "grid", size: 0 } } : b
  ),
} as never) as typeof saved;
const clamped = corrupt.boards.find((b) => b.id === boardId)!.background?.size;
assert.ok(
  clamped !== undefined && clamped >= 6,
  `a zero paper size must be clamped to a usable value, got ${clamped}`
);

const nan = migrateState({
  ...(saved as unknown as Record<string, unknown>),
  boards: saved.boards.map((b) =>
    b.id === boardId ? { ...b, background: { pattern: "grid", size: Number.NaN } } : b
  ),
} as never) as typeof saved;
assert.equal(
  nan.boards.find((b) => b.id === boardId)!.background?.size,
  undefined,
  "a NaN paper size is dropped so the default applies"
);
}

// =====
// F11 (security): the export preview renders the produced SVG through
// dangerouslySetInnerHTML, so hostile board text must not be able to introduce
// an element or an event handler. innerHTML does NOT execute <script>, but it
// DOES fire handlers such as <img onerror>, so escaping is the whole defence.
// =====
console.log("[REG-F11] export escapes hostile text (no injectable handlers)...");
{
  const payloads = [
    `</text><img src=x onerror="alert(1)">`,
    `"><svg onload="alert(1)">`,
    `</tspan></text><foreignObject><body onload="alert(1)">`,
    `<script>alert(1)</script>`,
  ];
}

const board = makeBoard("xss-board");
board.elements = payloads.map((text, i): TextElement => ({
  id: `x${i}`,
  type: "text",
  x: i * 10,
  y: i * 10,
  width: 200,
  height: 40,
  rotation: 0,
  opacity: 1,
  text,
  color: "#000000",
  fontSize: 16,

```

```

    fontFamily: "sans-serif",
    fontWeight: 400,
    align: "left",
    createdAt: 1,
    updatedAt: 1,
  }));

const svg = exportBoardToSvg(board, { background: "white" });
const tags = [...svg.matchAll(/<([a-zA-Z][^\/>]*)(<\/>)*>/g)];

// Escaped text legitimately CONTAINS the characters "onerror=", so assert on
// parsed tags rather than on substrings anywhere in the document – a naive
// substring check reports a false positive here.
const withHandler = tags.filter((m) => /on[a-z]+\s*=/i.test(m[2]));
assert.equal(withHandler.length, 0, "no exported tag may carry an on* event handler");

const injected = tags.filter((m) => /^(img|script|body|iframe)$/i.test(m[1]));
assert.equal(injected.length, 0, "hostile text must not introduce new elements");

for (const p of payloads) {
  assert.ok(!svg.includes(p), `payload must be escaped, not emitted raw: ${p.slice(0, 24)}`);
}
}

// =====
// F12: element-level graph vertices. A link lives in a specific text box, so
// that box is the vertex – the graph must show WHICH note points at the target,
// and clicking it must be able to reach the object itself.
// =====
console.log("[REG-F12] element vertices, shapes and navigation ids...");
{
  const note = (id: string, text: string): TextElement => ({
    id, type: "text", x: 40, y: 60, width: 200, height: 80,
    rotation: 0, opacity: 1, text, color: "#000", fontSize: 16,
    fontFamily: "sans-serif", fontWeight: 400, align: "left",
    createdAt: 1, updatedAt: 1,
  });

  const source = makeBoard("bSrc");
  source.name = "Source";
  const target = makeBoard("bTgt");
  target.name = "Physics";
  source.elements = [note("n1", "see [[Physics|the notes]] here"), note("n2", "unrelated")];

  const g = buildGraph([source, target], [], []);

  // The vertex is the NOTE, not the board.
  const wl = g.edges.find((e) => e.linkType === "wikilink")!;
  assert.ok(wl, "the wikilink produces an edge");
  assert.equal(wl.source, elementNodeId("bSrc", "n1"), "edge starts at the note that holds the link");
  assert.equal(wl.target, "bTgt", "edge ends at the linked board");
  assert.equal(wl.label, "the notes", "edge is labelled with the link's display alias");

  // That vertex exists, is typed, and knows where it lives.
  const elNode = g.nodes.find((n) => n.id === elementNodeId("bSrc", "n1"))!;
  assert.ok(elNode, "the linking element has its own vertex");
  assert.equal(elNode.kind, "element", "vertex is marked as an element");
  assert.equal(elNode.elementType, "text", "vertex carries its type so it can be drawn as one");
  assert.equal(elNode.boardId, "bSrc", "vertex knows its board");
  assert.equal(elNode.elementId, "n1", "vertex knows its element");
  assert.ok(elNode.name.includes("the notes"), "label shows display text, not raw [[markup]]");
  assert.ok(!elNode.name.includes("[[", "raw wikilink markup never reaches the label");

  // A note with no links must NOT become a vertex, or the graph fills with noise.
  assert.equal(
    g.nodes.find((n) => n.id === elementNodeId("bSrc", "n2")),
    undefined,
    "an element with no links stays out of the graph"
  );

  // Element vertices hang off their board rather than floating unattached.
  assert.ok(
    g.edges.some((e) => e.source === "bSrc" && e.target === elementNodeId("bSrc", "n1")),
    "board owns its element vertex"
  );

  // The id round-trips, which is what makes click-to-navigate possible.
  const parsed = parseElementNodeId(elementNodeId("bSrc", "n1"))!;

```

```

assert.deepEqual(parsed, { boardId: "bSrc", elementId: "n1" }, "element node id round-trips");
assert.equal(parseElementNodeId("bSrc"), null, "a board id is not mistaken for an element id");

// An on-canvas arrow between two shapes now shows as element-to-element,
// where it used to collapse into a self-loop and vanish.
const canvas = makeBoard("bCanvas");
const r1 = makeRect("r1", 0, 0);
const r2 = makeRect("r2", 300, 0);
const arrow: ConnectionElement = {
  id: "c1", type: "connection", x: 0, y: 0, rotation: 0, opacity: 1,
  source: { boardId: "bCanvas", elementId: "r1", anchor: "auto" },
  target: { boardId: "bCanvas", elementId: "r2", anchor: "auto" },
  linkType: "relates", routing: "curved", dashStyle: "solid",
  startArrowhead: "none", endArrowhead: "arrow", strokeWidth: 2,
  createdAt: 1, updatedAt: 1,
};
canvas.elements = [r1, r2, arrow];
const g2 = buildGraph([canvas], [], []);
const connEdge = g2.edges.find((e) => e.id === "conn-c1")!;
assert.ok(connEdge, "same-board shape-to-shape arrow now appears");
assert.equal(connEdge.source, elementNodeId("bCanvas", "r1"), "arrow starts at the source shape");
assert.equal(connEdge.target, elementNodeId("bCanvas", "r2"), "arrow ends at the target shape");
assert.equal(
  g2.nodes.find((n) => n.id === elementNodeId("bCanvas", "r1"))!.elementType,
  "rectangle",
  "shape vertices carry their own type"
);
}

// =====
// F13: focusElement centres the camera ON the object and selects it.
// =====
console.log("[REG-F13] focusElement lands on the object, not the board...");
{
  const st = useWhiteboard.getState();
  const boardId = st.createBoard("focus-target", null);
  const far = makeRect("far-rect", 4000, 2500, 200, 100);
  useWhiteboard.getState().addElementToBoard(boardId, far, false);

  // Stand somewhere else entirely, on another board.
  const other = useWhiteboard.getState().createBoard("elsewhere", null);
  useWhiteboard.getState().setActiveBoard(other);

  useWhiteboard.getState().focusElement(boardId, "far-rect");
  const after = useWhiteboard.getState();

  assert.equal(after.activeBoardId, boardId, "switches to the element's board");
  // The camera is the world point at the viewport centre, so centring means
  // camera === the element's centre.
  assert.equal(after.camera.x, 4100, "camera centred on the element's x");
  assert.equal(after.camera.y, 2550, "camera centred on the element's y");
  assert.deepEqual(after.selectedIds, ["far-rect"], "the element is selected on arrival");
  assert.ok(after.camera.zoom >= 0.6, "never arrives so zoomed out the element is a speck");

  // A missing element must be a no-op, not a jump to (0,0).
  const before = useWhiteboard.getState().camera;
  useWhiteboard.getState().focusElement(boardId, "does-not-exist");
  assert.deepEqual(useWhiteboard.getState().camera, before, "unknown element leaves the camera alone");
}

// =====
// F14: "switch away from my pen and back and my settings reset."
// FOUR separate causes, all from hand-written lists that were not updated when
// the pen set grew from 6 to 8. Each is pinned here.
// =====
console.log("[REG-F14] per-pen ink is independent and survives a round trip...");
{
  // (1) Every pen must have its OWN ink. ballpoint/gel/fineliner/fountain had
  // no case in the accessor switch and all fell through to `strokeColor`,
  // sharing one colour and size with each other AND with shape borders.
  const base = useWhiteboard.getState().style;
  let style = { ...base };
  const paint = (tool: ToolType, color: string, size: number) => {
    style = { ...style, ...setToolInk(style, tool, { color, size }) };
  };

  paint("ballpoint" as ToolType, "#111111", 3);
  paint("gel" as ToolType, "#222222", 5);
}

```

```

paint("fineliner" as ToolType, "#333333", 2);
paint("fountain" as ToolType, "#444444", 7);
paint("highlighter" as ToolType, "ffff00", 24);

assert.equal(getToolColor(style, "ballpoint" as ToolType), "#111111", "ballpoint keeps its own colour");
assert.equal(getToolColor(style, "gel" as ToolType), "#222222", "gel keeps its own colour");
assert.equal(getToolColor(style, "fineliner" as ToolType), "#333333", "fineliner keeps its own colour");
assert.equal(getToolColor(style, "fountain" as ToolType), "#444444", "fountain keeps its own colour");
assert.equal(getToolSize(style, "fountain" as ToolType), 7, "fountain keeps its own size");
assert.equal(getToolSize(style, "fineliner" as ToolType), 2, "fineliner keeps its own size");

// A pen must never write through to the SHAPE border, which is what the
// fall-through did.
assert.notEqual(style.strokeColor, "#444444", "pen ink does not leak into shape borders");

// Every pen offered in the picker must be independently addressable – this is
// the assertion that fails the moment someone adds a pen and forgets the map.
const inks = new Set(PEN_TYPES.map((t) => getToolColor(style, t as ToolType) + "/" + getToolSize(style, t as
  ToolType)));
assert.equal(inks.size >= 6, true, "the pens do not collapse onto shared ink");

// (2) setToolInk writes colour and size in ONE patch. Two separate patches
// spread together would both carry `toolInk` and the second would win,
// silently dropping the colour.
const both = setToolInk(style, "brush" as ToolType, { color: "#abcdef", size: 19 });
const merged = { ...style, ...both };
assert.equal(getToolColor(merged, "brush" as ToolType), "#abcdef", "combined patch keeps the colour");
assert.equal(getToolSize(merged, "brush" as ToolType), 19, "combined patch keeps the size");

// (3) A saved payload must survive migrate(). The tool whitelist was
// hand-written and still named only the pre-rework pens, so a saved
// `tool: "fountain"` was rejected and the pen reset on every reload.
const savedFountain = migrateState({
  version: 1, folders: [], boards: [], activeBoardId: null, activeFolderId: null,
  tool: "fountain" as ToolType,
}) as Parameters<typeof migrateState>[0];
assert.equal(savedFountain.tool, "fountain", "a saved new-generation pen survives a reload");

// (4) A pinned pen of a new generation must survive too – the pin whitelist
// had the same stale list, so pinning a gel pen DELETED it on reload.
const savedPins = migrateState({
  version: 1, folders: [], boards: [], activeBoardId: null, activeFolderId: null,
  pinnedPens: [
    { id: "p1", tool: "gel", color: "#123456", size: 4 },
    { id: "p2", tool: "fountain", color: "#654321", size: 6 },
    { id: "p3", tool: "not-a-pen", color: "#000000", size: 4 },
  ],
}) as unknown as Parameters<typeof migrateState>[0];
const keptTools = (savedPins.pinnedPens ?? []).map((p) => p.tool);
assert.deepEqual(keptTools, ["gel", "fountain"], "new-generation pins survive; junk is still rejected");
}

// =====
// F15: reaching for "the pen" returns the pen you were last holding.
// =====
console.log("[REG-F15] the pen button restores your pen, not a default...");
{
  useWhiteboard.getState().setTool("fountain" as ToolType);
  assert.equal(useWhiteboard.getState().lastStrokeTool, "fountain", "the pen in hand is remembered");

  useWhiteboard.getState().setTool("eraser");
  assert.equal(
    useWhiteboard.getState().lastStrokeTool,
    "fountain",
    "a non-drawing tool does not overwrite the remembered pen"
  );

  useWhiteboard.getState().setTool("select");
  useWhiteboard.getState().setTool(useWhiteboard.getState().lastStrokeTool);
  assert.equal(useWhiteboard.getState().tool, "fountain", "returning to the pen returns to FOUNTAIN");
}

// =====
// F16: a hand resting on the glass must not select, move or resize anything.
//
// The reported symptom: a palm coming down on a line of already-typed text
// opened the transform box mid-sentence. The canvas only suppressed touch
// while the nib was actually ON the glass, which misses both ways a palm

```

```
// really arrives – it lands BEFORE you start writing, and it stays down while
// you lift the pen between words.
// =====
console.log("[REG-F16] palm rejection and deliberate touch...");
{
  const finger = { width: 20, height: 22 };
  const palmEdge = { width: 30, height: 70 };
  const unknown = { width: 1, height: 1 };
  const T = 100_000; // an arbitrary "now"

  // -- the pen is decisive -----
  assert.equal(
    classifyTouch(finger, { penDown: true, lastPenAt: T, now: T }),
    "pen-down",
    "a touch while the nib is down is a palm"
  );

  // THE BUG: the pen lifts between words and the hand stays put. The old guard
  // went false the instant the nib left the glass, so every palm contact in
  // that gap reached the select tool and grabbed whatever it rested on.
  assert.equal(
    classifyTouch(finger, { penDown: false, lastPenAt: T - 200, now: T }),
    "pen-recent",
    "a touch 200ms after the pen lifted is still the same resting hand"
  );
  assert.equal(
    classifyTouch(finger, { penDown: false, lastPenAt: T - (PEN_SUPPRESSION_MS - 1), now: T }),
    "pen-recent",
    "suppression holds right up to the edge of the window"
  );
  assert.equal(
    classifyTouch(finger, { penDown: false, lastPenAt: T - PEN_SUPPRESSION_MS, now: T }),
    "allow",
    "and releases exactly at the window, so the pen never locks touch out"
  );
};

// -- no stylus, no cost -----
// This is what lets palm rejection be always-on rather than a setting nobody
// finds: a device that has never seen a pen can never open the window.
assert.equal(
  classifyTouch(finger, { penDown: false, lastPenAt: 0, now: T }),
  "allow",
  "a pure-touch device is completely unaffected by the pen rules"
);

// -- contact geometry -----
assert.equal(contactLooksLikePalm(palmEdge), true, "the edge of a hand is a palm");
assert.equal(contactLooksLikePalm(finger), false, "a fingertip is not");
assert.equal(
  contactLooksLikePalm(unknown),
  false,
  "1x1 is the spec default for 'cannot measure' – abstain, do not call it a fingertip"
);
assert.equal(contactLooksLikePalm({}), false, "a browser reporting nothing must not reject");
assert.equal(
  contactLooksLikePalm({ width: 8, height: PALM_CONTACT_PX }),
  true,
  "a palm reports a long thin ellipse, so the LARGER dimension decides"
);
assert.equal(
  classifyTouch(palmEdge, { penDown: false, lastPenAt: 0, now: T }),
  "contact-size",
  "size alone still catches a palm on a stylus-free tablet"
);
);

assert.equal(isRejectedTouch(finger, { penDown: false, lastPenAt: 0, now: T }), false);
assert.equal(isRejectedTouch(finger, { penDown: true, lastPenAt: T, now: T }), true);

// -- long press: hold selects, move pans -----
assert.equal(exceedsLongPressSlop(0, 0), false, "a perfectly still finger is a press");
assert.equal(
  exceedsLongPressSlop(4, 4),
  false,
  "fingers always wobble; a few pixels is still a press"
);
);
assert.equal(
  exceedsLongPressSlop(LONG_PRESS_SLOP_PX + 1, 0),
  true,

```

```

    "past the slop the user is panning and meant to grab nothing"
  );
  assert.equal(
    exceedsLongPressSlop(-20, 15),
    true,
    "slop is a radius, so direction cannot smuggle a drag past it"
  );
}

console.log("\n=== ALL REGRESSION TESTS PASSED (fix pass pinned) ===");

=====
// === END FILE: tests/regression_fix_pass.test.ts ===
=====

=====
// === START FILE: tests/review_fixes.test.ts ===
// File 175 of 194 | Lines: 179 | Size: 7037 bytes
=====

/**
 * Regressions from the review of the text-tool / toolbar work:
 * - Dictation: back-to-back phrases are all kept; only a true echo is dropped
 * - Export: the text a PDF shows matches the canvas, not the raw markup
 * - Inline markup: prices, arithmetic and issue numbers stay plain text
 * - Sticky colour: reading a colour and writing it back keeps it
 */

import assert from "node:assert/strict";
import { VoiceDictationController, isRepeatedFinal } from "../src/lib/whiteboard/speechRecognition";
import { markdownToPlainLines, tokenizeInline } from "../src/lib/whiteboard/textParser";
import { DEFAULT_STYLE, STICKY_COLORS, getToolColor, setToolInk, stickyColorKey } from "../src/lib/whiteboard/types";

const NL = String.fromCharCode(10);

let passed = 0;
let failed = 0;
async function test(name: string, fn: () => void | Promise<void>) {
  try {
    await fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(` ${err as Error}.message`);
  }
}

// -----
// A stand-in speech engine: the test plays the results Android delivers.
// -----

type Handler = ((ev: unknown) => void) | null;
class FakeRecognition {
  static last: FakeRecognition | null = null;
  continuous = false;
  interimResults = false;
  lang = "";
  maxAlternatives = 1;
  onstart: Handler = null;
  onresult: Handler = null;
  onerror: Handler = null;
  onend: Handler = null;
  onaudiostart: Handler = null;
  onaudioend: Handler = null;
  onsoundstart: Handler = null;
  onsoundend: Handler = null;
  onspeechstart: Handler = null;
  onspeechend: Handler = null;
  onnomatch: Handler = null;
  constructor() {
    FakeRecognition.last = this;
  }
  start() {
    this.onstart?.({});
  }

```

```

    }
    stop() {}
    abort() {}
    addEventListener() {}
    removeEventListener() {}
    dispatchEvent() {
        return true;
    }
}
/** One onresult carrying final phrases only - how Android sends most. */
finalOnly(text: string, confidence = 0) {
    const alt = { transcript: text, confidence };
    const result = Object.assign([alt], { isFinal: true, item: () => alt });
    const results = Object.assign([result], { item: () => result });
    this.onresult?.({ resultIndex: 0, results });
}
}

async function startedController(finals: string[]) {
    (globalThis as unknown as { window: unknown }).window = { webkitSpeechRecognition: FakeRecognition };
    const c = new VoiceDictationController({ onFinal: (t) => finals.push(t) });
    const ok = await c.start();
    assert.equal(ok, true, "controller should start on the fake engine");
    return FakeRecognition.last!;
}

async function main() {
    console.log(NL + "■■■ Review fixes ■■■");

    await test("dictation keeps two different phrases that arrive back-to-back", async () => {
        const finals: string[] = [];
        const rec = await startedController(finals);
        rec.finalOnly("first sentence");
        rec.finalOnly("second sentence");
        rec.finalOnly("third sentence");
        // The old guard dropped every second final-only result: 2 of 3 survived.
        assert.deepEqual(finals, ["first sentence", "second sentence", "third sentence"]);
    });

    await test("dictation keeps phrases Android reports with confidence 0", async () => {
        const finals: string[] = [];
        const rec = await startedController(finals);
        rec.finalOnly("hello there", 0);
        assert.deepEqual(finals, ["hello there"]);
    });

    await test("dictation drops the engine echoing the same phrase", async () => {
        const finals: string[] = [];
        const rec = await startedController(finals);
        rec.finalOnly("buy milk");
        rec.finalOnly("Buy milk ");
        assert.deepEqual(finals, ["buy milk"]);
    });

    await test("isRepeatedFinal: same words later on are a new phrase", () => {
        assert.equal(isRepeatedFinal("yes", 1000, "yes", 1500), true);
        assert.equal(isRepeatedFinal("yes", 1000, "yes", 9000), false);
        assert.equal(isRepeatedFinal("yes", 1000, "no", 1500), false);
        assert.equal(isRepeatedFinal("", 1000, "", 1500), false);
    });

    await test("export text: markers become what the canvas shows", () => {
        const text = ["# Title", "- [ ] Buy **milk**", "- [x] Done", "- item", "3. third", "> quoted", "plain [[Bo
ard|alias]] text"].join(NL);
        const lines = markdownToPlainLines(text);
        assert.deepEqual(
            lines.map((l) => l.text),
            ["Title", "■ Buy milk", "■ Done", "• item", "3. third", "■ quoted", "plain alias text"]
        );
        assert.ok((lines[0].scale > 1 && (lines[0].weight ?? 0) >= 600), "a heading is bigger and bold");
        assert.equal(lines[1].scale, 1);
    });

    await test("export text: blank lines and code blocks keep their lines", () => {
        const fence = "```";
        const text = ["a", "", fence, "x = 1", "y = 2", fence, "b"].join(NL);
        assert.deepEqual(
            markdownToPlainLines(text).map((l) => l.text),
            ["a", "", "x = 1", "y = 2", "b"]
        );
    });
}

```

```

    });
  });

  await test("prices are not maths, but real maths still is", () => {
    const kinds = (s: string) => tokenizeInline(s).map((t) => t.type);
    assert.ok(!kinds("costs $5 and $10 today").includes("math"));
    assert.ok(kinds("area is  $x^2$  here").includes("math"));
    assert.ok(kinds("$E = mc^2$").includes("math"));
  });

  await test("arithmetic is not italic, but *emphasis* still is", () => {
    const kinds = (s: string) => tokenizeInline(s).map((t) => t.type);
    assert.ok(!kinds("2*3*4 = 24").includes("italic"));
    assert.ok(kinds("this is *important* ok").includes("italic"));
    assert.ok(kinds("***bold** word").includes("bold"));
  });

  await test("issue numbers are not tags, but words (Hebrew too) are", () => {
    const tags = (s: string) => tokenizeInline(s).filter((t) => t.type === "tag").map((t) => t.value);
    assert.deepEqual(tags("see issue #42"), []);
    assert.deepEqual(tags("todo #urgent now"), ["urgent"]);
    assert.deepEqual(tags("#■■■■■■■■"), ["■■■■■■■■"]);
    assert.deepEqual(tags("page.html#section"), []);
  });

  await test("sticky colour survives a read-then-write round trip", () => {
    for (const key of Object.keys(STICKY_COLORS) as (keyof typeof STICKY_COLORS)[]) {
      const style = { ...DEFAULT_STYLE, stickyColor: key };
      const read = getToolColor(style, "sticky");
      const written = setToolInk(style, "sticky", { color: read });
      assert.equal(written.stickyColor, key, `${key} came back as ${written.stickyColor}`);
    }
  });

  await test("an unknown colour leaves the sticky colour alone", () => {
    const style = { ...DEFAULT_STYLE, stickyColor: "pink" as const };
    assert.equal(setToolInk(style, "sticky", { color: "#123456" }).stickyColor, "pink");
    assert.equal(stickyColorKey("#FCE7F3"), "pink");
    assert.equal(stickyColorKey("nonsense"), null);
  });

  console.log(` ${passed} passed, ${failed} failed`);
  if (failed > 0) process.exit(1);
}

void main();

=====
// === END FILE: tests/review_fixes.test.ts ===
=====

=====
// === START FILE: tests/search_scopes.test.ts ===
// File 176 of 194 | Lines: 260 | Size: 11771 bytes
=====

/**
 * Search scopes and nearest-first ranking (lib/whiteboard/search.ts).
 * - "This note" finds only the open note's content, in page order.
 * - "This folder" covers the folder and its sub-folders, not the rest.
 * - "Everywhere" puts this note, then its folder, then the folders around
 *   it, then everything else; within a group better matches come first.
 * - Markdown, case and Hebrew niqqud do not stop a match; handwriting,
 *   shapes and pictures are never results.
 */

import assert from "node:assert/strict";
import {
  buildSearchIndex,
  foldText,
  groupHits,
  highlightParts,
  makeSnippet,
  matchEntry,
  pdfHit,
  plainText,
  scopeInfo,

```